

2026 데이터+AI 혁신 챌린지

데이터+AI 크리에이터 캠프

대학부



과학기술정보통신부

NIA

지능정보원
한국지능정보사회진흥원

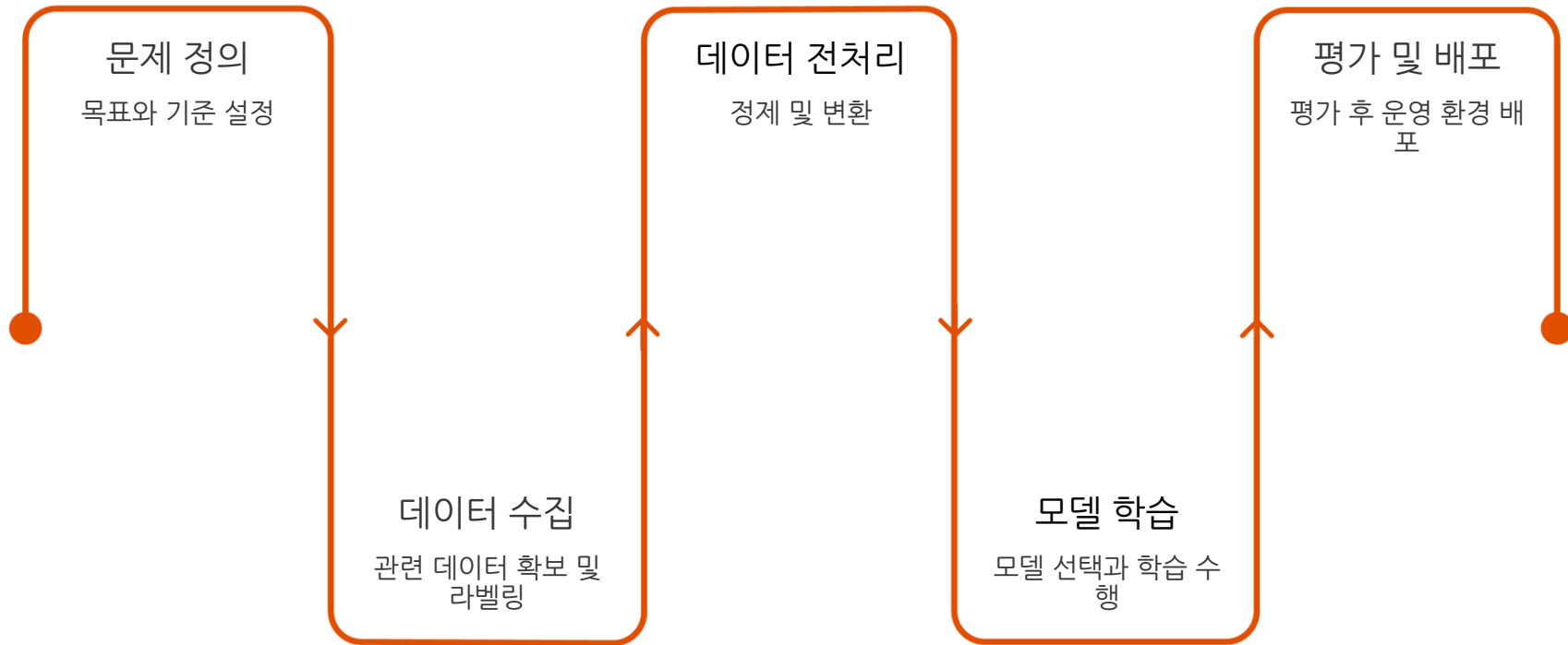




데이터 및 AI 개요

AI가 어떻게 동작하는지와 데이터가 왜 핵심인지 이해함. 인공지능의 흐름과 현대 딥러닝의 배경을 살펴보고 캠프 전체의 AI 파이프라인을 개괄함.

AI 파이프라인 로드맵



AI 프로젝트는 이 다섯 단계를 거침. 이번 캠프는 앞의 세 단계인 문제 정의, 데이터 수집, 전처리에 집중함. 이후 모델 학습과 배포의 기반을 다지는 구성이며, 단계별 핵심 도구와 기술을 함께 익힘.

① 문제 정의 — 무엇을 풀 것인가?

AI 프로젝트의 성패는 문제를 얼마나 잘 정의하느냐에 달려 있다.

핵심 질문 체크리스트

- 어떤 결과를 예측/분류/생성할 것인가?
- 어떤 데이터를 사용할 수 있는가?
- 정확도 몇 %면 실용적인가?
- 실패했을 때 비용은 얼마인가?



✗ 나쁜 예: "AI로 고객 만족도를 높이고 싶다"



✓ 좋은 예: "최근 3개월 구매 이력 기반으로 이탈 가능 고객을 예측해 이 탈률을 15% 줄인다"



② 데이터 수집 — 어디서, 어떻게 모을까?

모델의 품질은 데이터의 품질을 절대 넘을 수 없다. Garbage In, Garbage Out.

내부 데이터

- DB, 로그, CRM, ERP 시스템
- 가장 신뢰도 높고 도메인 특화된

공공 데이터

- AI HUB, 공공데이터포털, 통계청
- 무료, 라벨링 완료된 경우 많음

웹 크롤링

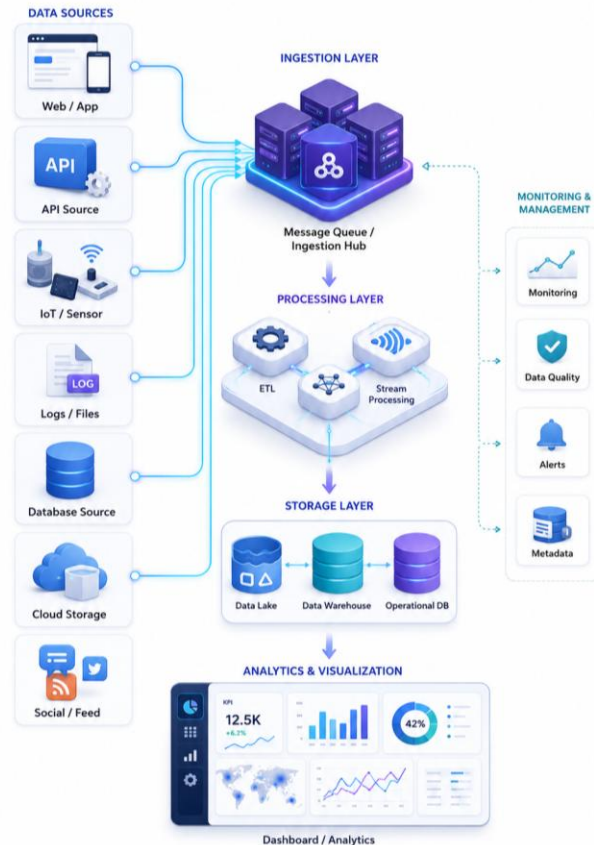
- BeautifulSoup, Selenium 활용
- 저작권·robots.txt 반드시 확인

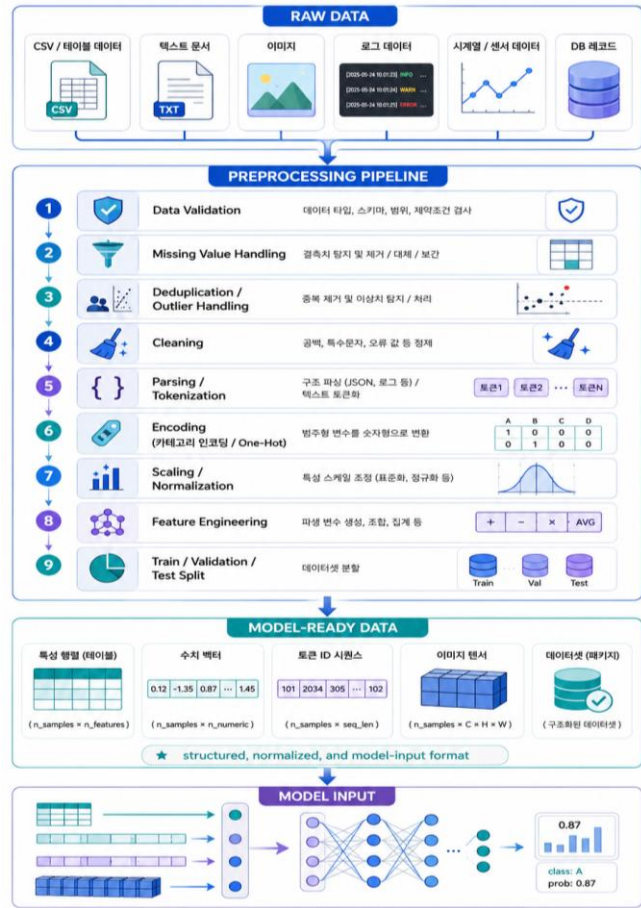
직접 수집 / 라벨링

- 설문, 실험, 어노테이션 툴 활용
- 비용이 크지만 맞춤 데이터 확보 가능

데이터 수집 시 체크포인트

- 충분한 양인가? (최소 수백~수천 건 이상)
- 편향(Bias)은 없는가?
- 라벨(정답)이 있는가?
- 개인정보·저작권 문제는 없는가?





③ 데이터 전처리 — 모델이 먹을 수 있는 형태로

실제 AI 프로젝트에서 전처리에 드는 시간은 전체의 60~80%를 차지한다.

01

결측치 처리

평균/중앙값 대체 또는 행 제거

02

이상치 제거

IQR, Z-Score 기반 탐지

03

인코딩

범주형 → 수치형 변환 (One-Hot, Label)

04

정규화/표준화

스케일 통일 (MinMax, StandardScaler)

05

피쳐 선택

중요 변수만 추려 차원 축소

주요 도구

- **pandas**: 데이터 조작 전반
- **scikit-learn**: 전처리 파이프라인
- **numpy**: 수치 연산

`fit()`
`transform()`

④ 모델 학습 — 데이터에서 패턴을 찾다

모델은 데이터 속 패턴을 수식으로 압축한다. 학습이란 그 수식의 파라미터를 최적화하는 과정이다.

모델 유형 선택 가이드

지도학습 (Supervised)

분류: 로지스틱 회귀, 랜덤 포레스트, XGBoost, 신경망

회귀: 선형 회귀, SVR, 딥러닝

비지도학습 (Unsupervised)

클러스터링: K-Means, DBSCAN

차원 축소: PCA, t-SNE

딥러닝 (Deep Learning)

CNN: 이미지 분류

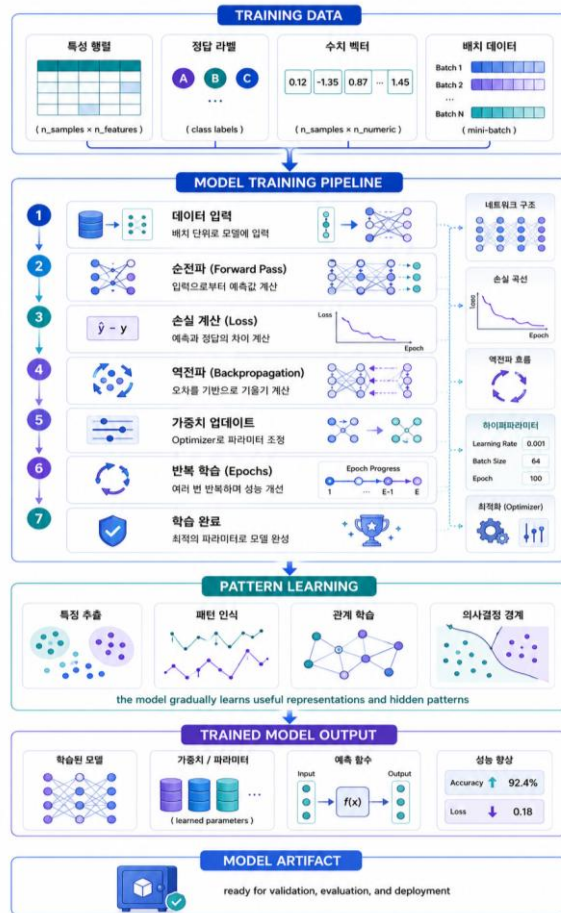
RNN/LSTM: 시계열, 텍스트

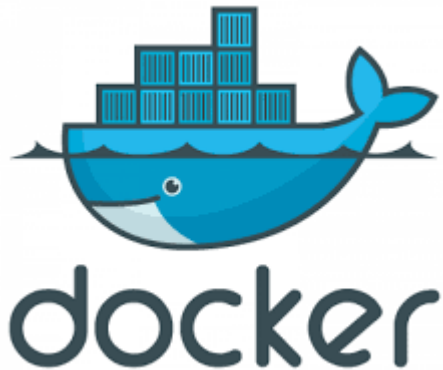
Transformer: 자연어 처리



과적합 vs 과소적합

- **과적합 (Overfitting):** 훈련 데이터에만 잘 맞고 새 데이터에 실패 → 드롭아웃, 정규화로 해결
- **과소적합 (Underfitting):** 모델이 너무 단순해 패턴을 못 잡음 → 모델 복잡도 증가





⑤ 평가 및 배포 — 모델을 세상에 내보내다

좋은 모델은 실험실에서 끝나지 않는다. 실제 환경에서 작동해야 비로소 가치를 만든다.

평가 지표 (Metrics)

정확도 (Accuracy)

전체 중 맞춘 비율. 클래스 불균형 시 주의

정밀도 / 재현율 (Precision / Recall)

스팸 탐지, 의료 진단 등 비대칭 문제에 핵심

F1-Score

정밀도와 재현율의 조화 평균

AUC-ROC

이진 분류 모델의 전반적 성능 측정

배포 후 모니터링 중요성

- **데이터 드리프트(Data Drift):** 실제 데이터 분포가 학습 데이터와 달라지는 현상
- 모델 성능 저하 감지 → 재학습(Retraining) 주기 설정
- A/B 테스트로 신규 모델 점진적 적용

데이터 기반 문제 정의

✗ 잘못된 문제 정의

"AI로 매출을 올리고 싶다"

측정 불가, 목표 불명확, 데이터 연결 없음

✓ 올바른 문제 정의

"고객 이탈률을 30일 이내에 15% 이상 줄이기 위해, 최근 6개월 구매 이력 데이터를 기반으로 이탈 가능 고객을 예측한다"

- 측정 지표 명확 (이탈률 15%)
- 데이터 소스 특정 (구매 이력)
- 기간 한정 (30일, 6개월)

가치 창출 프로세스: 데이터 → 인사이트 → 행동



Raw Data

수집된 원시 데이터



Information

정제된 데이터와 패턴



Knowledge

모델·통계 기반 인사이트

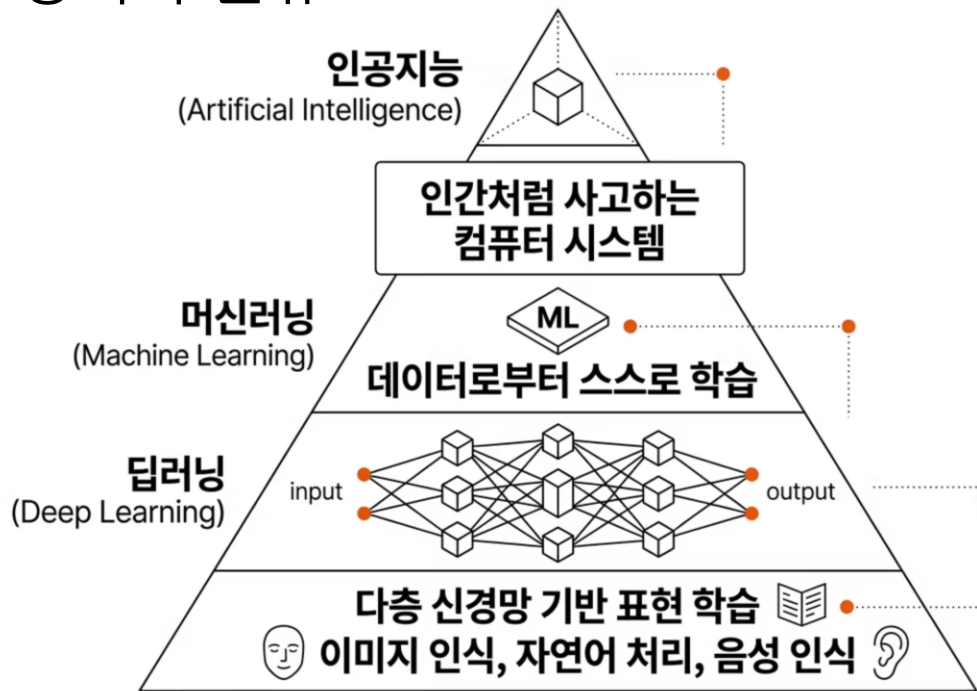


Action

의사결정 및 개선 실행

데이터 자체는 가치가 없음. **가공과 해석 과정**이 가치를 만들며, AI는 이를 자동화·가속화함.

AI의 현대적 정의와 분류



딥러닝(Deep Learning)은 머신러닝의 특수한 형태로, **사람의 뇌 신경망 구조에서 영감을** 받은 다층 구조를 사용함. 오늘 캠프에서 준비하는 데이터는 이 딥러닝 모델의 입력이 됨.

인공지능의 역사 ① — AI의 탄생 (1950년대 · 1960~70년대)

AI의 씨앗은 1950년대에 뿌려졌다. 초기의 낙관은 곧 첫 번째 겨울을 맞이했다.



1950년대 — AI의 탄생

- 앨런 튜링, "Computing Machinery and Intelligence" 논문 발표 (1950)
- 튜링 테스트: 기계가 인간처럼 대화할 수 있는지 판별하는 기준 제시
- 다트머스 회의(1956, 존 매카시 주도): "Artificial Intelligence" 용어 최초 공식 사용
- Logic Theorist(1956): 수학 정리를 자동 증명한 최초의 AI 프로그램
- 대표 인물: 앨런 튜링, 존 매카시, 마빈 민스키, 클로드 새너

1960~70년대 — 낙관과 첫 번째 AI 겨울

- ELIZA 챗봇(1966, MIT): 최초의 자연어 처리 프로그램 — 심리 상담사 역할 모방
- SHAKEY 로봇(1969, SRI): 최초의 자율 이동 로봇 — 환경 인식 후 경로 계획
- 퍼셉트론 한계 증명(Minsky & Papert, 1969): XOR 문제 해결 불가 → 연구 자금 급감
- 첫 번째 AI 겨울(1974~1980): 영국 Lighthill 보고서로 정부 지원 대폭 삭감
- "AI는 불가능하다"는 비관론 확산 — 연구자들 대거 이탈

인공지능의 역사 ② — 전문가 시스템과 규칙 기반 AI (1980년대 · 1997)

기업들이 AI를 처음 도입한 시대. 규칙 기반 AI의 전성기와 한계가 동시에 드러났다.



🌀 1980년대 — 전문가 시스템 붐

- 규칙 기반 전문가 시스템(Expert System): "IF-THEN" 규칙으로 전문가 지식을 코드화
- XCON(DEC사, 1980): VAX 컴퓨터 구성 자동화 → 연간 \$4천만 절감, 정확도 95%
- MYCIN(스탠퍼드): 혈액 감염 진단 AI — 전문의 수준의 정확도 달성
- 일본 제5세대 컴퓨터 프로젝트(1982): 국가 주도 AI 개발 — 서방 국가 긴장
- 두 번째 AI 겨울(1987~1993): 하드웨어 한계·유지비 급증·기대 미달로 재침체

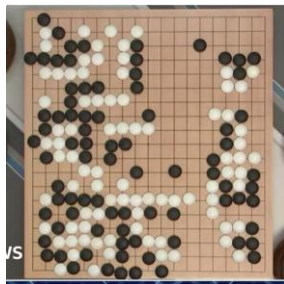


♟️ 1997 — 딥블루 vs 카스파로프

- IBM 딥블루, 세계 체스 챔피언 가리 카스파로프 격파 (3.5:2.5, 6게임)
- 초당 2억 개 체스 포지션 계산 — 전용 하드웨어 + 체스 전문가 지식 결합
- 1996년 1차전에서는 카스파로프가 승리(4:2) → 1년 후 재대결에서 역전
- "기계가 인간 지능을 넘을 수 있다"는 인식 전환 — 전 세계 1억 명 시청
- 규칙 기반 AI의 정점이자 한계: 체스 외 범용 지능은 여전히 불가능

인공지능의 역사 ③ — 딥러닝 혁명 (2012 · 2016~2017)

GPU와 빅데이터가 만나 딥러닝이 폭발했다. 알파고는 AI가 인간의 직관 영역까지 넘어섰음을 증명했다.



2012 — 딥러닝 혁명

- AlexNet(제프리 힌튼 팀), ImageNet 대회 오류율 26% → 15.3%로 대폭 감소
- 2위와의 격차 10%p 이상 — 딥러닝이 기존 방법론을 압도적으로 능가
- NVIDIA GPU 병렬 연산 활용 → 대규모 신경망 학습 비용 혁신적 절감
- 제프리 힌튼, 얀 르쿤, 요슈아 벤지오 — "딥러닝 3인방" (2018 ACM 튜링상 수상)
- 컴퓨터 비전·음성 인식·자연어 처리 전 분야 AI 성능 급등

★ 2016~2017 — 알파고 & Transformer

- 알파고(DeepMind, 2016): 이세돌 9단 4:1 격파 — "바둑은 AI가 못 이긴다"는 통념 붕괴
- 강화학습 + 딥러닝 결합 — 인간의 직관적 판단 영역까지 AI가 진입
- AlphaGo Zero(2017): 인간 기보 없이 자기 대국만으로 3일 만에 알파고 초월
- Transformer 아키텍처 등장(2017, Google, "Attention is All You Need")
- 자율주행·의료 영상 진단·실시간 번역 AI 상용화 본격 시작

인공지능의 역사 ④ — 언어 모델과 생성형 AI (2018~현재)

GPT와 ChatGPT가 AI를 모든 사람의 손에 쥐어줬다. 우리는 지금 그 역사의 한가운데 있다.

4. BERT의 등장



2018~2021 — 언어 모델 혁신

- BERT(Google, 2018): 양방향 문맥 이해 — 검색·번역 품질 혁신
- GPT-2(2019): 너무 강력해 OpenAI가 공개를 일시 보류한 언어 모델
- GPT-3(2020): 1,750억 파라미터 — 코드 작성·소설·번역 등 범용 텍스트 생성
- DALL-E(2021): 텍스트 설명만으로 이미지 생성 — AI 창작 시대 개막
- Codex(2021): GitHub Copilot의 기반 — AI가 코드를 자동 완성



2022~현재 — 생성형 AI 대중화

- ChatGPT(2022.11): 출시 5일 만에 100만 사용자, 2개월 만에 1억 명 — 역대 최단 기록
- GPT-4(2023): 변호사·의사 시험 상위 10% 수준 통과 — 전문직 영역 진입
- Gemini(Google), Claude(Anthropic), LLaMA(Meta) 등 LLM 경쟁 가속
- 이미지(Midjourney, Stable Diffusion), 영상(Sora, Veo) 생성 AI 대중화
- 기업 AI 도입 본격화: LG 챗엑스원(5만 명), MS Copilot, 롯데 Tech Assistant

AI는 지금 이 순간에도 진화 중 — 우리는 그 역사의 현재를 살고 있음.



글로벌 기업 AI 실제 적용 사례



Netflix — 추천 엔진

시청 콘텐츠의 **80%**가 AI 추천에서 발생 (검색 아님)
3단계 AI 파이프라인: 후보 생성(수만 개 → 1만 개) → 필터링
랭킹 → 다양성·비즈니스 제약 재랭킹

피처 스토어에서 시청 이력·시간대·디바이스 등 실시간 피처
통합

출처: Netflix TechBlog, "Foundation Model for
Personalized Recommend



Google — ML 시스템의 현실

실제 프로덕션 AI 시스템에서 **모델 코드는 단 5%**
나머지 **95%**는 데이터 파이프라인, 피처 스토어, 서빙 인프라,
모니터링

"모델은 쉬운 부분, 시스템이 진짜 엔지니어링"

출처: Google, "Hidden Technical Debt in Machine
Learning Systems" (NIPS 논문); dev.to/matt_frank_usa,
2026

KR 국내 기업 AI 실제 적용 사례

LG — 챗엑사원

- 자체 개발 EXAONE 4.0 기반 사내 AI
- 임직원 **5만 명** 1년 내부 검증 후 2025년 공개
- 특허·코드·수학 등 6,000만 건 전문 데이터 학습
- 멀티에이전트 구조로 복잡한 업무 자동화

출처: *blog.dfinite.ai*, "대기업 AI 활용 사례", 2025

롯데쇼핑 — AI 운영 어시스턴트

- Amazon Bedrock 기반 "Tech Assistant" 구축
- 3,900만 명 회원, 연 8조 원 거래액 플랫폼 운영
- 장애 대응 시간 30분 → 대폭 단축
- 분산된 운영 지식을 AI로 통합·검색 가능

출처: *AWS 공식 기술 블로그*, 2025

한샘 — AI 고객 서비스

- AI 에이전트 도입으로 24시간 무중단 고객 응대
- 문제 해결률 **2배** 증가 (48% → 86%)
- 상담원 인계율 **50% 감소**
- 반복 문의 응답 정확도 90% 이상

출처: *delight.ai*, "한샘 고객 서비스 사례", 2025

정보의 데이터화

비정형 정보(텍스트·이미지·소리·영상)를 AI가 학습 가능한 숫자 데이터로 변환하는 차시임.
피쳐 엔지니어링 기초와 데이터셋 구축 핵심 용어를 정리함.

비정형 정보 → 정형 데이터

비정형 데이터

-  텍스트
-  이미지
-  음성
-  영상

정형화 전략

- 텍스트 → 수치 벡터
- 이미지 → 픽셀 텐서
- 음성 → 특징 벡터
- 영상 → 프레임 시퀀스

AI 모델은 **숫자 행렬**만 입력으로 받음. 모든 데이터는 결국 벡터로 표현되며, 이 벡터들의 집합이 행렬이 됨. 따라서 데이터 분석은 벡터화된 정보를 해석하는 과정임.

데이터 타입별 분류

수치형

키, 몸무게, 온도

범주형

성별, 혈액형, 학점

시계열

시간 순서가 중요한 데이터임

텍스트 / 이미지

자연어, 픽셀 데이터. 전처리 필요함

데이터 타입을 정확히 파악해야 올바른 전처리 방법과 모델을 선택할 수 있음.



내 목소리 → 이진 데이터 → 파형 데이터

01

① 아날로그 음성

공기 진동(압력파). 연속적인 파동 신호. 예: '안녕하세요' 발화

02

② 샘플링 & 양자화

초당 44,100번 측정 (44.1kHz). 각 측정값을 정수로 반올림 (양자화)

03

③ 이진 데이터 (PCM)

각 샘플 → 16비트 이진수. 예: 0.73 → 0101111001001101. 파일: .wav, .pcm
이진값을 시간축에 플롯. 진폭(amplitude) vs 시간 그래프. AI 입력: 스펙트로그램 변환

04

④ 파형 데이터 시각화

핵심 개념

- 샘플링 레이트: 초당 샘플 수 (Hz)
- 비트 깊이: 샘플당 비트 수 (16bit = 65,536단계)
- PCM: Pulse Code Modulation (무압축 이진 오디오)

AI는 파형을 어떻게 쓰나?

- 파형 → 푸리에 변환 → 주파수 스펙트럼
- 스펙트럼 → 스펙트로그램 (2D 이미지)
- 스펙트로그램 → CNN/RNN 입력
- 예: 음성 인식(STT), 화자 식별



내가 쓴 글 → 토큰 → 벡터 데이터

내가 키보드로 입력한 문장은 NLP 파이프라인을 거쳐 컴퓨터가 이해할 수 있는 숫자 벡터로 변환된다. 예시 문장: '나는 AI 캠프에서 공부한다'

01

① 원시 텍스트

사람이 입력한 문자열. 예: '나는 AI 캠프에서 공부한다' — 컴퓨터는 아직 의미를 모름

02

② 토큰나이징

문장을 의미 단위(토큰)로 분리. ['나는', 'AI', '캠프', '에서', '공부', '한다'] → 각 토큰에 정수 ID 부여

03

③ 정수 인코딩 (Vocab)

단어장(Vocabulary) 기반 매핑. '나는' → 42, 'AI' → 317, '캠프' → 891 ... 희소 벡터 (One-Hot) 또는 정수 시퀀스

04

④ 임베딩 벡터

각 토큰 → 밀집 벡터(Dense Vector). 예: 'AI' → [0.21, -0.83, 0.47, ...] (300차원). 의미적으로 유사한 단어는 벡터 공간에서 가까움

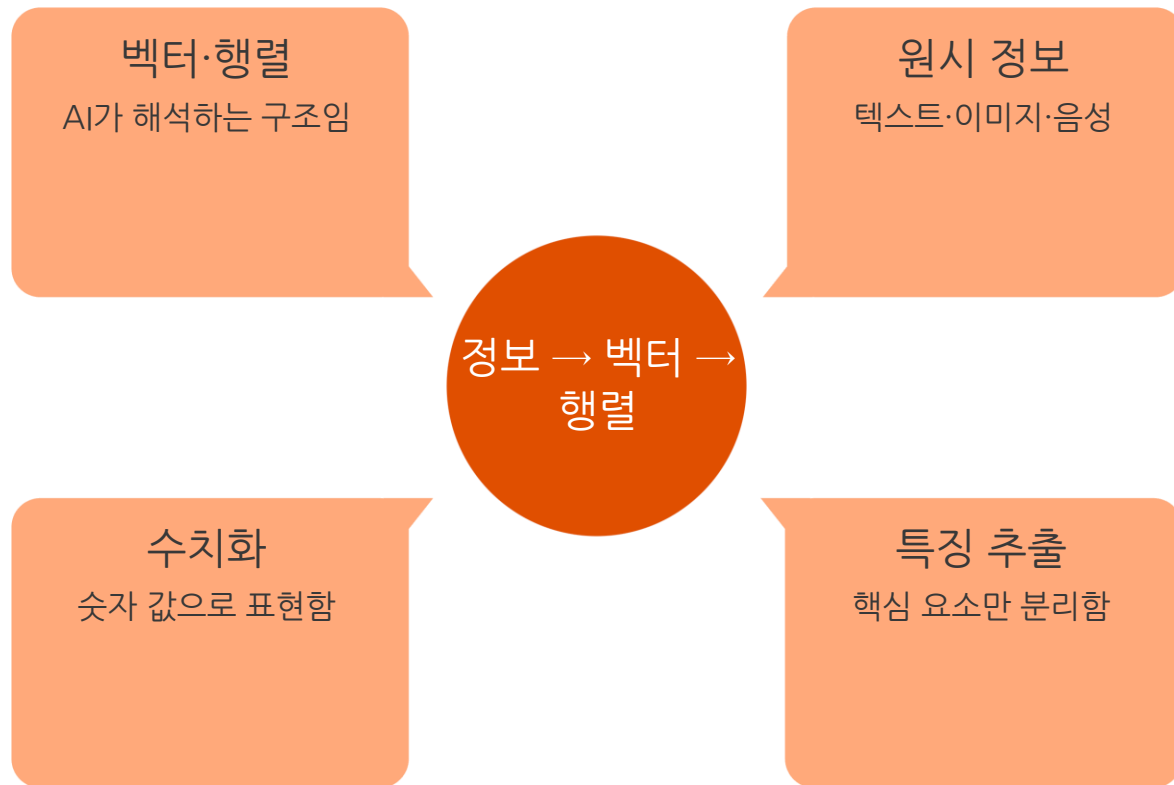
핵심 개념

- 토큰(Token): 모델이 처리하는 최소 언어 단위
- 어휘 사전(Vocab): 모든 토큰의 목록과 ID 매핑
- One-Hot: 해당 단어 위치만 1, 나머지 0인 희소 벡터
- 임베딩(Embedding): 의미를 압축한 밀집 벡터 (Word2Vec, BERT)

AI는 텍스트를 어떻게 쓰나?

- 토큰 시퀀스 → Transformer 입력
- 위치 인코딩(Positional Encoding) 추가
- Self-Attention으로 문맥 관계 학습
- 예: 번역(NMT), 감성 분석, ChatGPT

데이터의 본질: 모든 정보는 벡터와 행렬로



AI는 정보를 **숫자 배열**로 이해함.

사람의 특성도 눈·코·입·키·몸무게 같은 값을 수치화하면 하나의 **벡터**가 됨.

여러 벡터가 모이면 **행렬**이 되며, AI는 이 구조에서 패턴을 학습하고 예측함.

데이터 해석

선형 대수학이란?

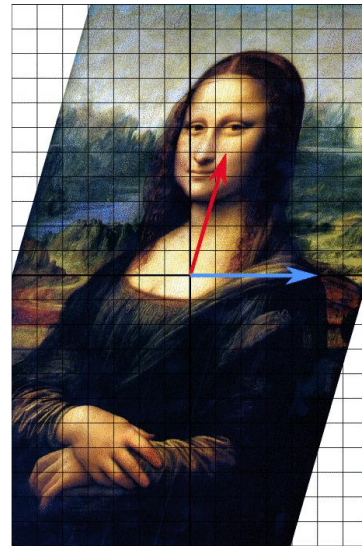
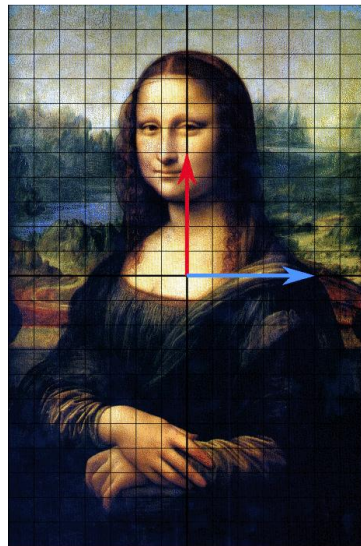
데이터의 생김새를 표현하는 언어를 배우는 학문

통계학이란?

다량 데이터를 다루는 법을 배우는 학문

해석학이란?

수식의 표현법을 배우는 학문

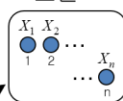


모집단

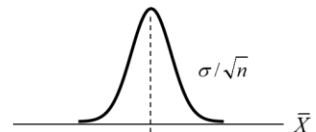
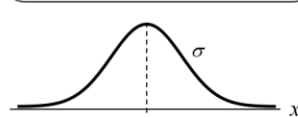


확률
표본
추출

표본



$$\begin{aligned}\bar{X} &= f(X_1, X_2, \dots, X_n) \\ &= \frac{X_1 + X_2 + \dots + X_n}{n}\end{aligned}$$



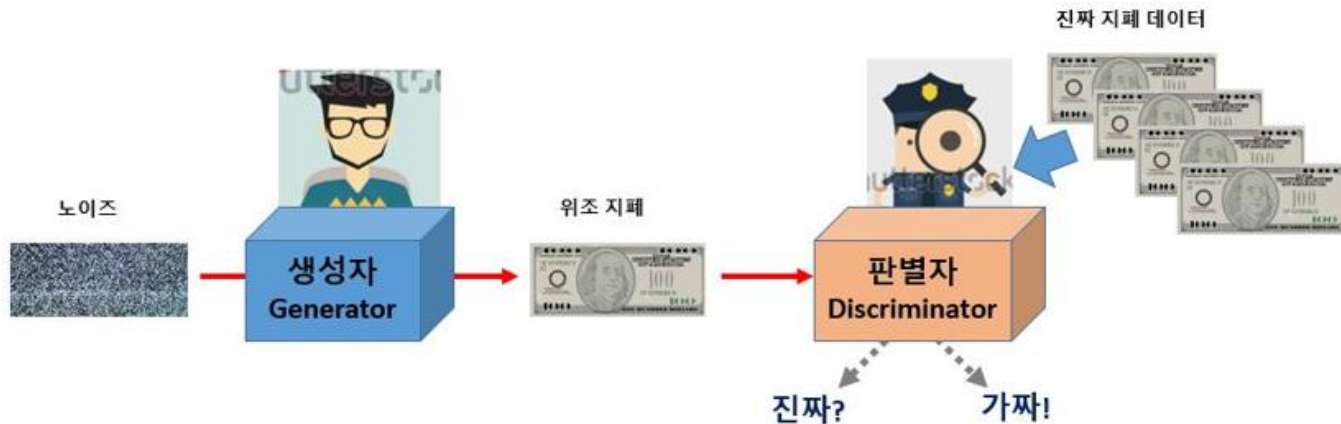
생성형 AI의 데이터 이해: GAN 사례

AI는 **원본 데이터의 분포**를 학습해 새로운 데이터를 생성함. 대표 사례가 GAN 임.

GAN은 진짜와 가짜를 구별하는 **판별자**와, 진짜 같은 이미지를 만드는 **생성자**가 경쟁하며 학습함.

이미지는 픽셀 값으로 이루어진 **숫자 행렬**이며, GAN은 그 행렬의 패턴과 분포를 모방해 실제 같은 결과를 만들어냄.

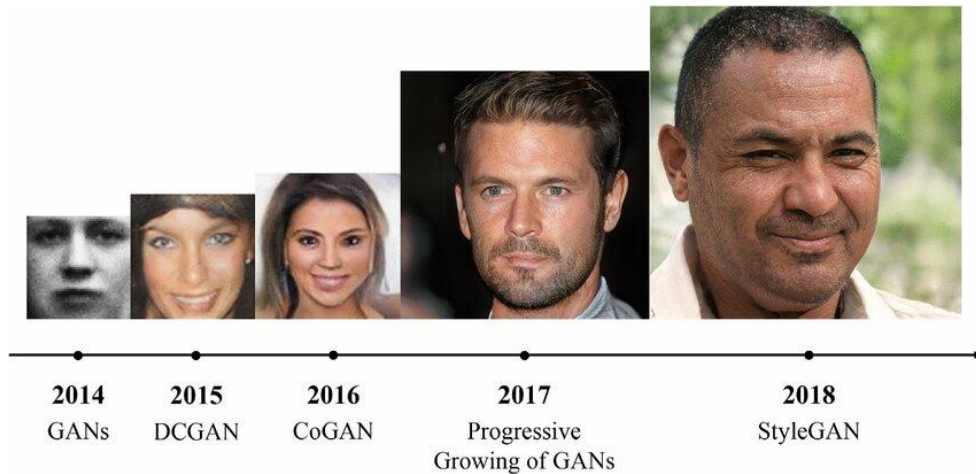
$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$



GAN 학습 과정: 노이즈 → 이미지

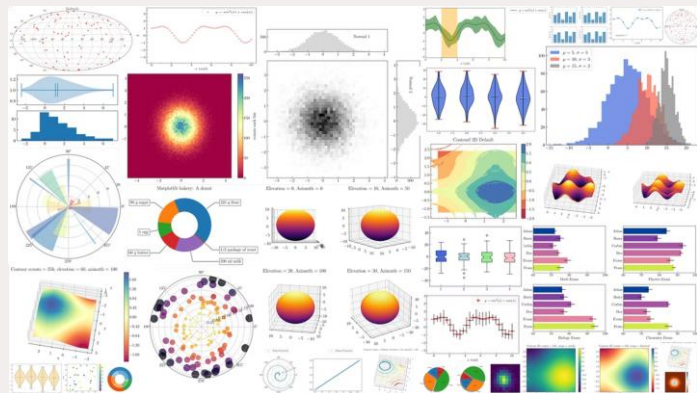
Generator는 랜덤 노이즈에서 시작해 학습을 거듭할수록 점점 실제에 가까운 이미지를 생성함.

- 1 Epoch 1
완전한 랜덤 노이즈. 픽셀이 무작위로 분포됨
- 2 Epoch 10
희미한 윤곽 등장. 형태가 보이기 시작함
- 3 Epoch 50
눈·코·입 형성. 얼굴 특징 구분 가능함
- 4 Epoch 200+
실제와 구분 불가능한 고품질 이미지 생성됨



데이터 통계 및 시각화

데이터 이해의 출발점은 EDA임. 기초 통계로 전체를 파악하고, 시각화로 패턴을 찾으며, 결측치·이상치를 점검함.



학습 데이터셋 필수 용어

용어	설명	예시
샘플 (Sample)	데이터셋의 개별 관측값	고객 1명의 데이터
피처 (Feature)	샘플을 설명하는 속성	나이, 소득, 지역
레이블 (Label)	예측 대상이 되는 정답값	이탈 여부 (0/1)
배치 (Batch)	한 번에 입력하는 샘플 묶음	Batch Size = 32

이 용어들은 이후 실습과 코드에서 반복 등장함. 지금 확실히 익혀두면 이후 내용 이해가 수월해짐.

탐색적 데이터 분석(EDA)이란?

EDA의 목적

- 데이터 구조 파악
- 분포·이상치 확인
- 전처리 방향 결정

EDA 주요 단계

1. 데이터 크기·타입 확인 (`df.info()`)
2. 기초 통계 요약 (`df.describe()`)
3. 결측치·중복 확인
4. 분포·상관관계 시각화

기초 통계: 중심 경향성과 산포도

평균 (Mean)

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

이상치에 민감함

중앙값 (Median)

정렬 후 중간값. 이상치에 강건함

분산 / 표준편차

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

평균 주변의 퍼짐 정도를 나타냄

사분위수 (IQR)

$$IQR = Q_3 - Q_1$$

이상치 탐지의 핵심 지표임

데이터 분포 이해 — 개요

데이터가 어떤 분포를 따르는지 파악하면 올바른 모델과 전처리 방법을 선택할 수 있다.

분포란 무엇인가?

- 데이터가 어떤 값에 얼마나 자주 나타나는지를 나타내는 함수
- 확률 밀도 함수(PDF): 연속형 데이터의 분포
- 확률 질량 함수(PMF): 이산형 데이터의 분포
- 누적 분포 함수(CDF): $P(X \leq x)$ 를 나타냄

분포를 알아야 하는 이유

- 이상치 탐지: 분포 꼬리 부분의 값 식별
- 모델 선택: 분포 가정에 맞는 알고리즘 선택
- 전처리 방향: 정규화·로그 변환 필요 여부 판단
- 가설 검정: 통계적 유의성 판단의 기반

연속형 분포

정규분포, 균등분포, 지수분포, 베타분포, 감마분포

이산형 분포

이항분포, 포아송분포, 기하분포, 초기하분포

연속형 분포 ① — 정규분포 & 균등분포

가장 자주 마주치는 두 연속형 분포. 자연 현상과 무작위 샘플링의 기반이다.

정규분포 (Normal Distribution)

자연 현상(키, 몸무게, 측정 오차)에서 가장 흔히 나타나는 종 모양 분포. 평균(μ)과 표준편차(σ)로 완전히 정의됨.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- 평균 = 중앙값 = 최빈값 (완전 대칭)
- 68-95-99.7 규칙: $\pm 1\sigma$ 에 68%, $\pm 2\sigma$ 에 95%, $\pm 3\sigma$ 에 99.7%
- 표준 정규분포: $\mu=0$, $\sigma=1$ 로 표준화한 형태

AI에서의 활용

- 가중치 초기화 (Xavier, He 초기화)
- 잔차(Residual) 분포 확인
- 가우시안 노이즈 추가 (데이터 증강)

균등분포 (Uniform Distribution)

모든 구간에서 동일한 확률을 가지는 분포. 연속형(Continuous)과 이산형(Discrete) 모두 존재함.

연속 균등분포

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

- 평균: $(a+b)/2$
- 분산: $(b-a)^2/12$

AI에서의 활용

- 난수 생성 및 무작위 샘플링
- 하이퍼파라미터 탐색 (Random Search)
- 드롭아웃 마스크 생성

연속형 분포 ② — 지수분포 & 베타분포

사건 간격과 확률 자체를 모델링할 때 핵심이 되는 분포들이다.

지수분포 (Exponential Distribution)

사건과 사건 사이의 대기 시간을 모델링하는 분포. 포아송 과정에서 자연스럽게 도출됨. 항상 양수 값만 가짐.

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

- λ : 단위 시간당 평균 발생 횟수 (rate parameter)
- 평균: $1/\lambda$, 분산: $1/\lambda^2$
- 무기억성(Memoryless): $P(X > s+t | X > s) = P(X > t)$

AI에서의 활용

- 서버 요청 간격, 장비 고장 시간 모델링
- 생존 분석 (Survival Analysis)
- 이상 탐지: 비정상적으로 짧은 간격 탐지

베타분포 (Beta Distribution)

0과 1 사이의 확률 값 자체를 모델링하는 분포. 두 형상 파라미터 α, β 로 다양한 모양을 표현함.

$$f(x) = \frac{x^{\alpha-1} (1-x)^{\beta-1}}{B(\alpha, \beta)}, \quad 0 \leq x \leq 1$$

- $\alpha=\beta=1$ 이면 균등분포와 동일
- $\alpha>\beta$ 이면 1에 치우침, $\alpha<\beta$ 이면 0에 치우침
- 평균: $\alpha/(\alpha+\beta)$

AI에서의 활용

- 베이지안 추론에서 사전 분포(Prior)로 활용
- A/B 테스트 전환율 모델링
- 드롭아웃 비율, 앙상블 가중치 모델링

이산형 분포 ① — 이항분포 & 포아송분포

성공/실패 횟수와 단위 시간 내 발생 횟수를 모델링하는 가장 중요한 이산 분포들이다.

이항분포 (Binomial Distribution)

n번의 독립 시행에서 성공 확률이 p일 때, 성공 횟수 X의 분포. 베르누이 시행의 합으로 구성됨.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

- 평균: np, 분산: np(1-p)
- n=1이면 베르누이 분포와 동일
- n이 크고 p가 작으면 포아송 분포로 근사 가능

AI에서의 활용

- 이진 분류 모델의 출력 확률 모델링
- 클래스 불균형 데이터 생성 (오버샘플링)
- 교차 검증 성능 통계 검정

포아송분포 (Poisson Distribution)

단위 시간(또는 공간)에 평균 λ번 발생하는 사건의 횟수 분포. 희귀 사건 모델링에 적합함.

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

- 평균 = 분산 = λ (포아송 분포의 특징)
- λ가 크면 정규분포로 근사 가능
- 사건들이 서로 독립이어야 함

AI에서의 활용

- 클릭 수, 구매 횟수 등 카운트 데이터 모델링
- 이상 탐지: 예상보다 많은 이벤트 발생 감지
- 자연어 처리: 단어 빈도 모델링

이산형 분포 ② — 기하분포 & 음이항분포

첫 번째 성공까지의 시행 횟수와 k번째 성공까지의 시행 횟수를 모델링한다.

기하분포 (Geometric Distribution)

성공 확률 p인 베르누이 시행을 반복할 때, 첫 번째 성공이 나올 때까지의 시행 횟수 X의 분포.

$$P(X = k) = (1 - p)^{k-1} p, \quad k = 1, 2, 3, \dots$$

- 평균: $1/p$, 분산: $(1-p)/p^2$
- 무기억성(Memoryless) 성질 보유
- 이산형 분포 중 유일하게 무기억성을 가짐

AI에서의 활용

- 첫 번째 이상 탐지까지의 시간 모델링
- 강화학습: 에피소드 길이 분포 모델링
- 재시도 횟수 기반 시스템 신뢰성 분석

음이항분포 (Negative Binomial Distribution)

성공 확률 p인 베르누이 시행에서 r번째 성공이 나올 때까지의 총 시행 횟수 분포. 기하분포의 일반화.

$$P(X = k) = \binom{k-1}{r-1} p^r (1 - p)^{k-r}$$

- $r=1$ 이면 기하분포와 동일
- 평균: r/p , 분산: $r(1-p)/p^2$
- 분산 > 평균 → 과산포(Overdispersion) 데이터에 적합

AI에서의 활용

- 포아송 분포로 설명 안 되는 과산포 카운트 데이터
- 텍스트 데이터의 단어 빈도 모델링 (NLP)
- 의료 데이터: 재입원 횟수, 재발 횟수 예측

분포 형태 분석 — 왜도 & 첨도

분포의 모양을 수치로 요약하면 전처리 방향과 모델 선택의 단서를 얻을 수 있다.

왜도 (Skewness) — 비대칭성

분포가 평균을 기준으로 얼마나 비대칭인지를 나타내는 지표.

$$\text{Skewness} = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^3$$

왜도 = 0

완전 대칭 (정규분포)

왜도 > 0 (양의 왜도)

오른쪽 꼬리가 긴 분포. 소득, 자산 데이터에 흔함. 로그 변환 권장.

왜도 < 0 (음의 왜도)

왼쪽 꼬리가 긴 분포. 시험 점수(상한 효과)에 흔함.

첨도 (Kurtosis) — 뾰족함

분포의 꼬리가 얼마나 두껍고 중심이 얼마나 뾰족한지를 나타내는 지표.

$$\text{Kurtosis} = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^4$$

첨도 = 3 (정규분포 기준)

Mesokurtic: 정규분포와 동일한 꼬리 두께

첨도 > 3 (Leptokurtic)

뾰족하고 꼬리가 두꺼움. 이상치 多. 금융 수익률에 흔함.

첨도 < 3 (Platykurtic)

납작하고 꼬리가 얇음. 균등분포에 가까운 형태.

❗ 실전 팁: |왜도| > 1이면 로그 변환(np.log1p), |왜도| > 2이면 Box-Cox 변환을 우선 고려함. 첨도가 높으면 이상치 처리 후 재확인 필요.

분포 선택 가이드 — 어떤 분포를 써야 할까?

데이터 특성	추천 분포	수식 핵심	실전 예시
연속, 대칭, 종 모양	정규분포	μ, σ	키·몸무게·측정 오차
연속, 구간 내 균등	균등분포	a, b	난수 생성·하이퍼파라미터 탐색
연속, 양수, 오른쪽 꼬리	지수분포	λ	서버 응답 시간·장비 수명
연속, 0~1 사이 확률	베타분포	α, β	A/B 테스트 전환율·사전 분포
이산, 성공/실패 횟수	이항분포	n, p	불량품 개수·클릭 여부
이산, 회귀 사건 횟수	포아송분포	λ	시간당 주문 수·오류 발생 수
이산, 첫 성공까지 횟수	기하분포	p	첫 구매까지 방문 횟수
이산, 과산포 카운트	음이항분포	r, p	재입원 횟수·단어 빈도

EDA에서 왜도·첨도가 중요한 이유

① 전처리 방향 결정

왜도·첨도를 확인하면 어떤 변환이 필요한지 즉시 판단할 수 있다.

|왜도| < 0.5 → 변환 불필요
정규분포에 가까움. StandardScaler 바로 적용 가능.

$0.5 \leq |왜도| < 1 \rightarrow$ 로그 변환 고려
 $\text{np.log1p}(x)$ 적용 후 왜도 재확인.

|왜도| ≥ 1 또는 첨도 높음 → 강한 변환 필요
Box-Cox, Yeo-Johnson 변환 또는 이상치 제거 선행.

EDA 실전 순서

01

df.describe()로 기초 통계 확인

02

df.skew() / df.kurtosis()로 왜도·첨도 수치 확인

03

히스토그램 + KDE로 분포 시각화

04

왜도 기준으로 변환 전략 결정 후 재확인

왜도와 첨도는 단순한 통계 수치가 아니다. EDA의 모든 후속 결정을 좌우하는 나침반이다.

② 모델 선택 기준

분포 형태는 어떤 알고리즘이 잘 작동할지를 결정한다.

- 선형 회귀 / 로지스틱 회귀: 피처가 정규분포에 가까울수록 성능 안정적
- 트리 기반 모델 (랜덤 포레스트, XGBoost): 왜도·첨도에 상대적으로 강건함. 변환 없이도 작동 가능.
- 신경망: 정규화 필수. 왜도가 크면 학습 불안정 (Gradient Exploding 위험)
- 나이브 베이즈: 정규분포 가정 → 왜도 높으면 성능 급락

③ 이상치 탐지 신호

- 첨도 > 5: 꼬리가 두꺼워 극단값 존재 가능성 높음 → IQR/Z-Score 탐지 즉시 수행
- 왜도 > 2: 한쪽 방향으로 극단값이 분포를 왜곡 중 → 박스플롯으로 시각 확인

Python 코드

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 왜도·첨도 일괄 확인
skew_df = pd.DataFrame({
    'skewness': df.skew(),
    'kurtosis': df.kurtosis()
}).sort_values('skewness', ascending=False)
print(skew_df)

# 왜도 높은 피처 시각화
high_skew = skew_df[skew_df['skewness'].abs() > 1].index
df[high_skew].hist(bins=30, figsize=(12, 6))
plt.tight_layout()
plt.show()
```

시각화 라이브러리 개요: 언제 무엇을 쓸까?

데이터 분석에서 시각화는 숫자를 이해로 바꾸는 핵심 도구임. 목적에 따라 라이브러리를 선택함.



Matplotlib

- 파이썬 시각화의 기반 라이브러리
- 세밀한 커스터마이징 가능
- 정밀 그래프, 서브플롯, 커스텀 레이어아웃에 사용함



Seaborn

- Matplotlib 기반의 통계 시각화 특화
- 적은 코드로 깔끔한 그래프 생성됨
- EDA, 분포 확인, 변수 관계 탐색에 사용함



Plotly

- 인터랙티브 시각화 지원함
- 웹/대시보드에 최적화됨
- 발표용, 대시보드, 탐색적 분석에 사용함

⚠ 이 캠프에서는 **Matplotlib + Seaborn** 중심으로 실습함. EDA → 전처리 → 모델 평가 전 단계에서 시각화를 활용함.



Matplotlib 실전 예시

Matplotlib은 가장 기본적이고 강력한 시각화 도구임. 아래는 데이터 분석에서 자주 쓰는 4가지 차트임.

히스토그램 — 분포 확인

데이터가 어떤 값에 몰려 있는지 확인함. 정규분포 여부와 치우침 파악에 유용함.

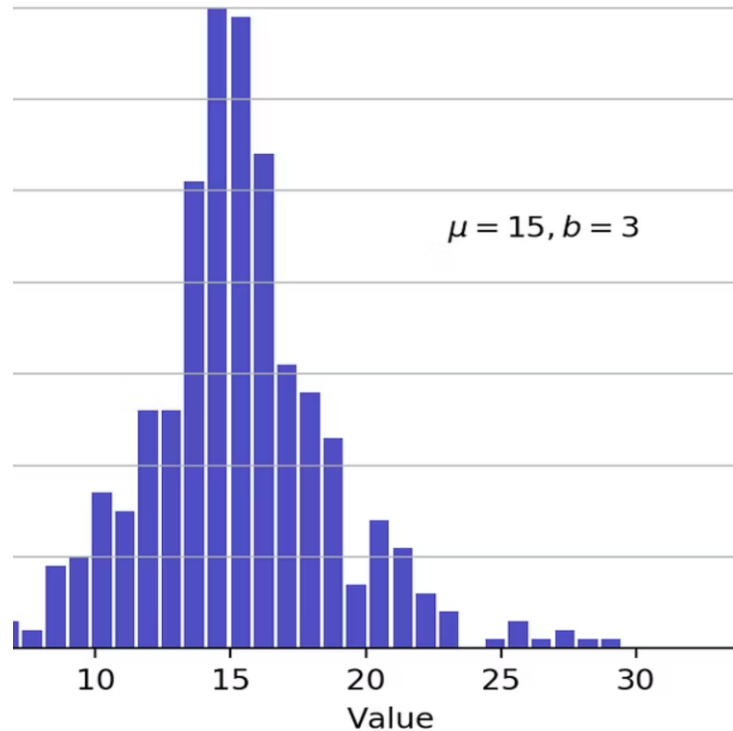
```
plt.hist(df['age'], bins=30,
color='steelblue',
edgecolor='white')
plt.title('나이 분포')
plt.xlabel('나이')
plt.ylabel('빈도')
```

선 그래프 — 추세 확인

시간에 따른 변화나 epoch별 loss 변화를 모니터링할 때 사용함.

```
plt.plot(epochs, train_loss, label='Train Loss')
plt.plot(epochs, val_loss, label='Val Loss')
plt.legend()
```

My Very Own Histogram





Seaborn 실전 예시 — 통계 시각화

Seaborn은 통계 관계를 빠르게 파악하는 데 유용함. EDA에서 자주 활용됨.

히트맵 — 상관관계 확인

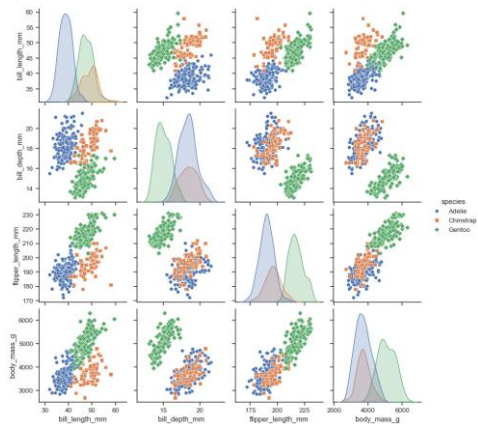
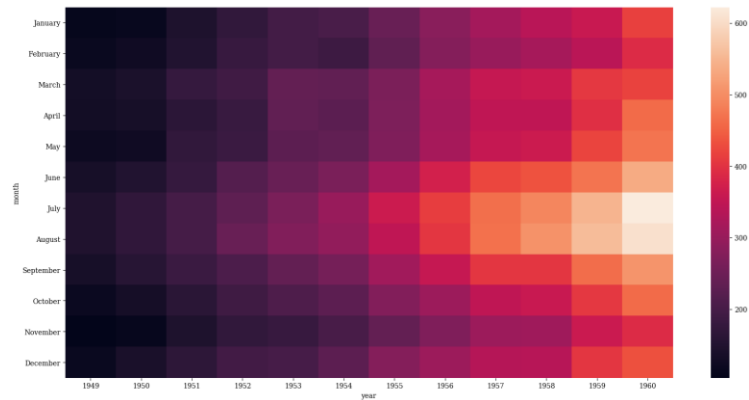
피처 간 상관계수를 색상으로 표현함.
다중공선성 탐지에 유용함.

페어플롯 — 변수 쌍 분석

산점도와 분포를 한 번에 확인함. 클래스별 분리 가능성 파악에 유용함.

```
corr = df.corr()
sns.heatmap(corr, annot=True,
            cmap='coolwarm', fmt='.2f')
```

```
sns.pairplot(df,
             hue='species',
             diag_kind='kde')
plt.suptitle('변수 쌍 관계 분석', y=1.02)
```



Seaborn 실전 예시 — 분포 & 이상치 탐지

박스플롯과 바이올린플롯은 데이터의 분포와 이상치를 동시에 파악하는 데 유용함.

박스플롯 — 분포 요약 & 이상치 탐지

중앙값, 사분위수, 이상치를 한눈에 보여줌.

- 박스: 25~75% 범위 (IQR)
- 가운데 선: 중앙값
- 점: 이상치

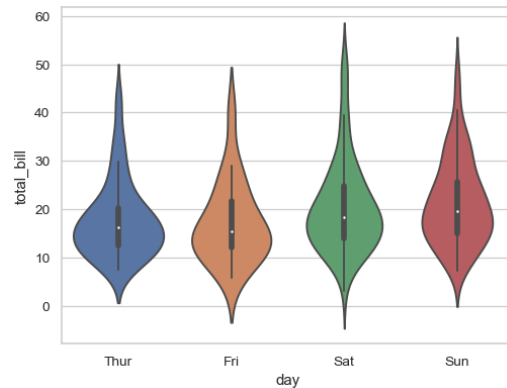
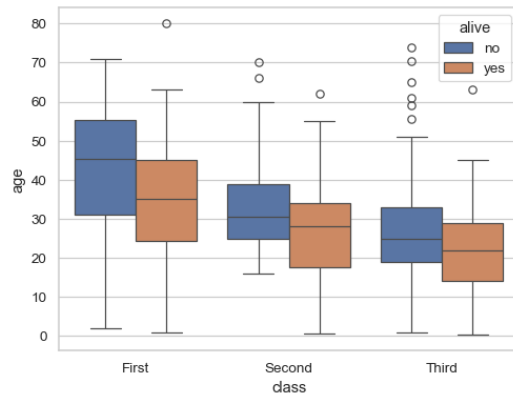
```
sns.boxplot(x='category', y='value',
data=df, palette='Set2')
plt.title('카테고리별 값 분포')
```

바이올린플롯 — 분포 형태 확인

박스플롯에 KDE가 결합된 형태임. 밀집 구간을 직관적으로 확인함.

- 넓은 부분: 데이터 밀집 구간
- 좁은 부분: 데이터 희박 구간

```
sns.violinplot(x='species',
y='petal_length', data=iris,
palette='muted', inner='quartile')
plt.title('종별 꽃잎 길이 분포')
```



📌 이상치 개수 확인 → 박스플롯 / 분포 형태까지 확인 → 바이올린플롯

결측치 처리 전략

1

결측치 탐지

`df.isnull().sum()`으로 컬럼별 결측 개수 확인함. 5% 미만이면 단순 처리 가능함

2

제거 (Deletion)

결측 비율 > 50%인 컬럼은 삭제함. 완전 무결 행만 쓰는 Listwise는 데이터 손실 주의됨

3

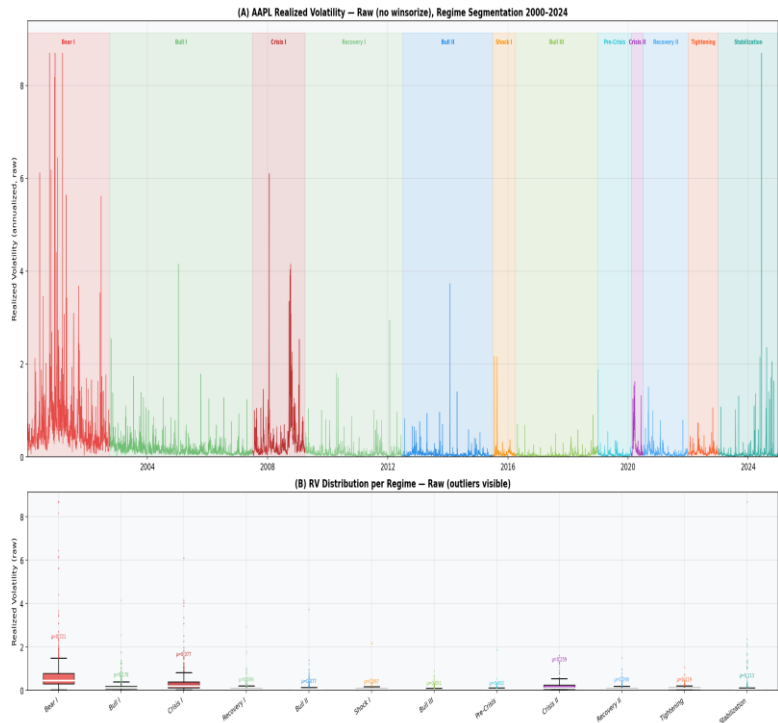
대체 (Imputation)

수치형은 평균/중앙값, 범주형은 최빈값으로 대체함. 고급 방식으로 KNN, MICE 활용함

4

모델 기반 처리

`is_null_flag`를 새 피처로 생성함. 결측 패턴이 의미 있으면 효과적임



이상치(Outlier) 탐지 및 처리

IQR 방법

$$Q_1 - 1.5 \times IQR$$

$$Q_3 + 1.5 \times IQR$$

이 범위를 벗어나면 이상치로 판단함. 박스플롯으로 함께 확인함.

Z-Score 방법

$$z = \frac{x - \mu}{\sigma}$$

$|z| > 3$ 이면 이상치로 판단함. 정규분포 데이터에 적합함.



이상치는 무조건 제거하지 말아야 함. 사기 탐지처럼 분석 목표가 될 수 있음.

이상치를 처리해야 하는 이유 — 실제 사례

❌ 처리 안 했을 때 생기는 문제

평균·분산 왜곡

연봉 데이터에 CEO 연봉(10억)이 포함되면 평균이 수천만 원 상승 → 일반 직원 분석 불가

선형 모델 성능 급락

선형 회귀는 이상치에 민감. 이상치 1개가 회귀선 전체를 끌어당겨 예측 오차 폭증

정규화 스케일 오염

MinMaxScaler 사용 시 이상치가 최대·최솟값이 되어 나머지 데이터가 0~0.01 범위로 압축됨

처리 전 반드시 물어야 할 질문

- 이 이상치는 입력 오류인가, 실제 현상인가?
- 분석 목표가 이상치 탐지 자체인가?
- 이상치 제거 후 데이터 분포가 어떻게 바뀌는가?

이상치 하나가 모델 전체를 망친다. 그러나 때로는 이상치 자체가 핵심 신호다.

✅ 이상치가 오히려 핵심인 경우

금융 사기 탐지 (Fraud Detection)

정상 거래 99.9% 중 이상 거래 0.1%가 이상치. 이상치를 제거하면 탐지 대상 자체가 사라짐

의료 이상 탐지

환자 대부분의 정상 수치 중 극단값이 질병 신호. 이상치 = 조기 진단 단서

제조 불량 감지

센서 데이터에서 정상 범위를 벗어난 값이 설비 고장 예고. 이상치 제거 시 경보 시스템 무력화

상황별 처리 전략

상황

전략

입력 오류 (음수 나이 등)

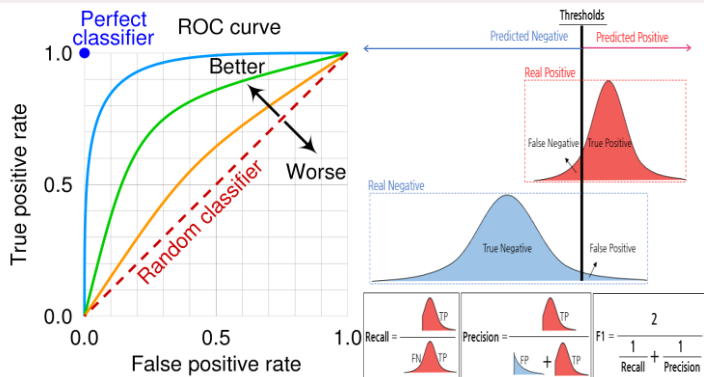
제거 또는 결측치로 대체

극단값이지만 실제 데이터

IQR 클리핑 (상·하한 고정)

이상치가 분석 목표

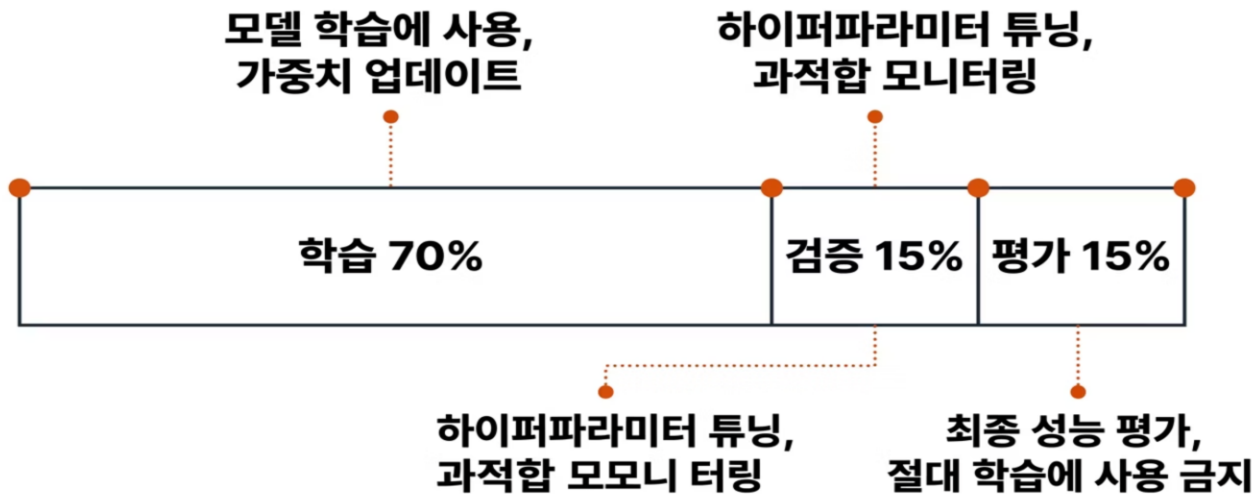
제거 금지, 별도 레이블링



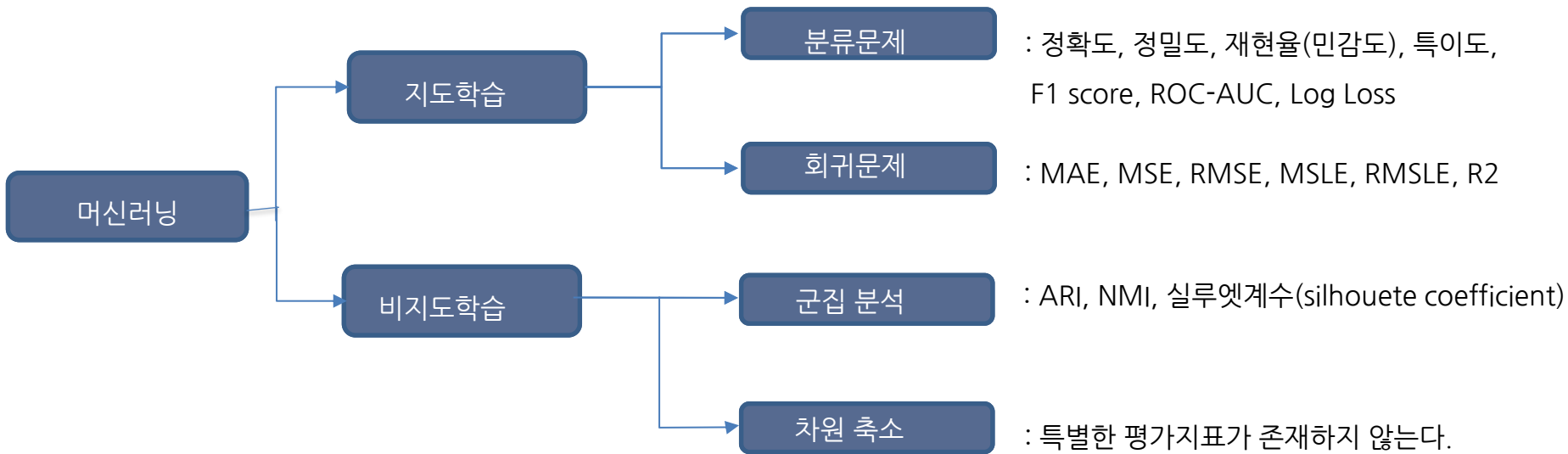
평가지표

데이터를 통해 모델링을 진행한 이후
해당 모델을 평가하는 기준은 어떻게 마련할 것인가?

Train / Validation / Test 분할



평가지표



		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

- TP(True Positive) : 실제 Positive인 정답을 Positive라고 예측 (True)
- TN(True Negative) : 실제 Negative인 정답을 Negative라고 예측 (True)
- FP(False Positive): 실제 Negative인 정답을 Positive라고 예측 (False) - *Type I error*
- FN(False Negative): 실제 Positive인 정답을 Negative라고 예측 (False) - *Type II error*

예측 클래스 (Predicted Class)

	Negative(0)	Positive(1)
Negative(0)	TN (True Negative)	FP (False Positive)
Positive(1)	FN (False Negative)	TP (True Positive)

이때, 긍정과 부정에 상관없이 우리가 관심있는 결과가 Positive가 됨을 명심하자.

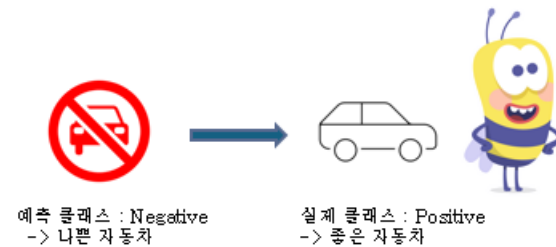
Type I error

예측 클래스
(Predicted Class)

실제 클래스
(Actual Class)

	Negative(0)	Positive(1)
Negative(0)	TN (True Negative)	FP (False Positive)
Positive(1)	FN (False Negative)	TP (True Positive)

* 실제 Negative인 정답을 Positive라고 예측 (False)

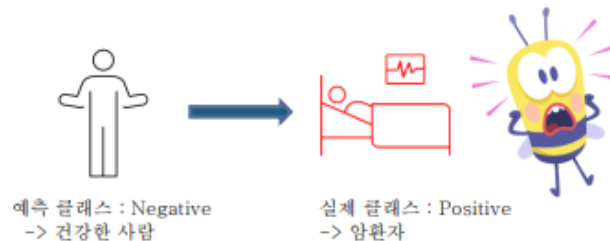
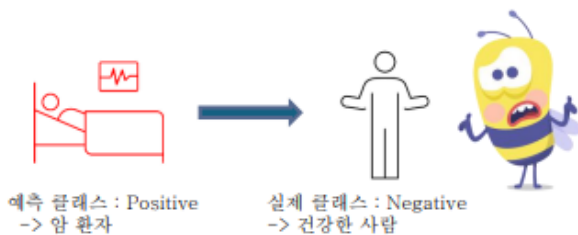


Type II error

예측 클래스
(Predicted Class)

실제 클래스
(Actual Class)

	Negative(0)	Positive(1)
Negative(0)	TN (True Negative)	FP (False Positive)
Positive(1)	FN (False Negative)	TP (True Positive)



Accuracy (정확도)

예측 클래스
(Predicted Class)

실제 클래스
(Actual Class)

	Negative(0)	Positive(1)
Negative(0)	TN (True Negative)	FP (False Positive)
Positive(1)	FN (False Negative)	TP (True Positive)

$$\frac{TP + TN}{TP + TN + FP + FN}$$

❖ 정의 : 전체 예측 건수에서 정답을 맞춘 건수의 비율

❖ 단점 : 데이터의 불균형이 심할 경우 정확한 성능을 측정하기 어려움

만약, 성별분류 문제에서 데이터의 정답이 90%가 남성이고 10%가 여성일 경우 모델이 모든 데이터를 남성으로 예측한다고 해도 정확도가 90%이므로 제대로 된 평가를 할 수 없다.

Recall (재현율, 민감도)

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

$$\frac{TP}{TP + FN}$$

❖ 정의 : 실제로 정답이 True 인 것들 중 분류기가 True로 예측한 비율
데이터에서 True가 발생하는 확률이 적을 때 사용하면 좋은 평가지표

❖ 단점 : 남성 10%, 여성 90%의 불균형 데이터의 분류 문제에서 만약 True로만 답하는 분류기가 있다고 할 때 , 남성의 경우 Accuracy는 낮지만 Recall은 1이 된다.

Precision (정밀도)

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

$$\frac{TP}{TP + FP}$$

❖ 정의 : True 로 예측한 것들 중 실제 정답이 True인 비율

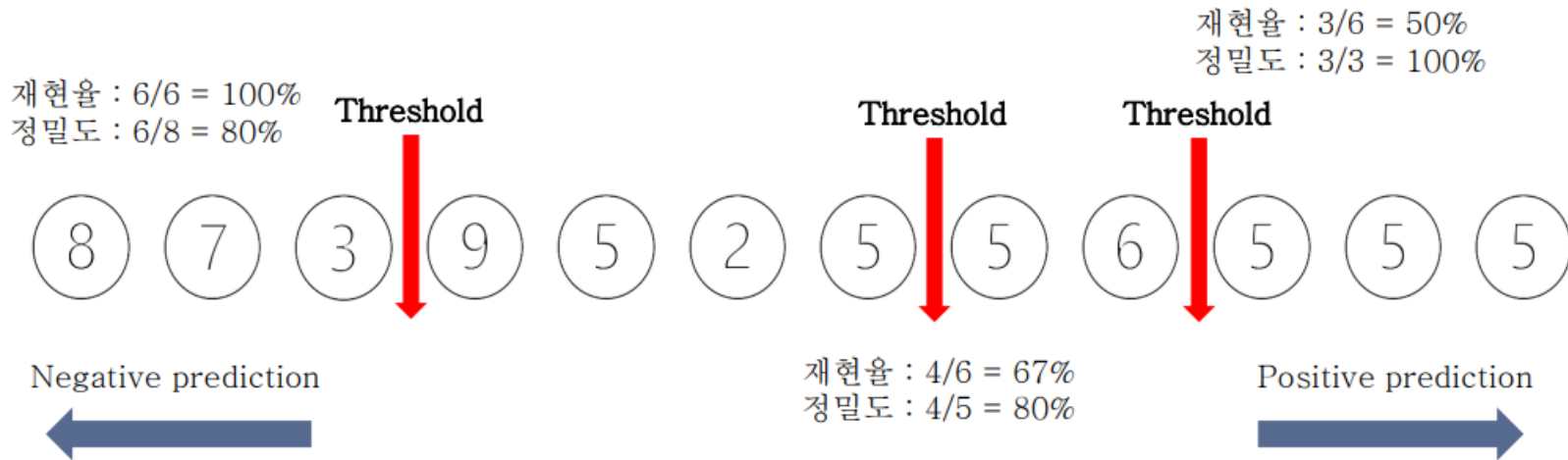
즉, 언제나 True만 답하는 분류기는 Recall은 1이지만 precision은 0에 가깝다.

❖ 단점 : Recall의 장점이 Precision의 단점이 된다. 두 지표는 서로 반대의 개념이기

때문이다.

Threshold

- * 재현율(Recall) : 실제로 정답이 True 인 것들 중 분류기가 True로 예측한 비율
- * 정밀도(Precision) : True 로 예측한 것들 중 실제 정답이 True인 비율



❖ Threshold : 모델이 0,1을 나누는 기준

예를들어, 모델의 Threshold가 0.5이면 0.5보다 작으면 0, 크면 1로 예측한다.

F1 Score

$$2 \times \frac{Precision \times Recall}{Precision + Recall}$$

❖ 정의 : Precision과 Recall의 조화평균

반대 개념인 두 지표를 모두 균형 있게 반영하기 위함이다. 이 지표는 어느 한쪽에 치우치지 않았을 때 높은 값이 가지게 된다.

❖ Example

: Recall 1, Precision 0.01의 값을 가지는 분류기가 있다고 하자.

산술평균 : $(1 + 0.01) / 2 = 0.505$

조화평균 : $2 * (1 * 0.01) / (1 + 0.01) = 0.019$

예측 클래스
(Predicted Class)

실제 클래스
(Actual Class)

	Negative(0)	Positive(1)
Negative(0)	TN (True Negative)	FP (False Positive)
Positive(1)	FN (False Negative)	TP (True Positive)

- True Positive (truth = 1, guess = 1)
- True Negative (truth = 0, guess = 0)
- False Positive (truth = 0, guess = 1) - Type I error
- False Negative (truth = 1, guess = 0) - Type II error

▷ **Accuracy** : 분류기가 정답을 맞춘 비율

▷ **Recall** : $\frac{TP}{TP+FN}$ → 즉, Type II error 가 더 중요할 경우에 살펴봐야 할 평가지표이다.

▷ **Precision** : $\frac{TP}{TP+FP}$ → 즉, Type I error 가 더 중요할 경우에 살펴봐야 할 평가지표이다.

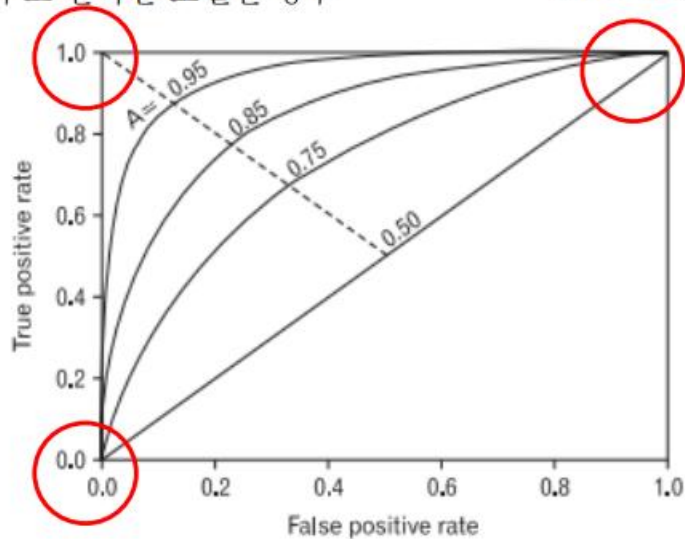
▷ **F1 Score** : 재현율과 정밀도 모두를 조화롭게 살펴볼 때 측정하는 평가지표이다.

ROC-AUC

ROC curve 아래 면적을 AUC(Area Under the Curve) 라고 부른다.
아래 면적이 넓을 수록 좋은 성능을 가진 모델

잘못 예측한 것 없이 모두 맞춘 경우로 완벽한 모델인 경우

모든 예측을 Positive(1)으로 한 경우



모든 예측을

Negative(0) 으로 한 경우

MAE (Mean Absolute Error)

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

예측값과 실제값의 차이의 절댓값을 모두 더해 평균한 값
에러의 절댓값 그 자체이기 때문에 낮을수록 좋은 값

MSE (Mean Squared Error)

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

예측값과 실제값을 제곱하여 평균한 값
제곱을 하기 때문에 큰 오차가 강조되는 효과를 가진다.

▷ 단점

에러를 제곱하기 때문에 1 미만의 에러는 작아지고, 그 이상의 에러는 커지는
값의 왜곡이 발생
MAE에 비해 이상치에 민감하다.

깊이 들어가기 — To the Deep Learning

딥러닝이라는 기술을 공부하기 앞서, 우리가 어떤 문제에서의 어떤 값을 최적화하는지 배우고 학습시키고자 하는 대상이 무엇인지 구체화 하기.

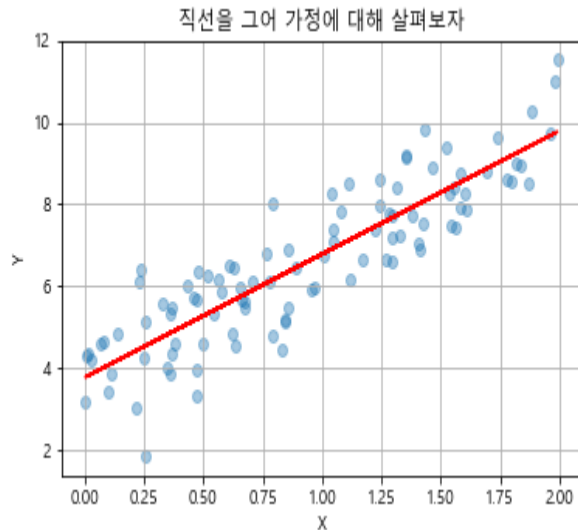
Goal

현실에서의 문제를 데이터를 활용하여 해결 or 차선택 제시

- 1인당 GDP 와 삶의 만족도 간의 관계
- 대선 주자의 소득과 지지율의 관계 ex) 선형회귀, DT
- 고객의 사진을 보고 고객의 나이 예측

기계에게 목적을 부여하기 위한 작업?

- 비용함수 정의
- ex) $cost(W, b) = \frac{1}{N} \sum_{(x,y) \in D} (y - prediction(x))^2$

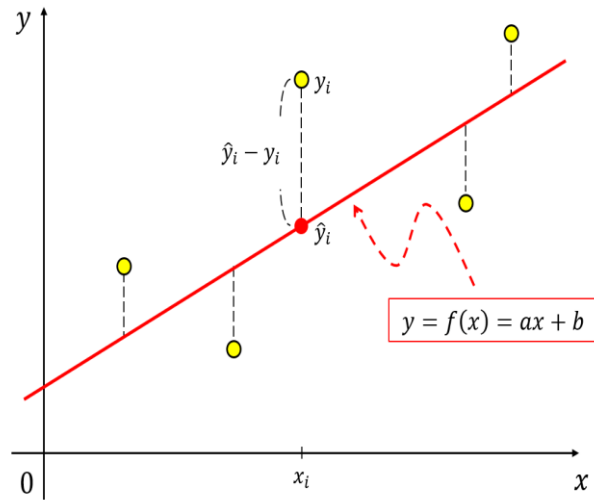


기계에게 내 의도를 전해주자

컴퓨터에게 의도를 전달하려면 알맞은 **목적(비용함수)**을 부여해야 한다.

- **비용함수(Cost Function)**: 설계자의 의도에 맞게 정의 가능
- 예측값과 실제값의 **오차**를 수치화하는 함수
- 회귀 분석의 대표 비용함수 → **MSE**

→ *MSE 최소화 = 비용 최소 = 목적 달성*



MSE: 평균제곱오차

예측값과 실제값의 차이를 제곱해 평균낸 값

수식

$$cost(W, b) = \frac{1}{N} \sum_{(x,y) \in D} (y - prediction(x))^2$$

의미

- 오차가 클수록 비용이 기하급수적으로 증가
- 큰 오차에 더 민감 → 큰 실수를 줄이도록 학습

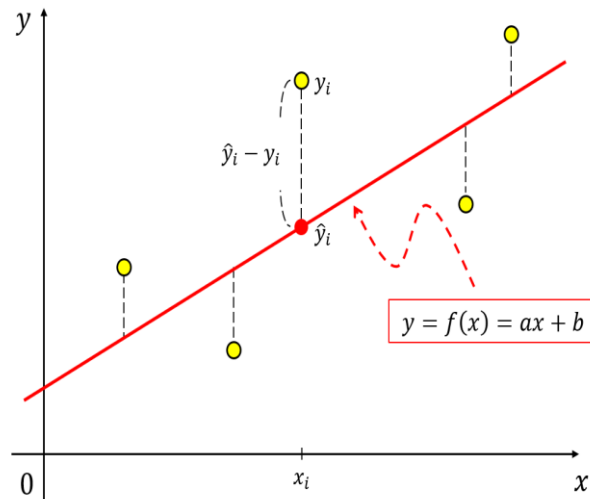
비용을 최소화하는 방법

비용함수가 **최소가 되는 W, b** 를 찾아야 한다.

How?

- 해석적 방법 : 직접 미분해서 0이 되는 지점 계산
- 경사하강법(Gradient Descent) : 기울기 따라 점진적 이동

→ 변수가 많을 땐 경사하강법이 표준



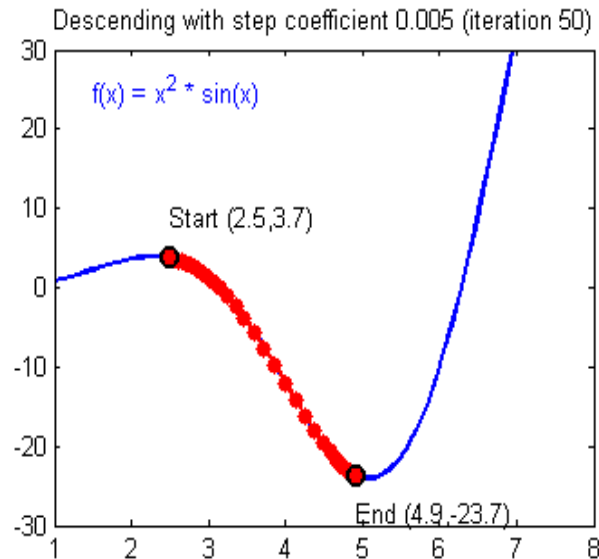
해석적 방법 — 쌍으로 미분하기

- 정규방정식 (Normal Equation)

$$\theta = (X^T X)^{-1} X^T Y$$

- 비용함수가 **Convex** → 전역 최솟값 1개 존재
- 변수 X 가 많으면 **역행렬 계산이 매우 무거움**

→ 한 번에 점프하지 말고, 차근차근 내려가자
경사하강법 (Gradient Descent)



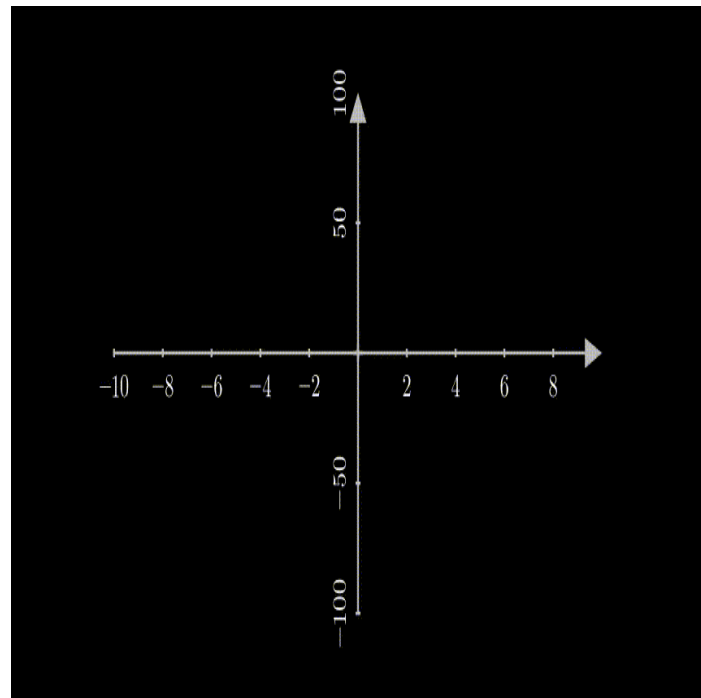
경사하강법의 물음들

Q1. 왜 경사하강법인가?

- 실제 함수는 미분계수 구하기가 어렵다
- 해석적인 근 $\rightarrow$ 계산 복잡도가 너무 큼
- 반복적인 방법(**Iterative**)이 더 효율적

Q2. 우리는 무엇을 찾는가?

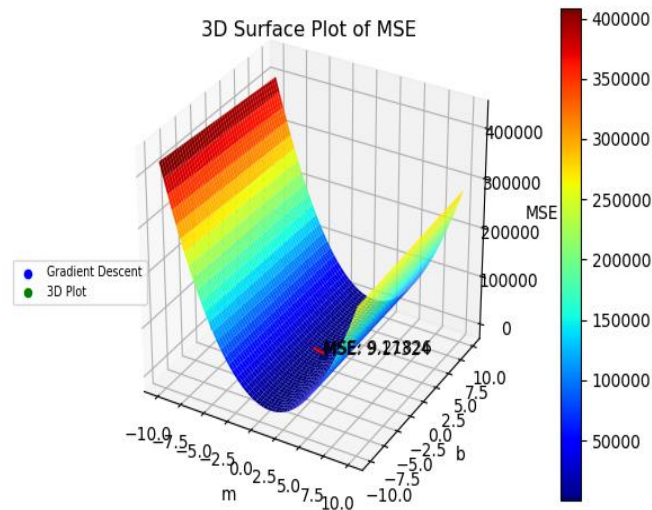
- 입력 x 는 **고정**, 변하는 것은 파라미터 θ
- 선형회귀에서는 θ = 직선의 **기울기**
- 비용함수 $MSE(\theta)$ 의 최솟점을 찾는 θ
- 우리가 찾는 것은 θ, β, w
- $$\frac{\partial}{\partial \theta} MSE(\theta) = \frac{2}{m} \sum^m (\theta^T x^{(i)} - y^{(i)}) x_j^{(i)}$$



Learning Rate — 보폭의 문제

- 경사하강법은 **한 걸음씩** 비용이 작아지는 곳으로 이동
- 그렇다면 **보폭은 얼마로?** → 그것이 **Learning Rate (α)**
- 매 step마다 기울기 $\times \alpha$ 만큼 파라미터 업데이트
- α 설정은 **설계자의 몫** (대표적 하이퍼파라미터)

※ 옆 그래프는 *Cost* 의 파라미터 공간 (변수 2개, z축 = *Cost*)



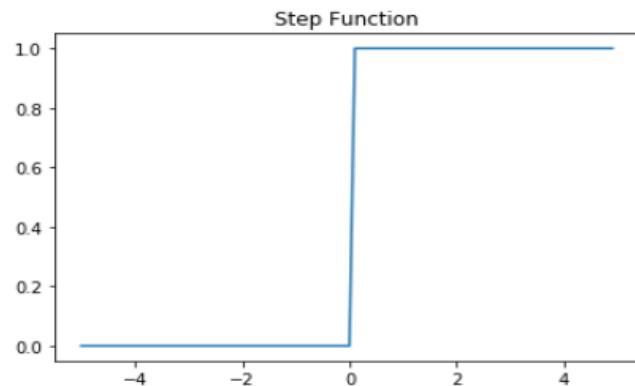
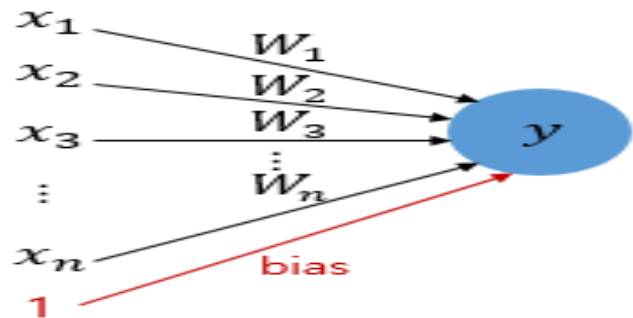
고전 ML의 한계와 퍼셉트론

고전 ML은 왜 한계가 있었나?

- 분류가 **선형**이라 비선형 문제(이미지·음성)에 무력
- 해법: 퍼셉트론을 쌓고 **오차 역전파**로 학습

퍼셉트론(Perceptron)이란?

- Frank Rosenblatt이 제안한 초기 인공신경망
- 여러 입력 $\times$ w (가중치) $\rightarrow$ 임계 초과 시 신호 출력
- 출력이 정답과 맞도록 w 를 조정

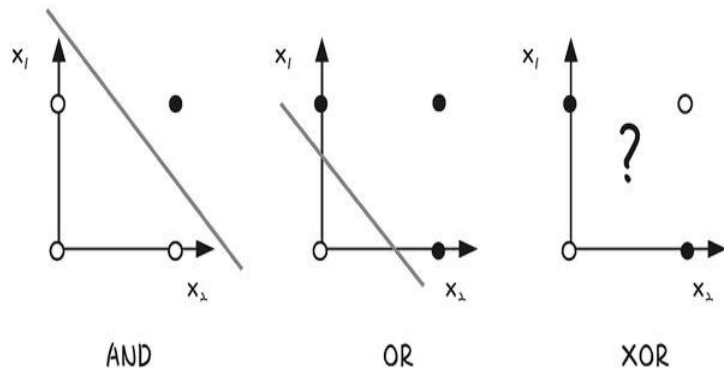
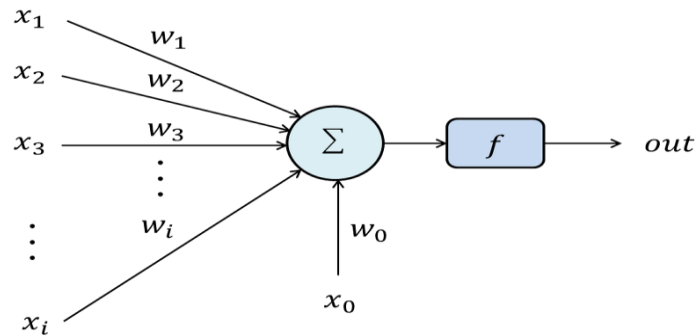


단층 퍼셉트론의 한계: XOR

- 단층 퍼셉트론: bias + x를 입력받아 선형 결합 → 비선형 함수로 한 번 변환
- 하지만 한 번만으로는 **XOR**(같으면 0, 다르면 1) 분류 불가

선을 긋는다 = 무엇을 한다?

- 목적은 **데이터 분류**, 출력은 0/1
- 일종의 **부등식의 영역**을 나누는 경계



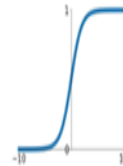
비선형 함수(활성화 함수)

- Activation function이라 부름 — Sigmoid, ReLU 등
- 왜 필요한가? 선형 함수만 쌓으면 은닉층을 늘려도 의미 없음
 - $y = ax \rightarrow y = bx \rightarrow y = cx$ 는 결국 $y = abc \cdot x$
- 퍼셉트론 = 선형 결합 $\rightarrow$ 비선형 함수 통과
- 현재는 **ReLU**가 Sigmoid보다 더 널리 쓰임

Activation Functions

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

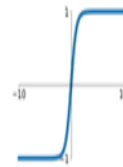


Leaky ReLU
 $\max(0.1x, x)$



tanh

$$\tanh(x)$$

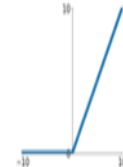


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

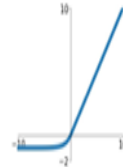
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

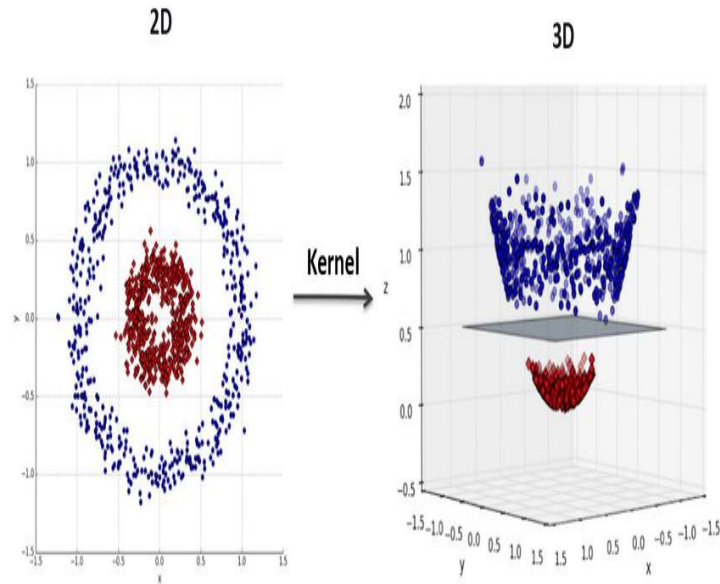


"선형 분류만 가능하다"의 의미

- 퍼셉트론은 결국 **선형 결합**만 수행
- 많이 쌓아도 한 퍼셉트론이 하는 일은 **직선 긋기**
- 정의역(시점)을 바꾸면? → **시점 변환**

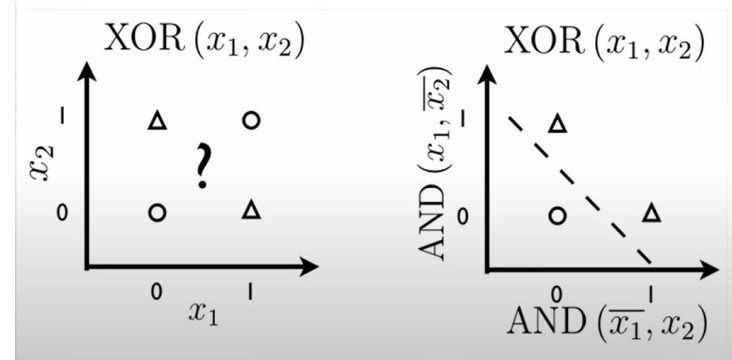
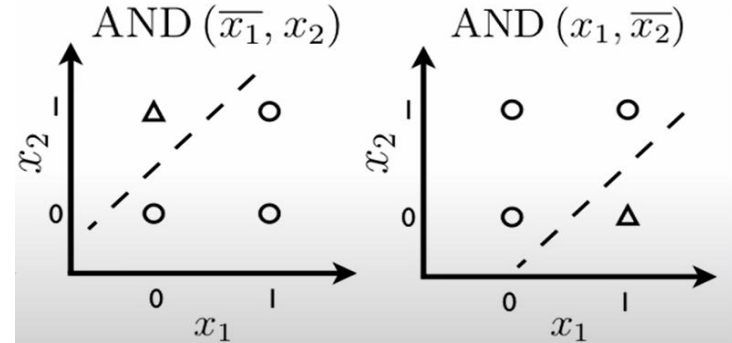
예: $AND(\neg x_1, x_2)$ 로 비선형 분류

- $AND(\neg x_1, x_2)$: 둘 다 1이면 1, 그 외 0
- 단일 퍼셉트론은 선형, 하지만 **벡터의 관점**을 바꾸면 비선형 표현 가능



다층 퍼셉트론 = 시점 변환

- 각 퍼셉트론은 선형 결합(직선 긋기)만 가능
- 층을 넘어갈 때 **축(시점)이 바뀜** → 합성함수의 정의역 공간
- 최종적으로 x 관점에서 **비선형 출력**



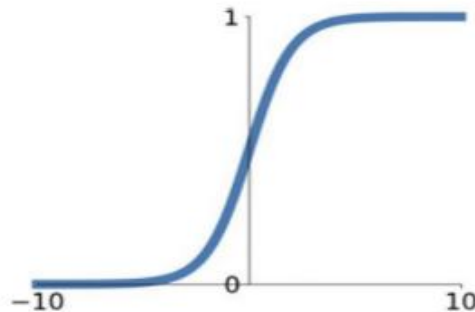
$XOR(A_1, A_2)$	$AND(\overline{x_1}, x_2)$	$AND(x_1, \overline{x_2})$	x_1	x_2
0	0	0	0	0
0	0	0	1	1
1	1	0	0	1
1	0	1	1	0

활성화 함수란?

- 입력 신호의 총합을 출력 신호로 변환하는 함수

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



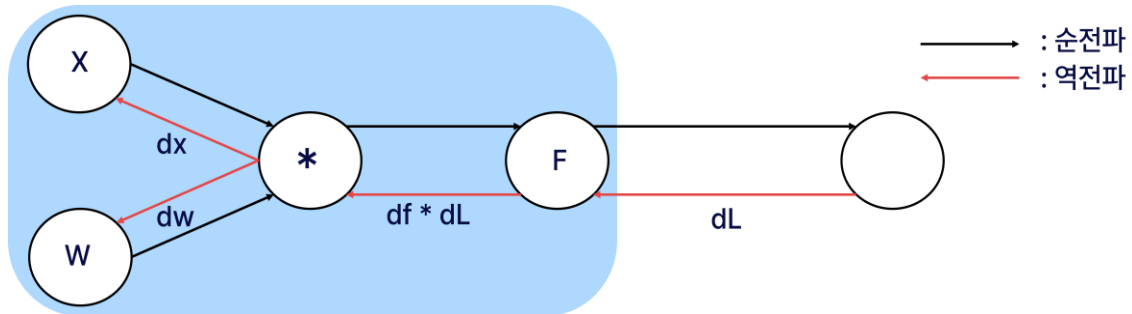
특징

- 과거에 가장 많이 사용되었던 활성화 함수
- 출력 값이 0부터 1사이의 값이 출력

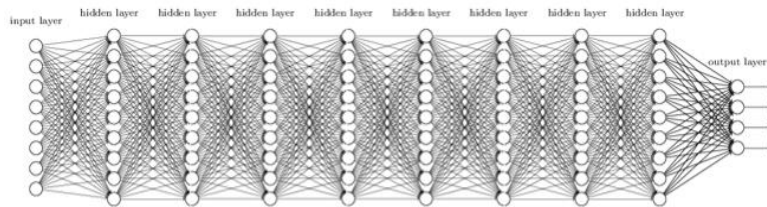
단점

1. Non-zero centered
2. Gradient vanishing

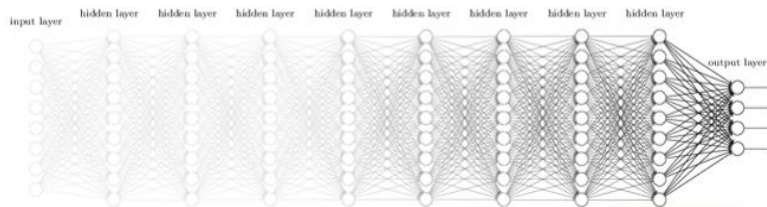
Non-zero Centered



Gradient Vanishing



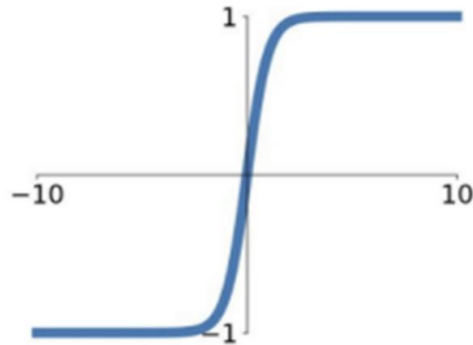
Deep Neural Network



Vanishing Gradient

tanh

$\tanh(x)$



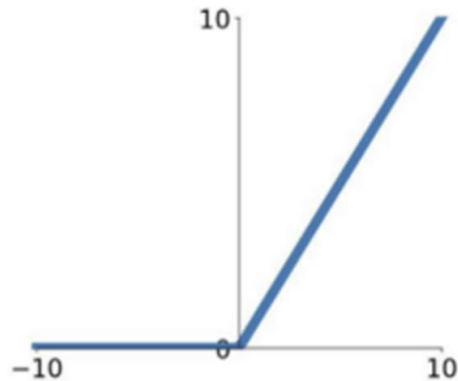
특징

Sigmoid 함수에서 non-zero centered 단점 극복
기울기의 최대값이 1로 더 증가

단점

여전히 Gradient vanishing 문제가 존재

ReLU

$$\max(0, x)$$


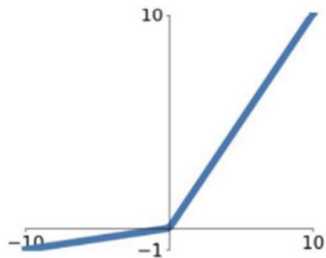
특징

Gradient vanishing 문제를 해결
현재 가장 많은 논문에서 사용되고 있
는 함수

단점

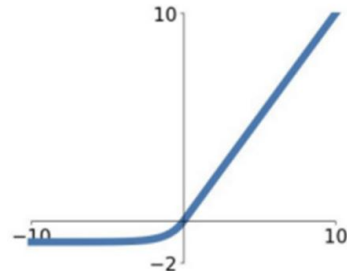
입력 값이 음수일 때는 모든 값이 0으로 수
렴

Leaky ReLU
 $\max(0.1x, x)$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



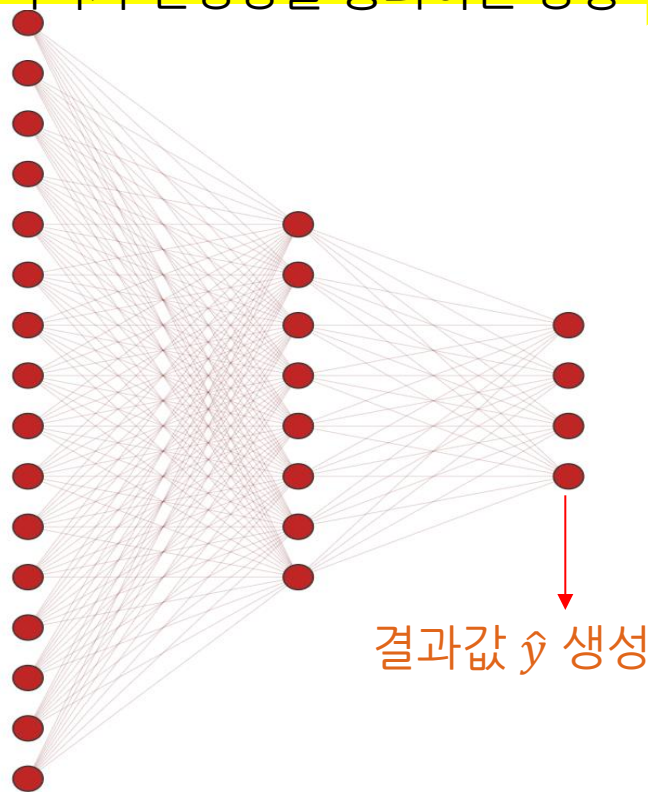
Maxout
 $\max(w_1^T x + b_1, w_2^T x + b_2)$

깊이 들어가기 — 신경망의 핵심: 오차역전파법

데이터 준비가 끝났다면, 이제 신경망이 **오차역전파법(Backpropagation)**으로 어떻게 학습하는지 살펴봄.
핵심은 손실을 기준으로 각 가중치의 기여도를 계산해, 이를 뒤로 전파하며 값을 갱신하는 원리임.

1. 좀 더 복잡한 Task 를 수행하기 위해 신경망 모델 사용
2. 단일 신경망의 단위를 퍼셉트론이라 칭함
3. 단일 퍼셉트론은 선형분류밖에 안됨
4. 비선형 분류를 하기 위해 신경망을 쌓는다. (활성화 함수 작용)
5. 신경망을 쌓는 것은 벡터를 바라보는 시점을 변경하는 것
6. 다층 신경망을 MLP (Multi Layered Perceptron) 라고 부른다!
7. 이러한 MLP 는 어떻게 학습시킬까?
8. 우선 데이터를 신경망에 통과시켜보자... (Feed-Forward 방식)

데이터가 신경망을 통과하는 방향



MLP 를 지난 결과값..

- 비선형 분류가 가능하다는 사실을 알게 됐다!
- Feed Forward 방식에서 $\hat{y}$ 와 실제 y 를 비교하여 오차를 구함
- 비용함수를 정의해서 네트워크가 얼마나 엉망진창인지 판단
- 즉, 우리는 Cost 를 어떻게 줄여 네트워크를 좋게 만들지 알아야함

우리의 가정 $cost(W, b) = \frac{1}{N} \sum_{(x, y) \in D} (y - prediction(x))^2$

1. Cost Function 은 MSE
2. 각 뉴런의 노드와 출력값 0~1 사이 값 (일종의 확률)
3. 효율적인 학습법인 경사 하강법을 어떻게 적용할지 생각하는 임무
4. 16 X 8 X 4 신경망 가정

- 16개 노드의 입력층, 8개 노드의 은닉층, 4개의 노드 출력층
- 각 노드를 $a_i^{(0)}$ 이라 표현 (위 첨자는 층 표현, 아래 첨자는 노드 표현)
- 마지막 층은 $n_L - 1$ (2번 층 5번 노드 $\rightarrow a_5^1$)
- 활성화 함수는 σ , 가중치는 w

Ex) $a_1^1 = \sigma(w_1 a_1^0 + b_1)$ b 는 각 가중치 (각 편향은 노드마다 1개 고정)

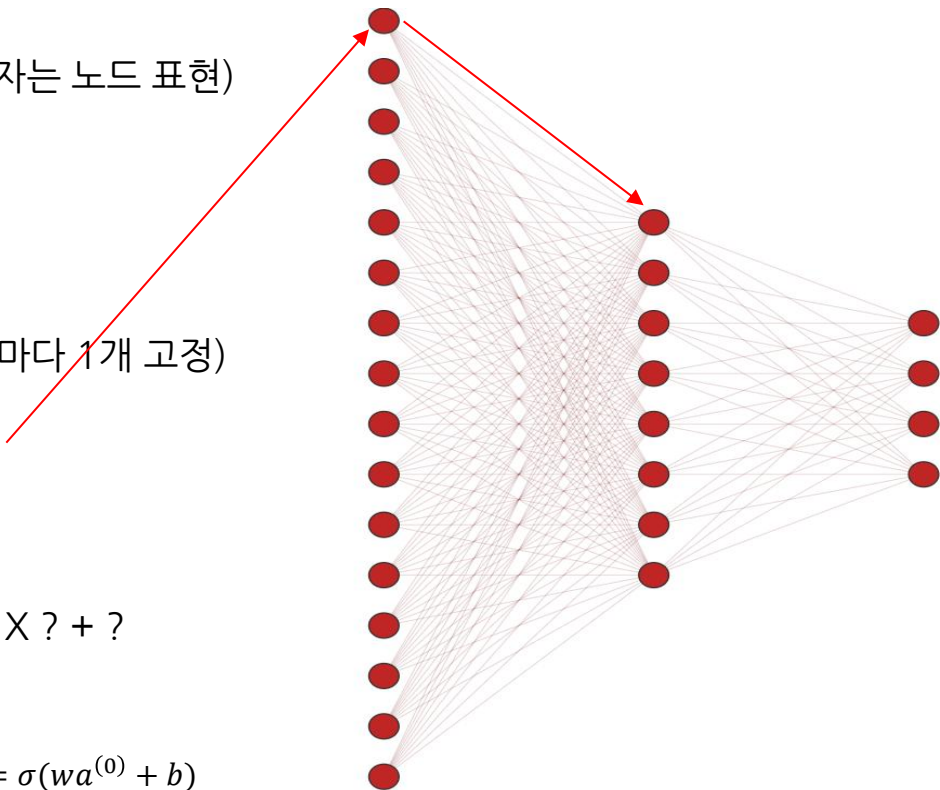
(1층 첫번째 노드에서 2층 첫번째 노드로 향하는 식)

a_1^1 로 향하는 가중치 개수는?

입력층과 은닉층 노드 간의 가중치, 편향의 총 개수는? $? \times ? + ?$

$$\mathbf{a}^{(1)} = \sigma \begin{bmatrix} w_{00} & w_{01} & \cdots & w_{0n} \\ \vdots & & & \\ w_{n0} & w_{n1} & \cdots & w_{nn} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix}$$

$a^1 = \sigma(wa^{(0)} + b)$
행렬로 간단히 표현



Work Flow

- 각각의 가중치를 연결의 세기라고 한번 생각해보자.
- 각 연결의 세기를 조정하면 출력 결과도 달라질 것
- 우리는 신경망이 잘 작동해서 4 개의 출력 노드 중 정답 노드를 강하게 활성화 하길 원함 (출력 노드의 활성화 정도 0~1 은 확신의 정도로 생각하자)

우리의 과제는?

- Cost Function 을 설정하여 목적 갖게 하기.
- Cost 관점에서 입력 변수는 수많은 가중치 행렬 w , 종속 변수는 Cost 값
- 이 값을 최소화 하기위해 가중치를 조정해야 한다.
- 이 과정에서 우리는 경사하강법을 통해 파라미터 공간에서(w 로 이루어진 공간) 점진적으로 Cost Function 의 값이 최솟값과 가까워지도록 하고싶다.
- MSE 의 식을 미분해서 각 w 의 기울기가 특정 점에서(random) 얼마나 되는지 판단하자

효율적인 경사하강법

- 아무리 가중치 w 가 많아도 우리는 각 w 에 대해 Cost Function 을 편미분 하여 각 w 로의 가파른 정도를 알 수 있다. (Gradient Vector)

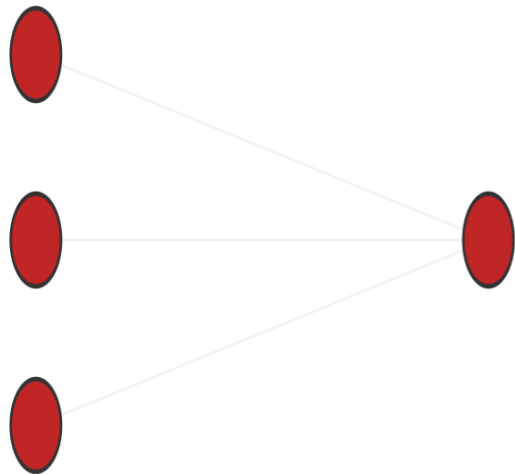
$$\nabla C = \begin{bmatrix} w_0 = 0.5 \\ w_1 = 3.9 \\ w_2 = 0.02 \end{bmatrix}$$

임의의 3X1 신경망에서 Cost Function 에 서
각 가중치에 대한 미분 값 표현

이 식에서 w_1 의 음의 방향으로 한 걸음 이동하는 게 가장 효과적일 것!

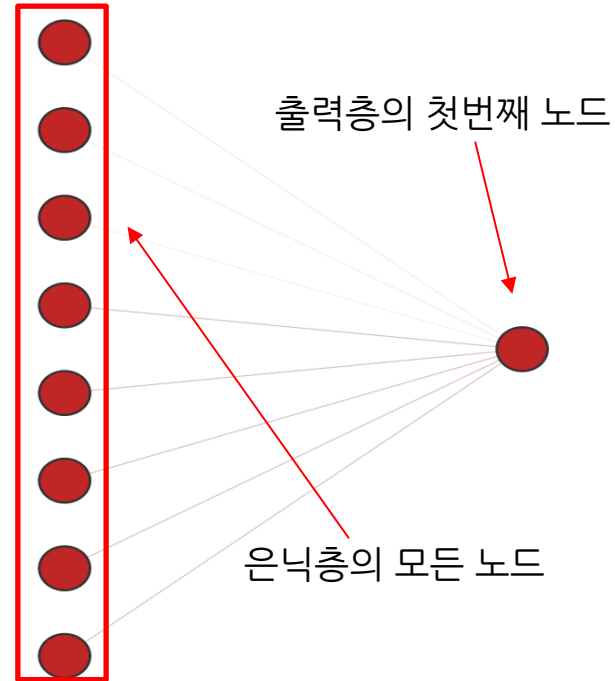
미분해서 나온 각 값이 Cost Function 에 대한 연결의 세기

- 이 과정을 최종 출력층에서 부터 거꾸로 반복하다 보면 모든 가중치에 대한 최종 Cost 에 대한 연결의 세기가 나오지 않을까?



가정한 신경망으로 돌아가보자..

- $C_1 = (y - b)^2$ (출력층 첫번째 노드에 대한 부분적 비용함수!)
- C_1 에 대한 비용함수를 구하기 위해 8차원 공간에서 각 w 에 대한 편미분을 통해 Gradient Vector 구하자!
- 출력층 노드가 4개라고 했으므로 C_1, C_2, C_3, C_4 에 대해 모두 구하자.
- 총 네 개의 부분 비용함수를 더한 후 4로 나눠 각 가중치의 평균 구하기
- 평균을 내어 최종적인 총 비용함수 C 에 대한 가중치들의 연결의 세기를 구한 후 알맞게 이동한다.



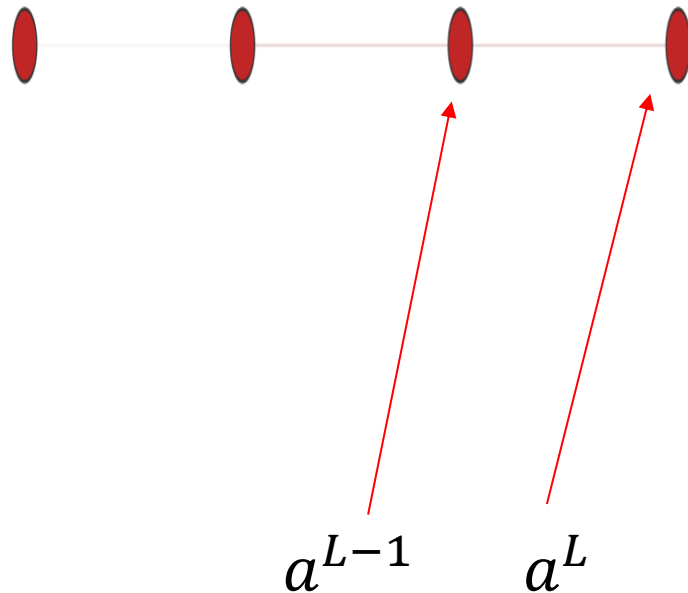
수학적으로 살펴보기

1 1 1 1 신경망 가정해서 살펴보자

- a^L 은 출력층의 노드
- 비용함수 $C_0 = (a^L - y)^2$
- $a^L = \sigma(W^L a^{L-1} + b^L)$ (W 는 a^L 로 향하는 가중치)
- $z^L = w^L a^{L-1} + b^L$ 이라 정의하기

그렇다면 C_0 는 W^L 에 대해 얼마나 민감할까? : W 에 대해 미분!

- $w^L \rightarrow z^L \rightarrow a^L$



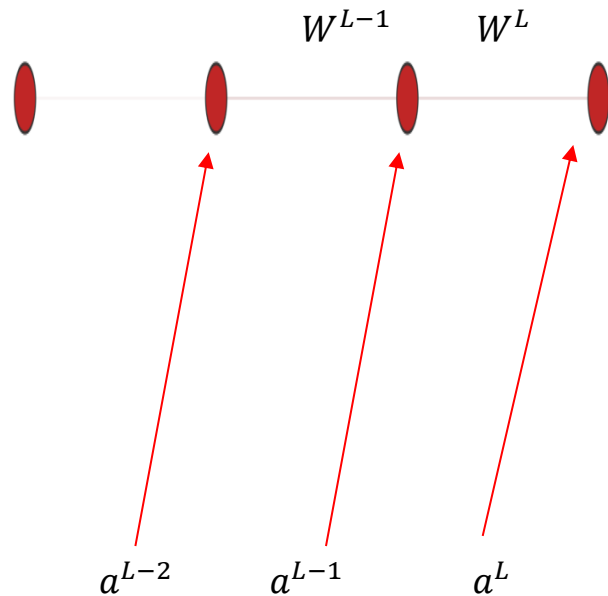
수학적으로 살펴보기

- $\frac{\partial C_0}{\partial W^L} = \frac{\partial Z^L}{\partial W^L} \times \frac{\partial a^L}{\partial Z^L} \times \frac{\partial C_0}{\partial a^L}$
- $\frac{\partial C_0}{\partial a^L} = 2(a^L - y)$
- $\frac{\partial a^L}{\partial Z^L} = \sigma'(z^L)$
- $\frac{\partial Z^L}{\partial W^L} = a^{L-1}$

여기서 a^{L-1} 은 a^{L-2} 의 영향을 받는 값이다.

따라서 이 과정을 입력층 노드에 까지 진행 (밑으로 갈수록 식이 길다)

$$\frac{\partial C_0}{\partial W^{L-1}} = \frac{\partial Z^{L-1}}{\partial W^{L-1}} \times \frac{\partial a^{L-1}}{\partial Z^{L-1}} \times \frac{\partial C_0}{\partial a^{L-1}} \rightarrow \frac{\partial C_0}{\partial a^{L-1}} = \frac{\partial C_0}{\partial a^L} \times \frac{\partial a^L}{\partial Z^L} \times \frac{\partial Z^L}{\partial a^{L-1}}$$



→ 이미 구함, 전 w 참조

수학적으로 살펴보기

$$C_0 = (a_0^L - y_0)^2, C_1 = (a_1^L - y_1)^2$$

$$\frac{\partial C_0}{\partial w_0^L} = \frac{\partial z_0^L}{\partial w_0^L} \times \frac{\partial a_0^L}{\partial z_0^L} \times \frac{\partial C_0}{\partial a_0^L}$$

$$\frac{\partial C_1}{\partial w_1^L} = \frac{\partial z_1^L}{\partial w_1^L} \times \frac{\partial a_1^L}{\partial z_1^L} \times \frac{\partial C_1}{\partial a_1^L}$$

$$\frac{\partial C_0}{\partial w^L} = \frac{\partial C_0}{\partial w_0^L} + \frac{\partial C_0}{\partial w_1^L} \quad (\text{일반화 할 때 sigma 이용})$$

$$\frac{\partial C_0}{\partial w_0^{L-1}} = \frac{\partial z_0^{L-1}}{\partial w_0^{L-1}} \times \frac{\partial a_0^{L-1}}{\partial z_0^{L-1}} \times \frac{\partial C_0}{\partial a_0^{L-1}}$$

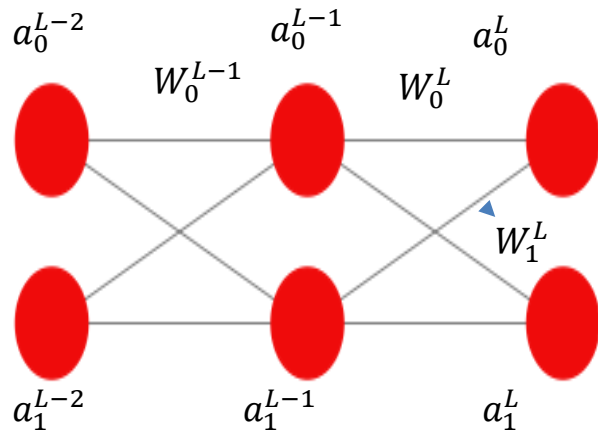
$$a^L = \sigma(W^L a^{L-1} + b^L)$$

$$z^L = w^L a^{L-1} + b^L$$

$$C_0 = (a^L - y)^2$$

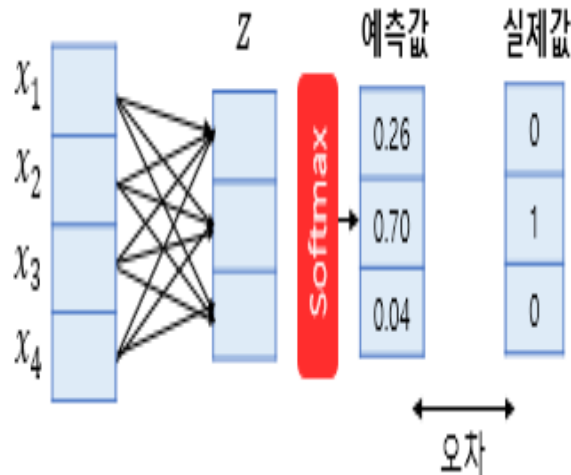
$$a^{L-1} = \sigma(W^{L-1} a^{L-2} + b^{L-1})$$

$$z^{L-1} = w^{L-1} a^{L-2} + b^{L-1}$$



출력층에서 일어나는 일

- 사실 출력층 노드는 그 자체로 정답 값과 비교되지 않음
- Softmax 라는 함수를 거쳐 각 정답 노드에 얼마만큼의 영향력이 있는지 살펴 봄
- $Softmax(t_i) = \frac{\exp(t_i)}{\sum_{j=1}^K \exp(t_j)}$ 의 형태
- K 개의 클래스에서 i 번 원소가 정답일 확률
- 그렇다면 정답인지 아닌지에 대한 성능을 판단은 어떻게 할 것인가?



역전파 전체 흐름 정리

순전파

손실 계산

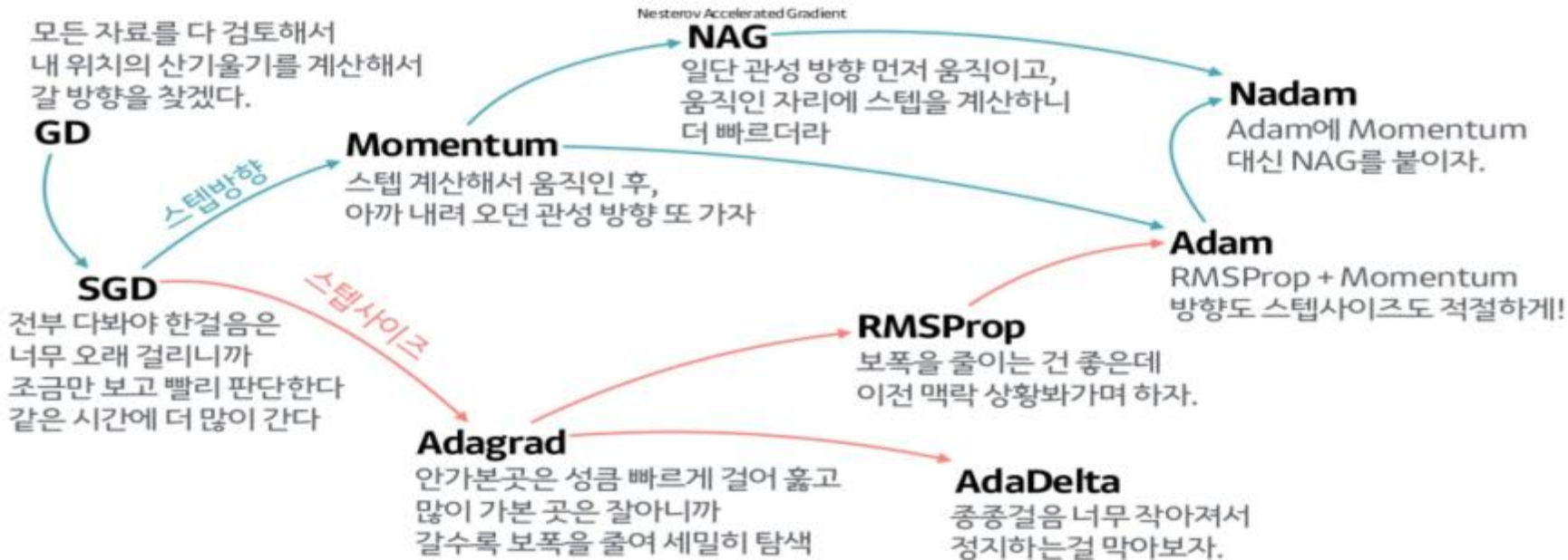


가중치 업데이트

역전파

이 4단계를 **에포크(Epoch)**마다 반복함. 에포크가 증가할수록 손실이 감소하고 모델 정확도가 향상됨. 딥러닝 학습의 전체 흐름임.

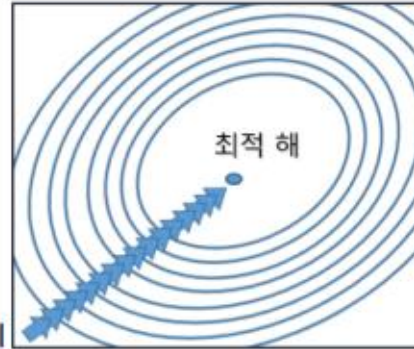
산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



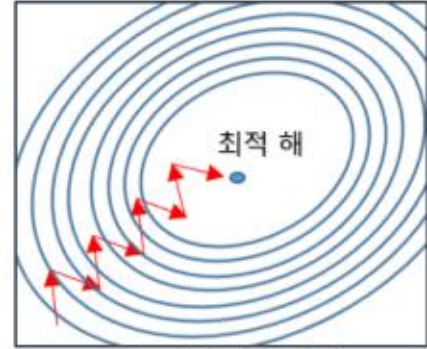
SGD(Stochastic Gradient Descent)

Gradient Descent를 전체 데이터(Batch)가 아닌
일부 데이터의 모음(Mini-Batch)를 사용하는 방법

- 전체 입력 데이터를 통해 기울기를 구하는 것이 아닌
일부 데이터만을 사용하여 학습 속도 증가



경사 하강법



확률적 경사 하강법

단점

- 미니 배치를 기준으로 학습하기 때문에 학습 효율 감소
- 여전히 극소점에 수렴할 수 있다는 문제점 존재
- 안장점과 같은 특수한 데이터에서는 오히려 수렴 속도 감소

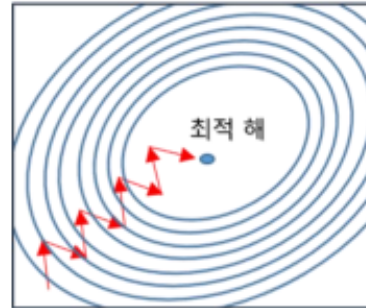
$$W(t+1) = W(t) - \alpha \frac{\partial}{\partial w} Cost(w)$$

Momentum

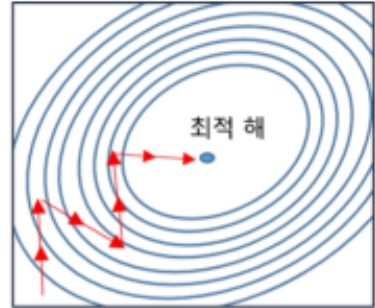
- GD에 관성을 더해 주는 방법
- 이전의 가중치의 수정 방향을 참고하여 갱신 방향을 결정
- 기존의 갱신 방향을 참고하여 진동 횟수 감소
- 이전의 갱신 값 고려하여 더욱 빠르게 갱신 가능
- 기울기가 0일 때 학습이 종료되지 않기 때문에 극솟값이 아닌 최소값에 수렴 가능

단점

- 여전히 진동하기 때문에 최적 해에 직선으로 다가가지 못함
- 이전 방향을 그대로 참고하기 때문에 오히려 최소값을 지나칠 수 있음



확률적 경사 하강법



모멘텀

$$V(t) = m * V(t - 1) - \alpha \frac{\partial}{\partial w} Cost(w)$$

$$W(t + 1) = W(t) + V(t)$$

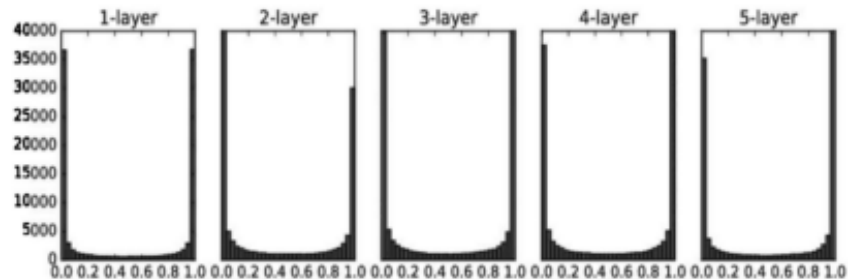
가중치 초기화(Initialization)

모델을 본격적으로 학습하기 전, 모델에 존재하는 가중치를 초기화하는 것
가중치 최적화와 함께 모델 학습에 있어서 중요한 요소 중 하나

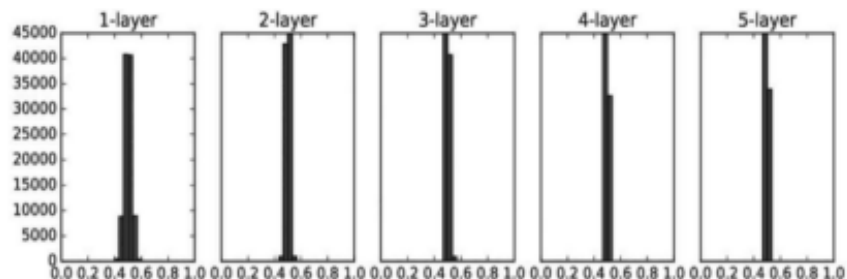
Random Initialization

정규분포를 활용하여 가중치를 초기화하는 방법
평균과 표준편차를 설정하여 가중치를 초기화

평균을 0, 표준편차를 1으로 세팅했을 때



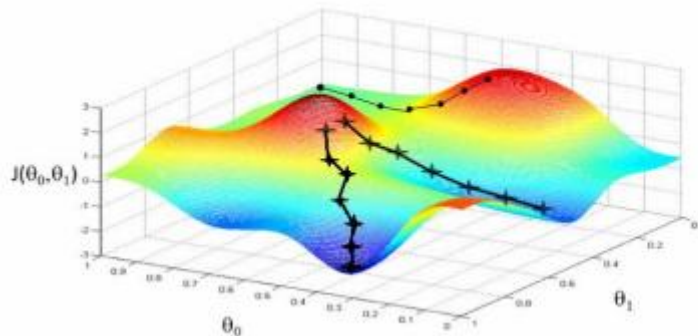
평균을 0, 표준편차를 0.01으로 세팅했을 때



가중치 초기화의 중요성

가중치는 동일한 값이 아닌 랜덤하게 설정해야 한다.
 모든 가중치가 동일하다면 시간만 오래 걸릴 뿐, 학습의 효율은 매우 떨어지게 된다
 모든 가중치를 제대로 활용하기 위해선 효율적인 초기값 세팅이 필요

가중치의 초기값 위치에 따른 어떤 변화가 있을까?



가중치의 최초의 위치가 추후에 업데이트되는 방향을 결정
 초기값 위치가 극솟값으로 수렴하거나 최솟값으로 수렴하는 지를
 결정하는데 영향을 미침
 최적화 방법과 함께 효율적인 학습을 위해 초기값을 잘 결정해야 함

Xavier Initialization

Layer들의 입력/출력 노드 개수에 따라 파라미터 값을 초기화

Sigmoid함수, tanh함수와 함께 주로 사용되는 초기화 방법

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

He Initialization

ReLU함수에 최적화된 파라미터 초기화 방법

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in}}}$$

2026 데이터+AI 혁신 챌린지
데이터+AI 크리에이터 캠프



감사합니다



과학기술정보통신부

NIA 지능정보원
한국지능정보보호진흥원

1차시 미니퀴즈 : AI 개요와 정보의 데이터화

- Q1. AI 데이터 파이프라인의 단계를 순서대로 옳게 나열한 것은?
- ① 데이터 수집 → 문제 정의 → 학습 → 전처리 → 평가·배포 ② 문제 정의 → 데이터 수집 → 전처리 → 모델 학습 → 평가·배포 ③ 전처리 → 수집 → 문제 정의 → 학습 → 배포 ④ 문제 정의 → 학습 → 수집 → 전처리 → 평가
- Q2. 텍스트를 토큰으로 쪼갠 뒤 숫자 벡터로 변환하는 과정을 무엇이라 하는가?
- ① 정규화 ② 임베딩 ③ 시각화 ④ 크롤링 ⑤ 배포
- Q3. AI, 머신러닝, 딥러닝의 관계로 옳은 것은?
- ① 셋은 서로 독립적인 분야다 ② 딥러닝 ⊃ 머신러닝 ⊃ AI ③ AI ⊃ 머신러닝 ⊃ 딥러닝 ④ 머신러닝 ⊃ AI ⊃ 딥러닝
- Q4. 두 신경망(생성자·판별자)이 경쟁하며 실제와 유사한 데이터를 만들어내는 모델은?
- ① CNN ② RNN ③ GAN ④ SVM ⑤ KNN

2차시 미니퀴즈 : 통계·분포·전처리

- Q1. 이상치(Outlier)에 가장 강건한 중심 경향성 지표는?
① 평균 ② 중앙값 ③ 분산 ④ 표준편차 ⑤ 첨도
- Q2. 평균과 분산이 모두 λ 로 같고, 단위 시간당 사건 발생 횟수를 모델링하는 분포는?
① 정규분포 ② 이항분포 ③ 포아송분포 ④ 기하분포 ⑤ 베타분포
- Q3. 피처의 왜도(Skewness)가 +2.5로 나타났다. 가장 우선적으로 고려할 처리는?
① 그대로 사용 ② StandardScaler만 적용 ③ 로그/Box-Cox 변환 ④ 피처 삭제
- Q4. IQR 방법에서 이상치로 판단하는 기준은?
① 평균 $\pm 1\sigma$ 밖 ② 중앙값 $\pm 2 \times \text{IQR}$ 밖 ③ $Q1 - 1.5 \times \text{IQR}$ 미만 또는 $Q3 + 1.5 \times \text{IQR}$ 초과 ④ 최솟값과 최댓값
- Q5. 결측 비율이 50%를 넘는 컬럼에 대한 일반적인 처리 전략은?
① 평균 대체 ② 최빈값 대체 ③ 컬럼 삭제 ④ KNN 대체 ⑤ 무시

3차시 미니퀴즈 : 평가지표·경사하강법·퍼셉트론

- Q1. 실제 Positive인 것들 중에서 모델이 Positive로 맞게 예측한 비율은?
 - ① 정확도(Accuracy) ② 정밀도(Precision) ③ 재현율(Recall) ④ F1 Score ⑤ AUC
- Q2. 암 환자를 '건강하다'고 잘못 판정하는 오류의 유형은?
 - ① Type I 오류(FP) ② Type II 오류(FN) ③ TP ④ TN
- Q3. 단층 퍼셉트론이 풀 수 없는 것으로 증명된 논리 연산은?
 - ① AND ② OR ③ NAND ④ XOR ⑤ NO
- Q4. 경사하강법에서 한 걸음의 '보폭'을 결정하는 하이퍼파라미터는?
 - ① 배치 크기 ② 에포크 ③ 학습률(Learning Rate) ④ 가중치 ⑤ 편향
- Q5. 클래스 불균형이 심한 데이터에서 가장 신뢰하기 어려운 지표는?
 - ① 정확도 ② 재현율 ③ 정밀도 ④ F1 Score

4차시 미니퀴즈 : 활성화 함수·오차역전파·옵티마이저

- Q1. $f(x) = \max(0, x)$ 로 정의되는 활성화 함수는?
• ① Sigmoid ② tanh ③ ReLU ④ Softmax ⑤ ELU
- Q2. Sigmoid를 깊은 신경망에 쓸 때, 미분 최대값이 0.25라서 발생하는 문제는?
• ① Dying ReLU ② Gradient Vanishing ③ 과적합 ④ 대칭성 문제
- Q3. 오차역전파에서 가중치의 기울기를 층별로 이어 계산할 때 사용하는 미분 법칙은?
• ① 곱의 법칙 ② 몫의 법칙 ③ 연쇄법칙(Chain Rule) ④ 로피탈의 정리
- Q4. 이전 이동 방향을 관성처럼 누적하여($V(t) = m \cdot V(t-1) - \alpha \nabla C$) 진동을 줄이는 옵티마이저는?
• ① SGD ② Momentum ③ Adagrad ④ RMSProp ⑤ AdaDelta
- Q5. ReLU 계열 활성화 함수와 짝을 이루는 가중치 초기화 방법은?
• ① Xavier 초기화 ② 0으로 초기화 ③ He 초기화 ④ 균등 초기화

2026 데이터+AI 혁신 챌린지

데이터+AI

크리에이터
캠프

대학부

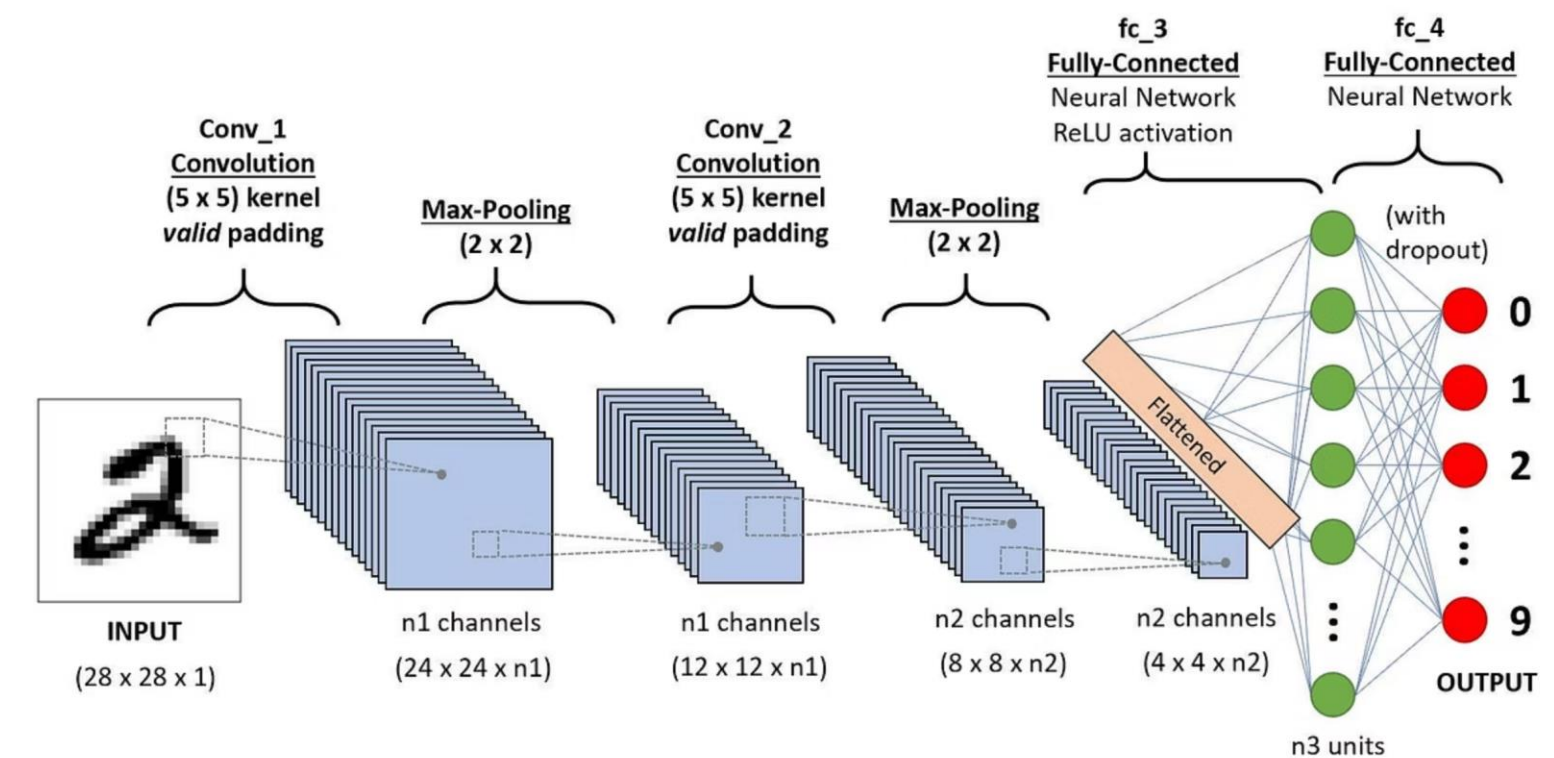


과학기술정보통신부

NIA 지능정보원
한국지능정보사회진흥원

CNN — 합성곱 신경망

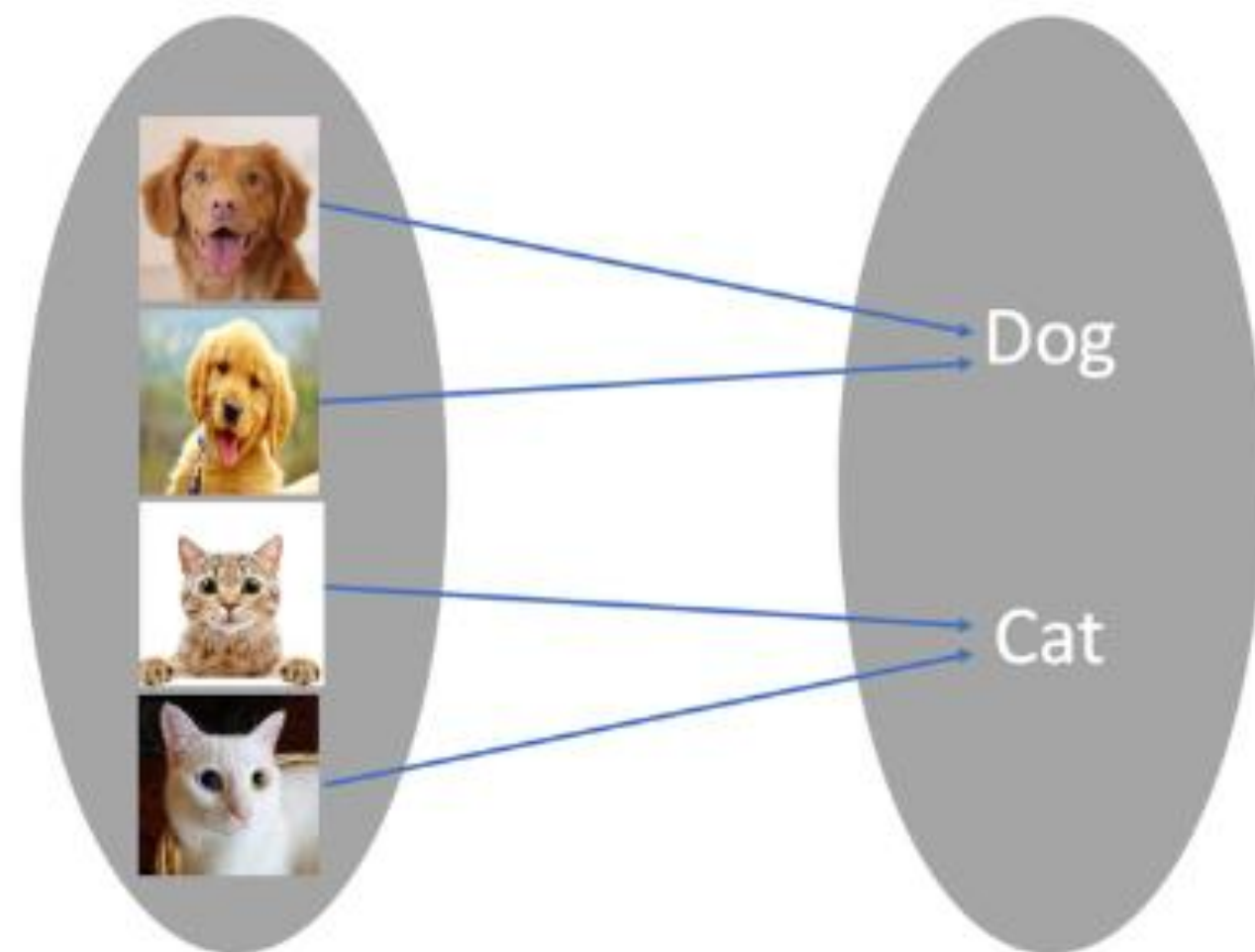
이미지를 "보는" 방법 — 공간 정보를 보존하며 특징을 계층적으로 학습하는 신경망



컴퓨터 비전은 컴퓨터가 이미지, 비디오 등의 입력에서 정보를 추출할 수 있도록 하는 인공지능(AI)의 한 분야



주요 과제 : 이미지 분류



4 → 4	2 → 2	3 → 3
4 → 4	9 → 9	0 → 0
5 → 5	7 → 7	1 → 1
9 → 9	0 → 0	3 → 3
6 → 6	7 → 7	4 → 4

주요 과제 : 이미지 분할

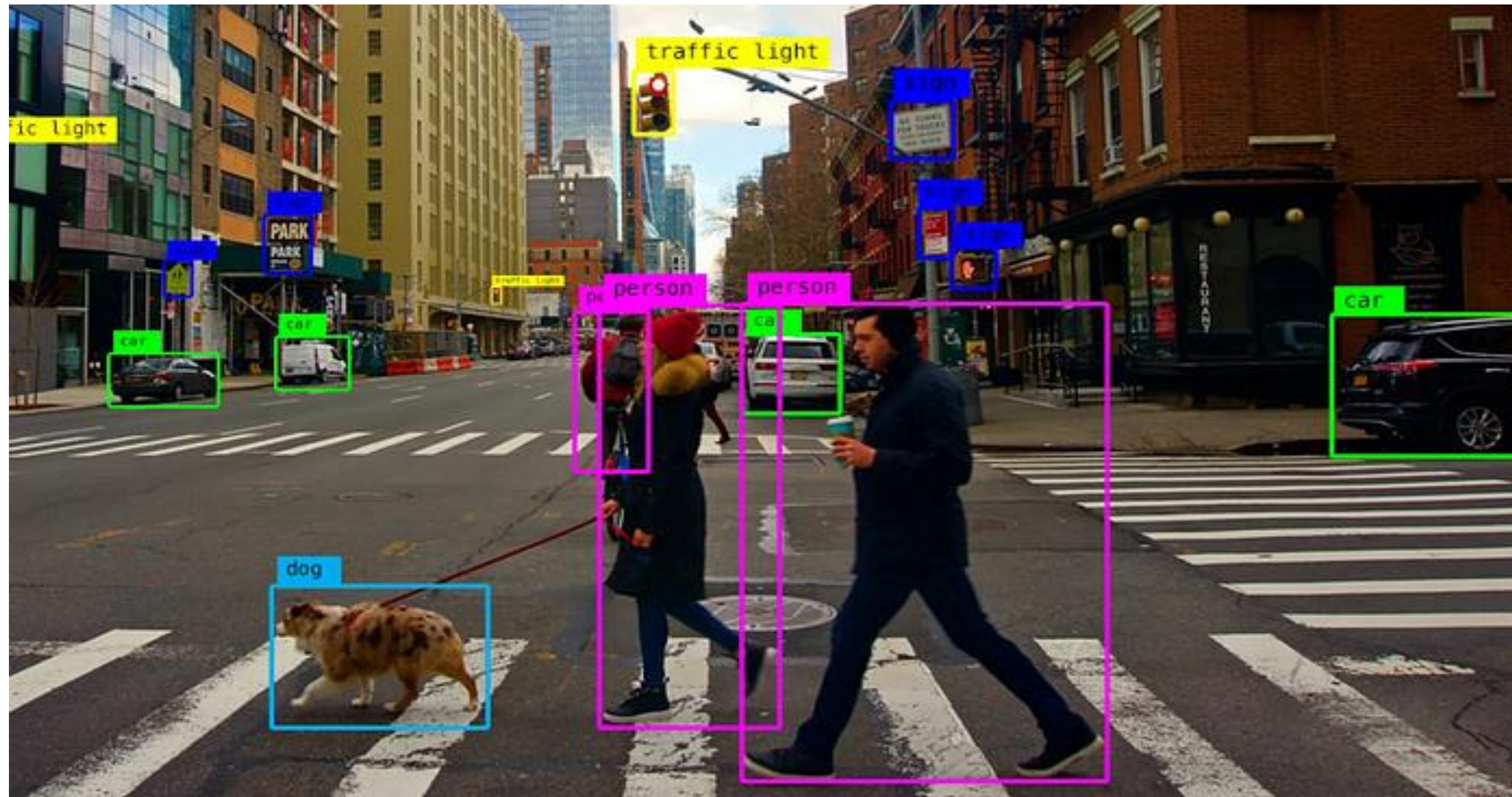


Semantic Segmentation

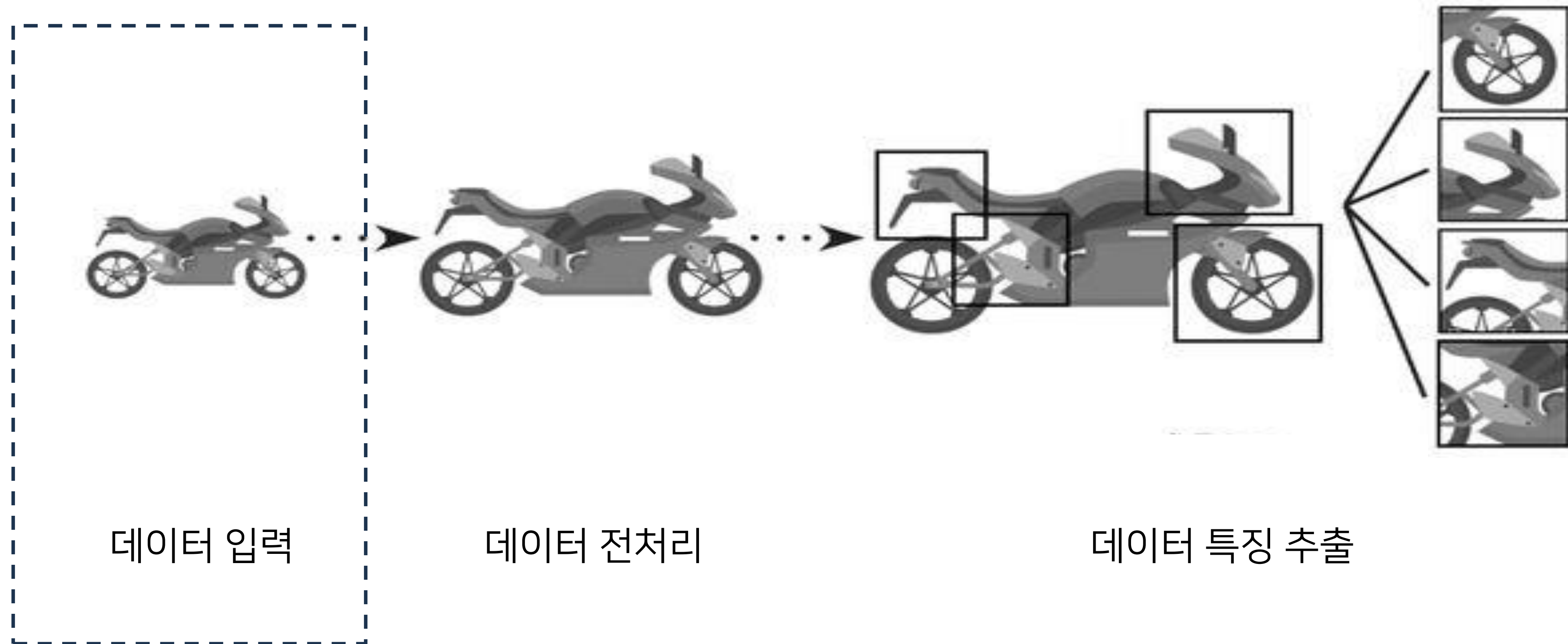


Semantic Instance Segmentation

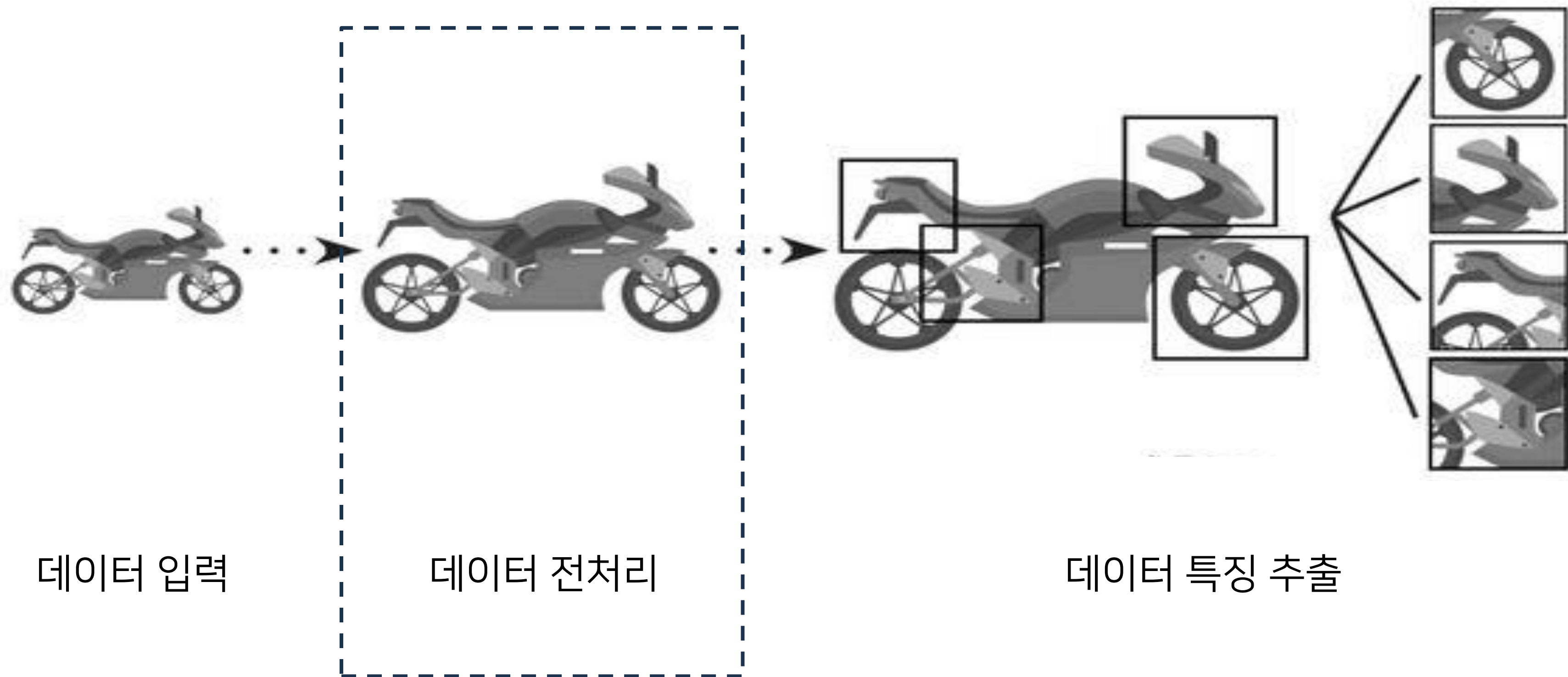
주요 과제 : 객체 탐지



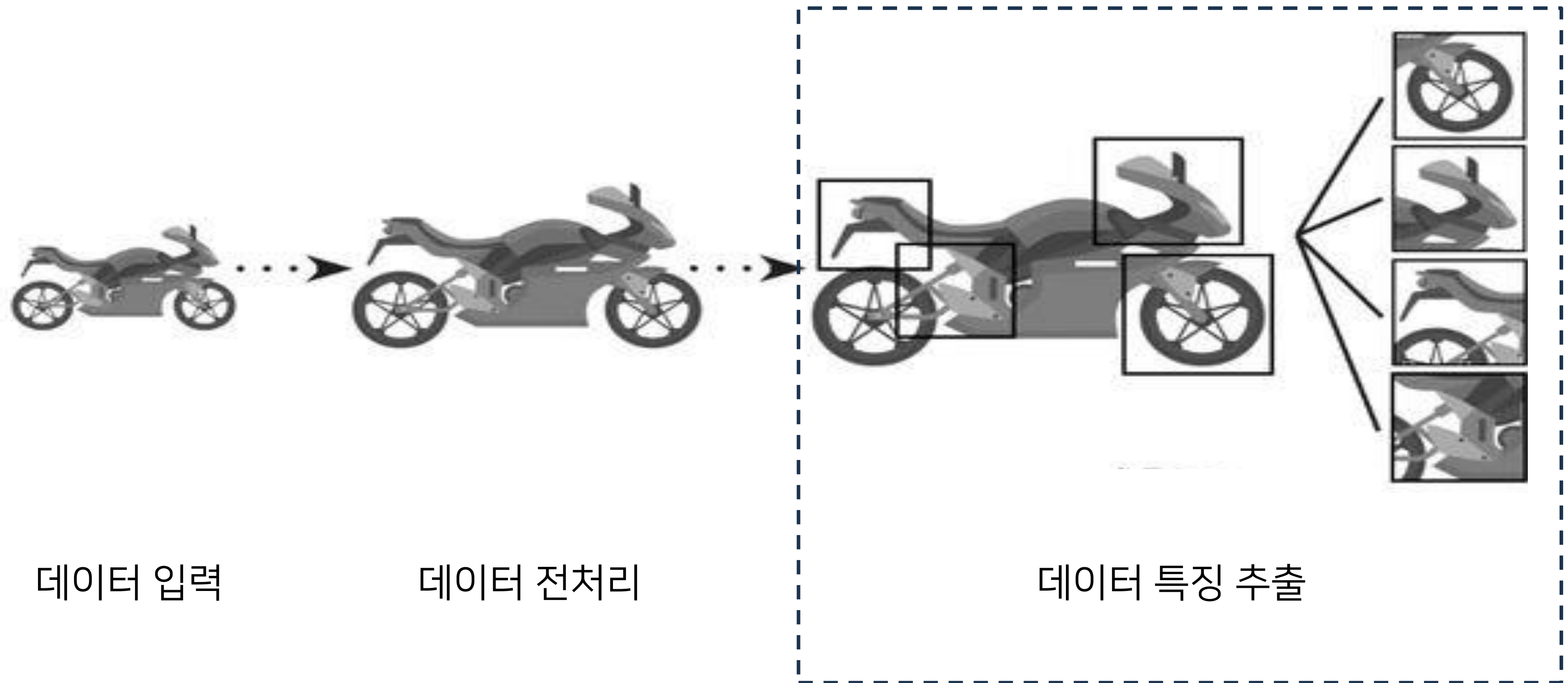
데이터 입력 : 이미지 텐서화



데이터 전처리 : Resizing, Normalization, etc.



데이터 특징 추출 : CNN, RESNET, etc.

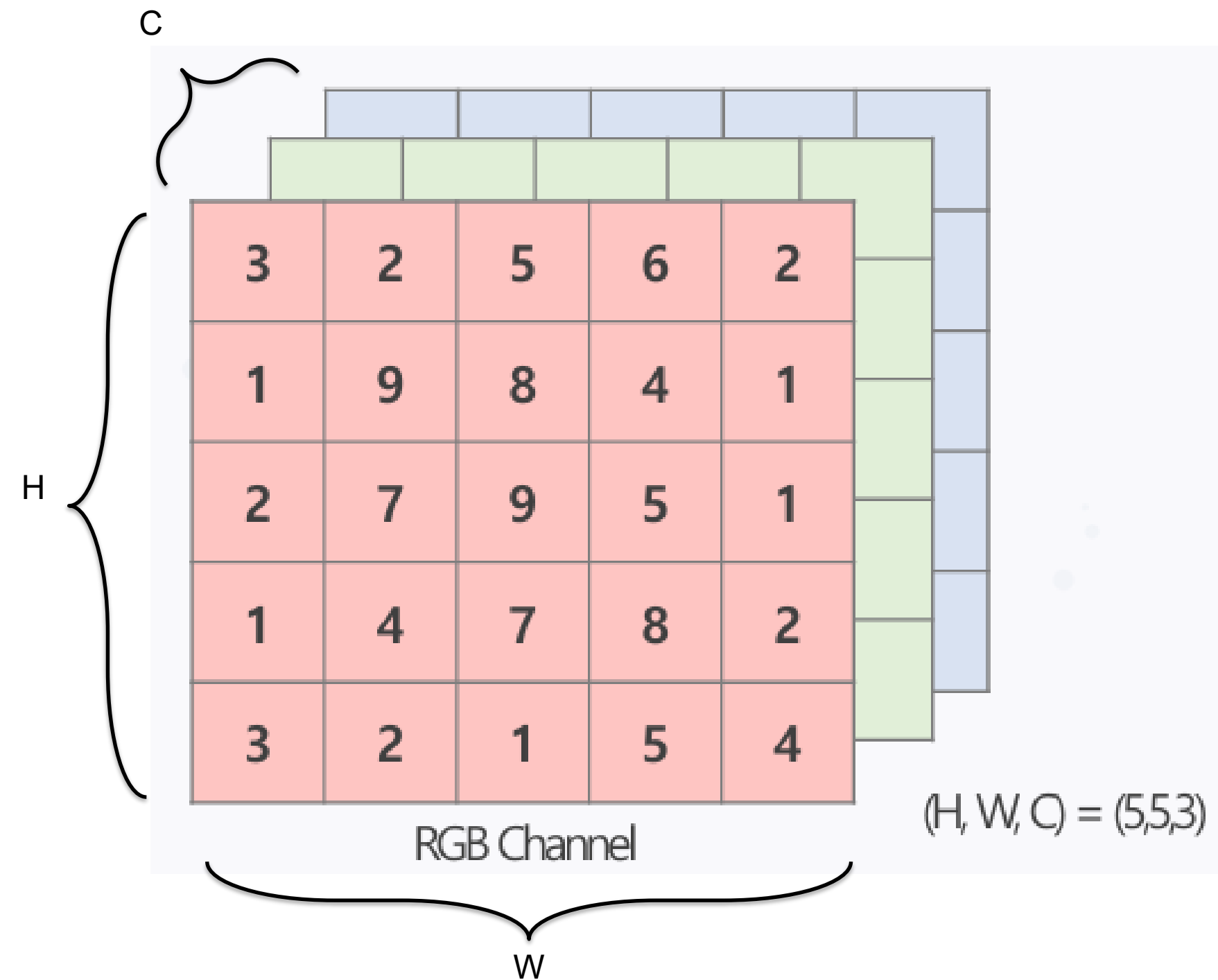


이미지를 컴퓨터가 인식할 수 있는 형태로 수치화(텐서화) 하는 작업은 필수적인 과정

- 이미지 데이터들은 (H, W, C) 형태의 텐서로 이루어짐
- RGB 형태의 이미지는 H x W의 2차원 텐서가 3개 쌓인 형태
- 각 픽셀은 0~255의 값을 가짐



28 x 28
784 pixels



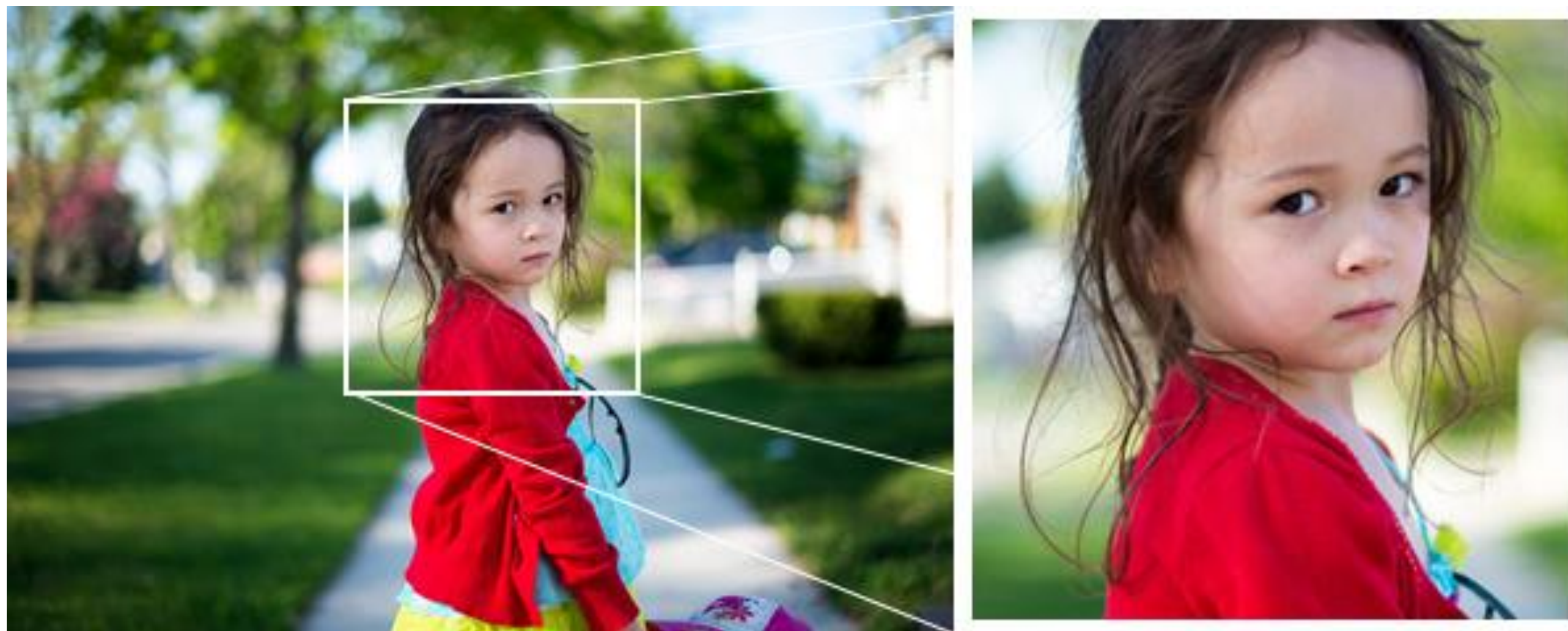
이미지를 컴퓨터가 인식할 수 있는 형태로 수치화(텐서화) 하는 작업은 필수적인 과정

- 모델 학습 안정성을 위해 각 픽셀의 값을 0에서 1사이로 scaling 하는 과정이 필요
- Pytorch tensor 변환 시 자동으로 이루어짐

$$Scaling : \{0, 255\} \rightarrow \{0, 1\}$$

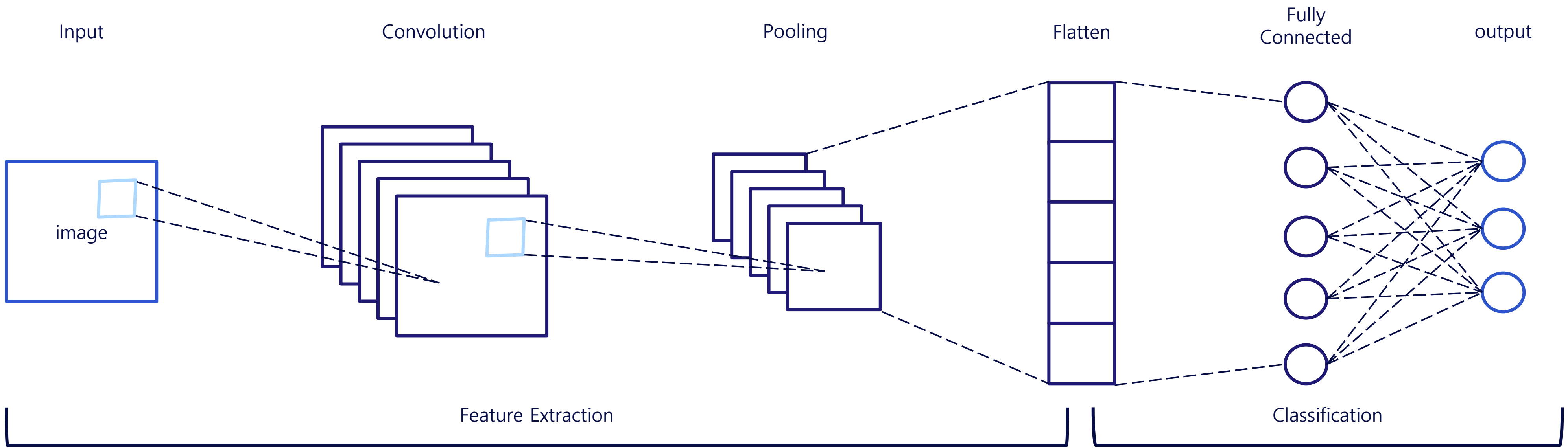
데이터 전처리는 원시 데이터를 분석과 모델링에 적합하도록 정리하고 변환하는 작업

학습 효율 및 정확도 향상을 위해 진행하는 것으로 경우에 따라 모델 성능에 막대한 영향을 끼침



CNN(Convolution Neural Network)

Convolution 연산을 통해 이미지의 지역 정보(Region Feature, Graphic Feature)를 학습하는 Neural Network



CNN(Convolution Neural Network)

공간 정보가 중요

- 공간적으로 가까운 픽셀은 값이 비슷하거나 이미지의 RGB 채널은 서로 밀접하게 관련됨

CV(Computer Vision)

컴퓨터를 통해 이미지, 비디오 등에서 정보를 추출할 수 있도록 하는 인공지능(AI)의 한 분야



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



Red



Green



Blue



RGB color

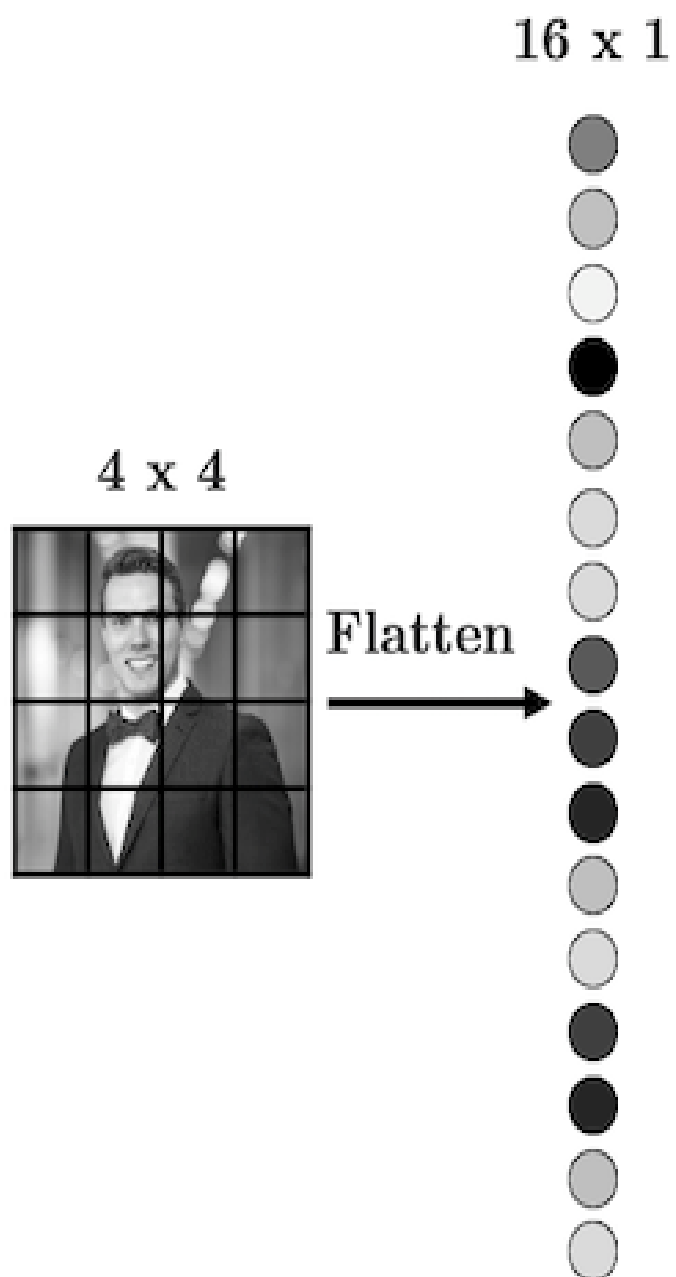
CNN(Convolution Neural Network)

MLP(Multi Layer Perceptron)

CNN(Convolution Neural Network)

이미지 픽셀 값을 Flatten 시켜 Input으로 사용
- 데이터의 **형상 정보 유지 못 함**

이미지의 지역 정보(Region Feature)를 Input으로 사용
- 데이터의 **형상 정보 유지함**



CNN(Convolution Neural Network)

기본적인 구조

Convolution Layer

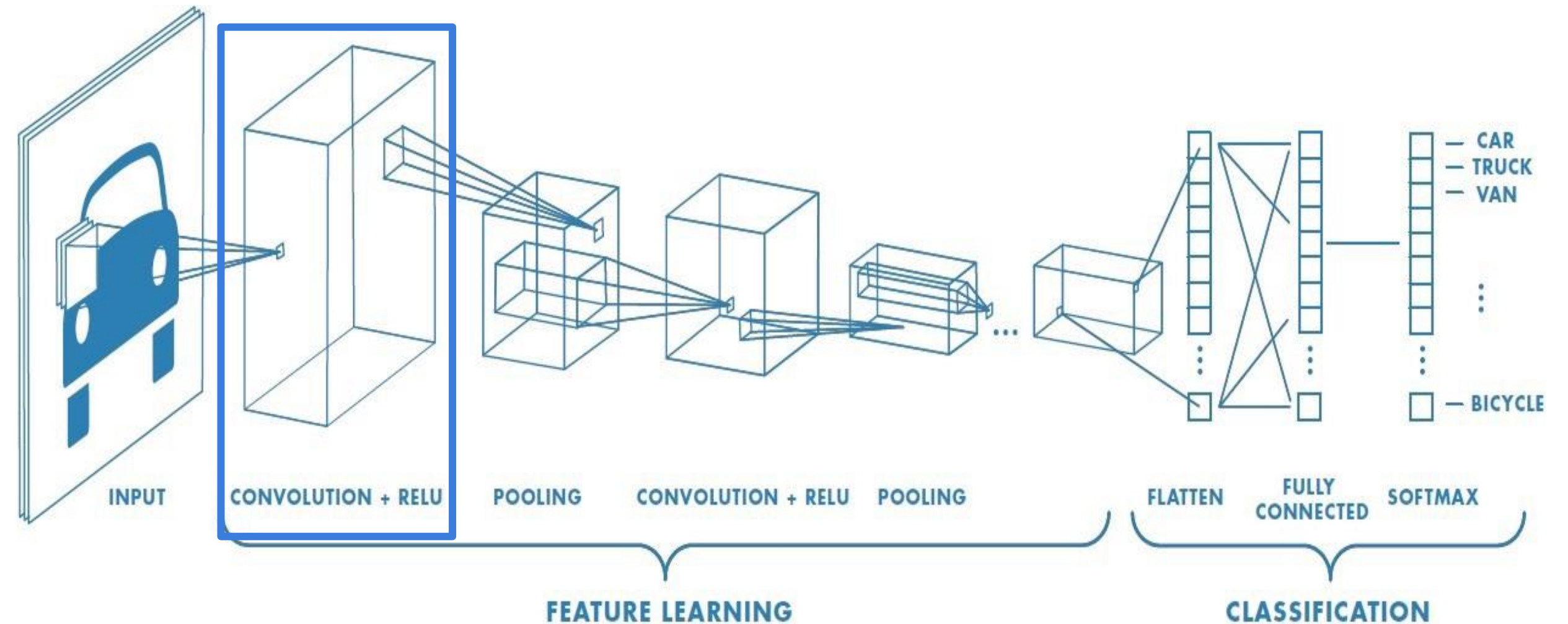
- Region Feature를 뽑아내기 위한 계층

Pooling Layer

- Feature Dimension을 줄이기 위한 계층

Fully Connected Layer

- 최종적인 분류를 위한 계층

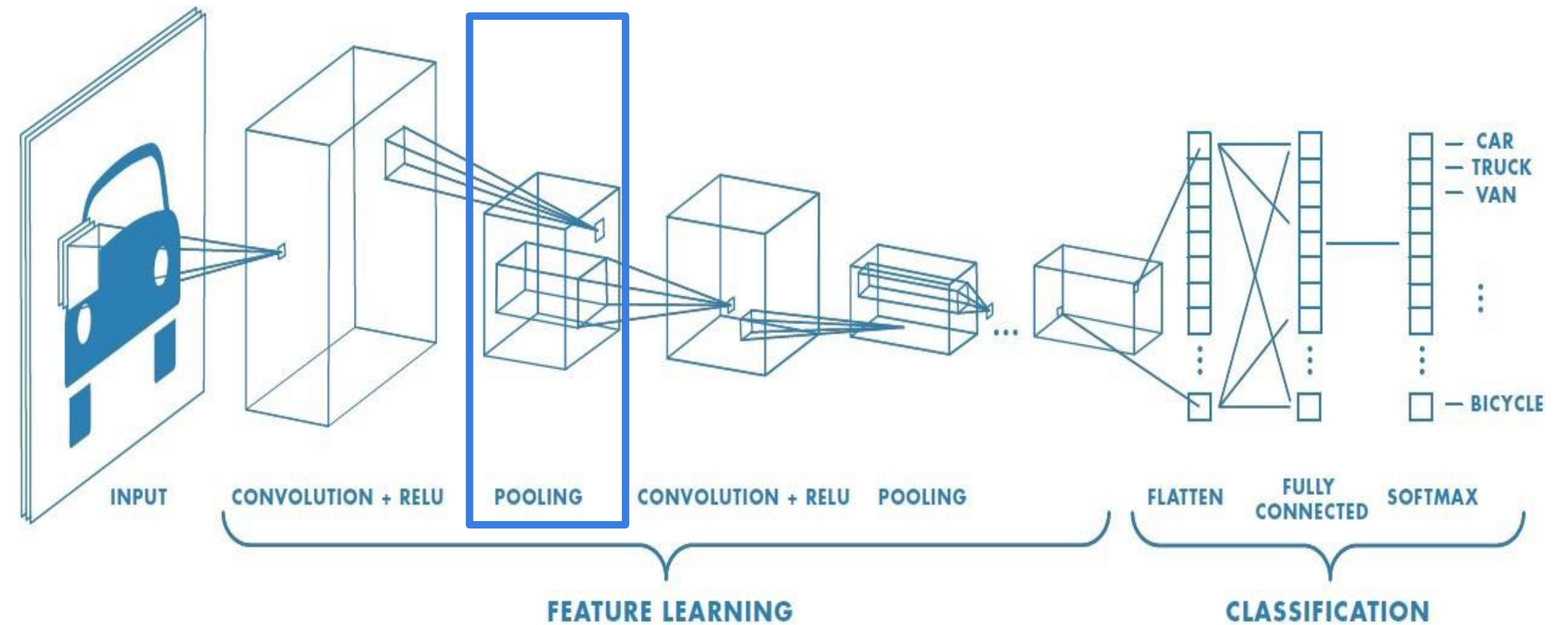


기본적인 구조

Convolution Layer
- Region Feature를 뽑아내기 위한 계층

Pooling Layer
- Feature Dimension을 줄이기 위한 계층

Fully Connected Layer
- 최종적인 분류를 위한 계층



기본적인 구조

Convolution Layer

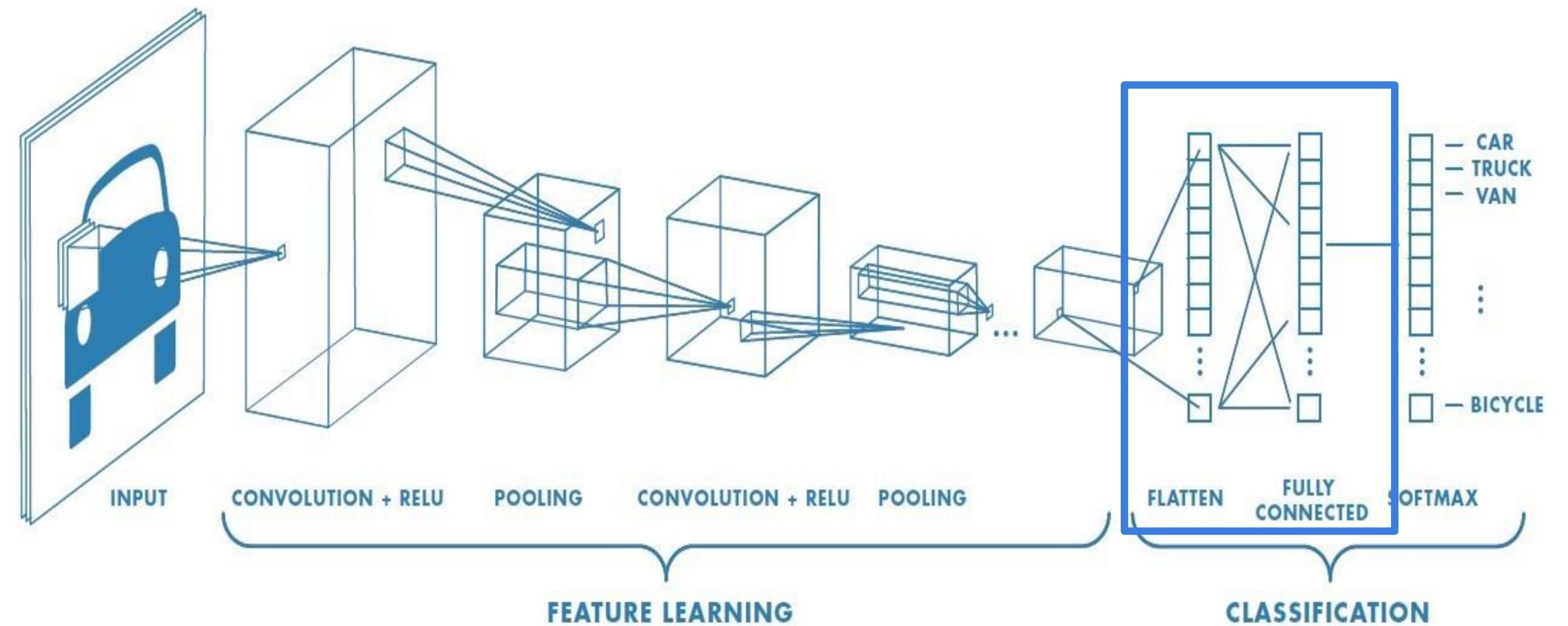
- Region Feature를 뽑아내기 위한 계층

Pooling Layer

- Feature Dimension을 줄이기 위한 계층

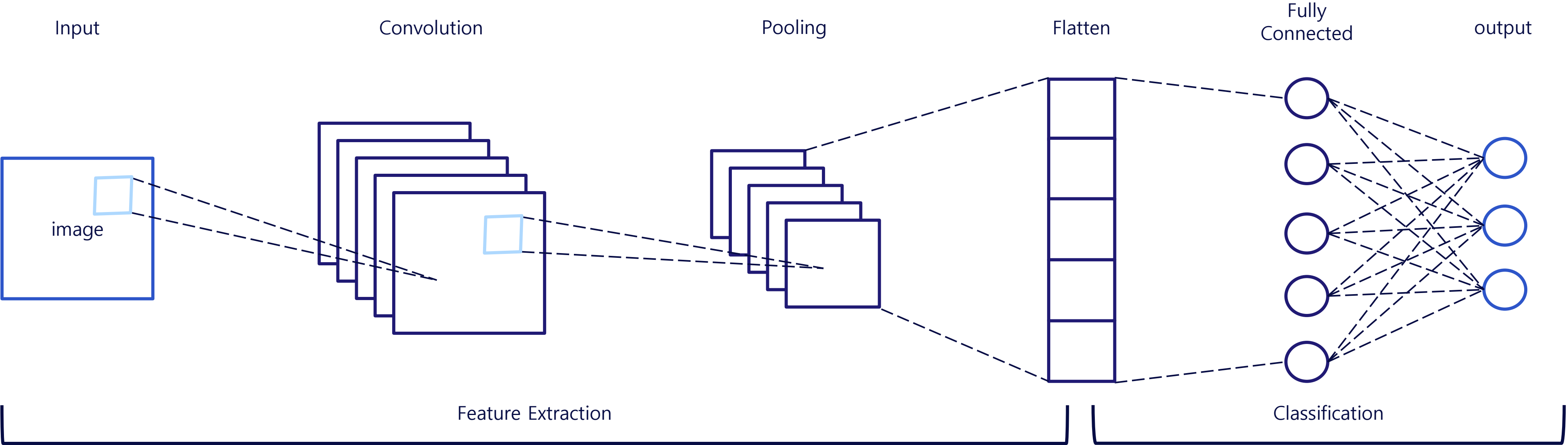
Fully Connected Layer

- 최종적인 분류를 위한 계층



Convolution Layer

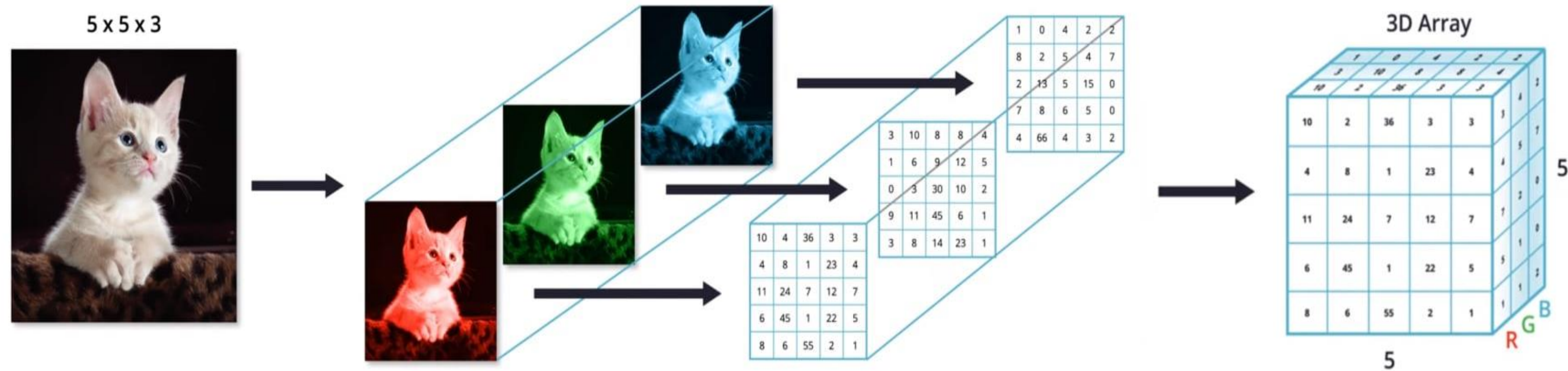
Input과 Filter간의 Convolution 연산을 수행하는 Layer



Channel

흑백 이미지는 2차원 형상의 데이터로, 3차원 형상으로 표현하면 1개의 채널로 구성
- ex. (5, 5, 1)

컬러 이미지는 각 픽셀을 RGB 3개의 실수로 표현한 3차원 형상의 데이터로, 3개의 채널로 구성
- ex. (5, 5, 3)

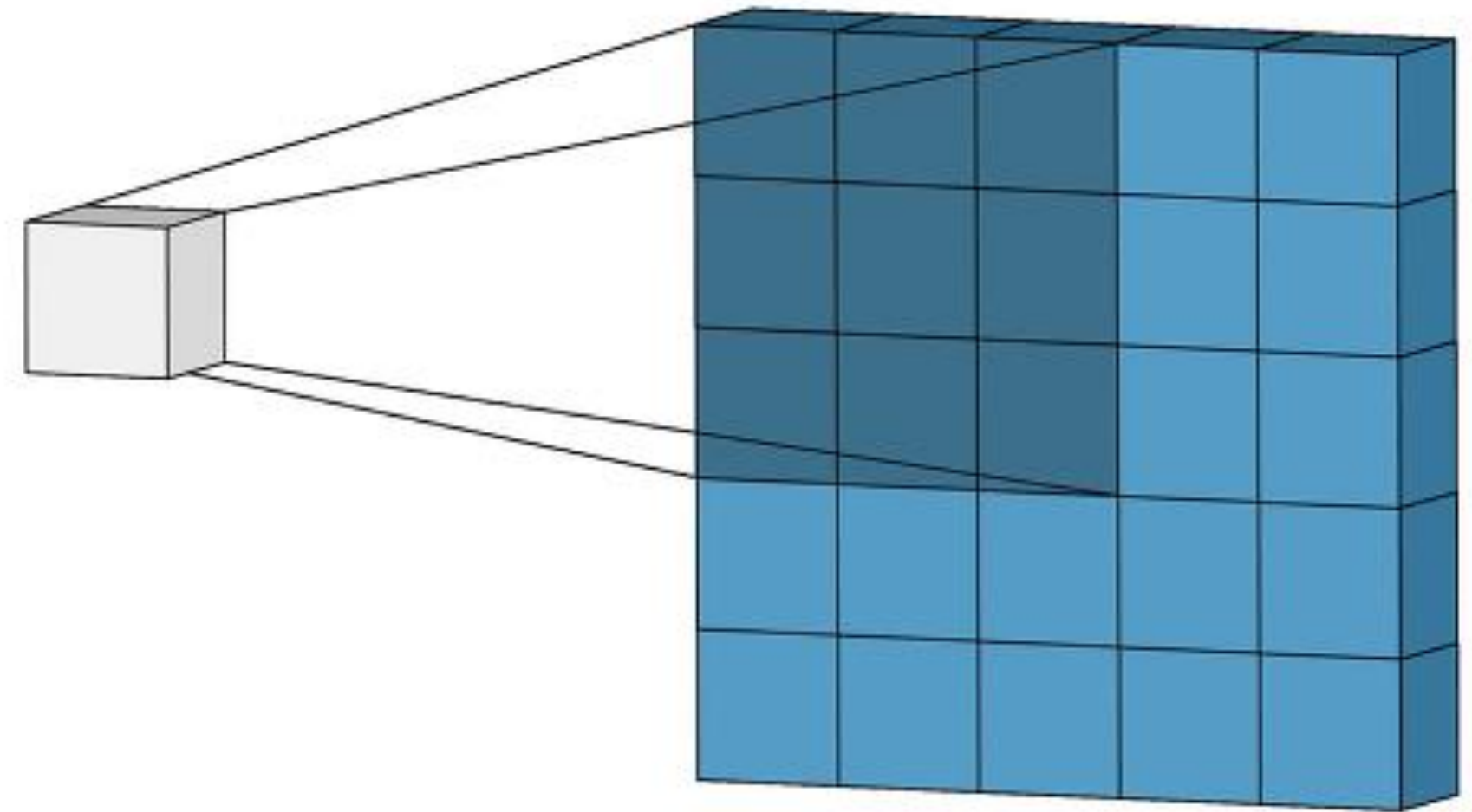


Filter(=Kernel)

이미지의 Region별 Feature를 찾아내기 위한 파라미터

- 이미지의 각 픽셀별로 weight를 설정하면 엄청나게 많은 수의 파라미터가 필요

일반적으로 (3, 3), (4, 4)와 같은 정사각행렬로 정의함



Convolution 연산

Filter를 일정 간격만큼 이동하여 Input과 대응하는 위치의 원소끼리 곱한 후 그 총합을 구하는 연산

1	2	3	0	2	1
0	3	1	2	1	4
4	1	2	1	0	3
3	2	3	2	1	0
4	0	2	3	1	3
2	1	1	1	0	3

Input



1	2	0
2	3	1
1	0	1

Filter



21	21	14	19
25	18	16	11
27	23	20	14
20	19	22	20

Output

ex) $1 \times 1 + 2 \times 2 + 3 \times 0 + 0 \times 2 + 3 \times 3 + 1 \times 1 + 4 \times 1 + 1 \times 0 + 2 \times 1 = 21$

Convolution 연산

Filter를 일정 간격만큼 이동하여 Input과 대응하는 위치의 원소끼리 곱한 후 그 총합을 구하는 연산

1	2	3	0	2	1
0	3	1	2	1	4
4	1	2	1	0	3
3	2	3	2	1	0
4	0	2	3	1	3
2	1	1	1	0	3

Input



1	2	0
2	3	1
1	0	1

Filter



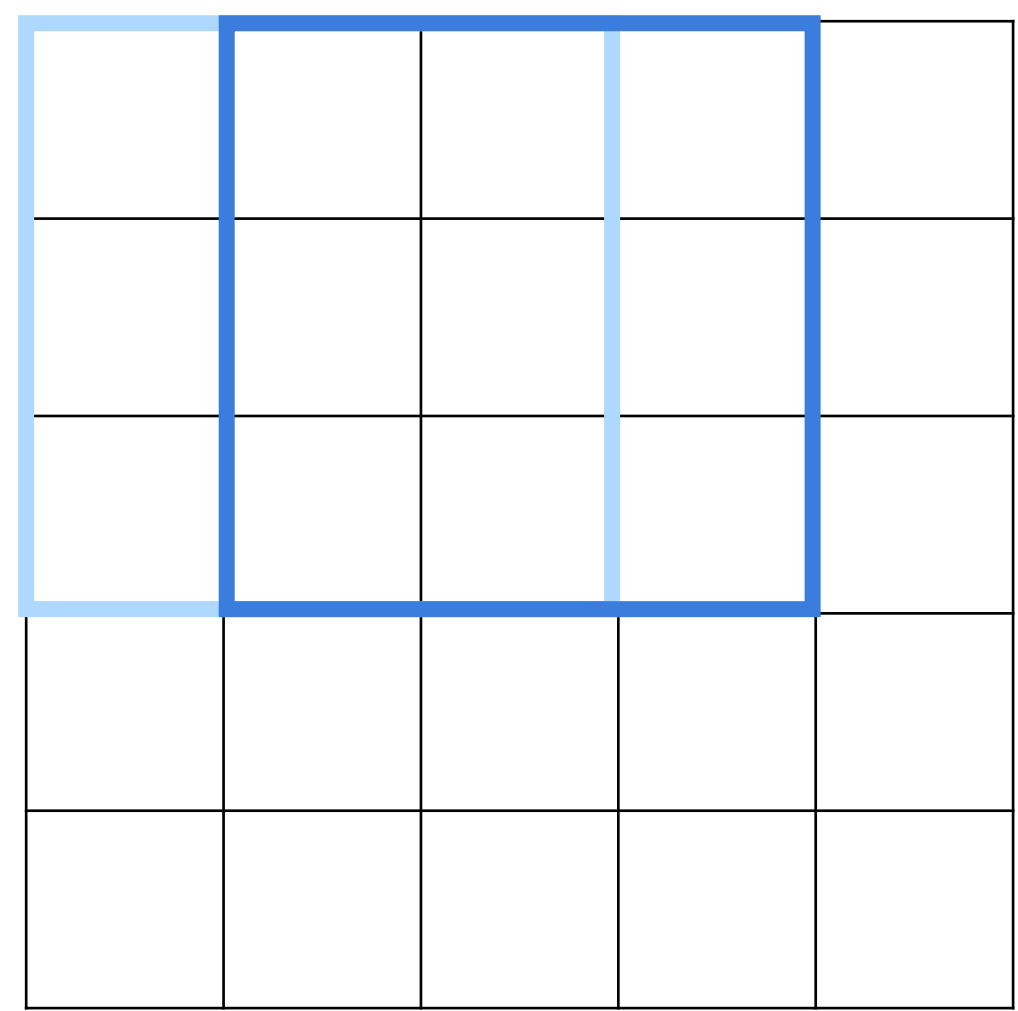
21	21	14	19
25	18	16	11
27	23	20	14
20	19	22	20

Output

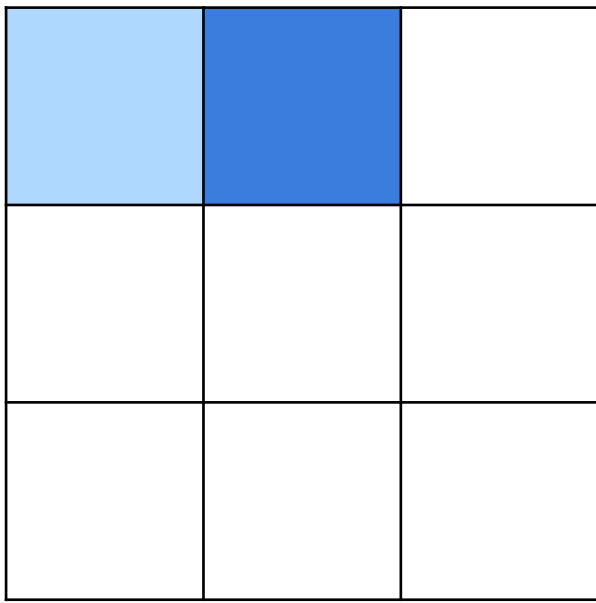
ex) $1 \times 1 + 2 \times 2 + 3 \times 0 + 0 \times 2 + 3 \times 3 + 1 \times 1 + 4 \times 1 + 1 \times 0 + 2 \times 1 = 21$

Stride

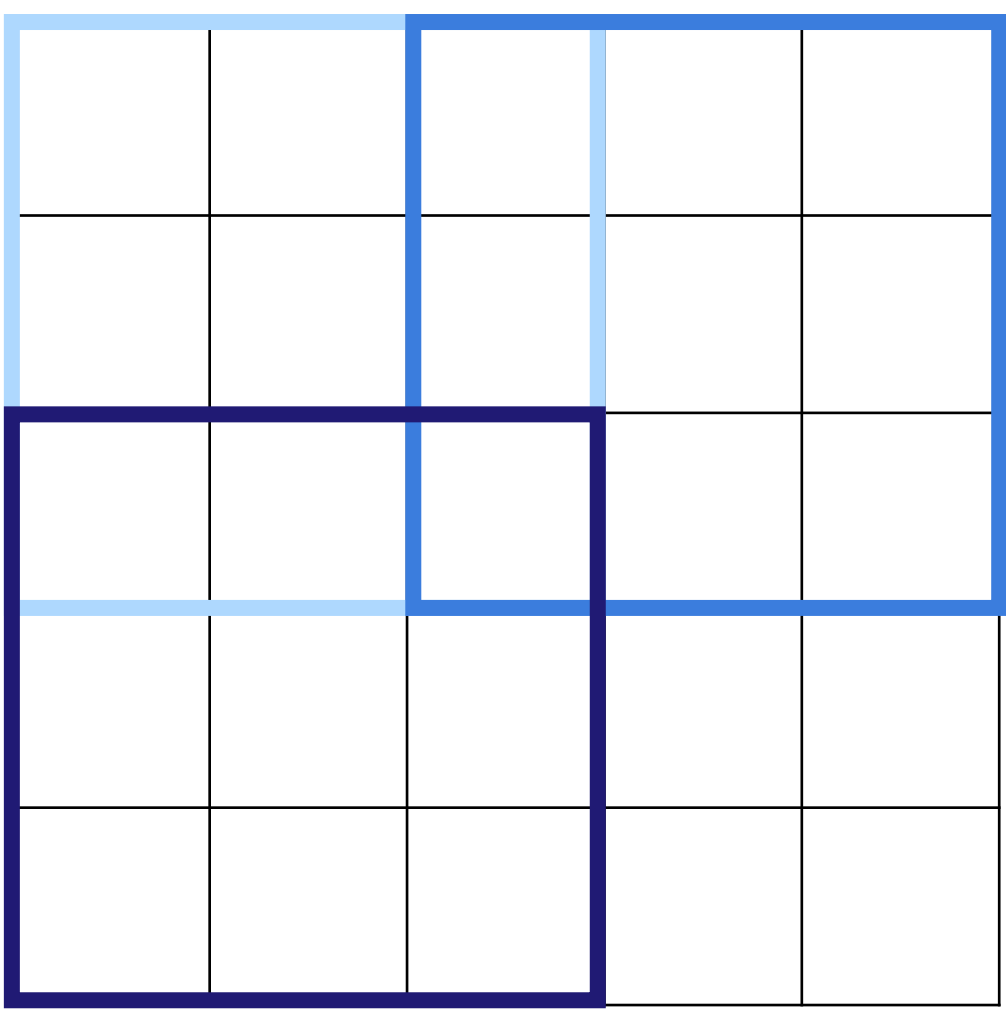
Feature를 뽑을 때 Filter가 Input 이미지를 돌면서 이동하는 간격



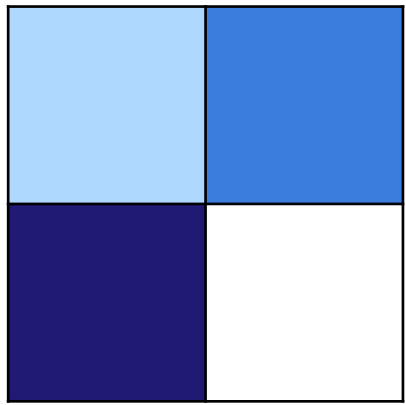
Convolution
with Stride = 1



Output



Convolution
with Stride = 2



Output

Padding

Convolution 연산을 수행하기 전에 데이터 주변을 특정 값(ex. 0)으로 채우는 것

- 주로 출력 크기를 조정하기 위해 사용
- Zero Padding, Full Padding 등

Padding

0	0	0	0	0
0				0
0				0
0				0
0				0
0	0	0	0	0

Padding Size = 1

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0				0	0
0	0				0	0
0	0				0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Padding Size = 2

정리

Input에 대해 설정한 Filter(=Kernel) size, Stride, Padding에 따라 Convolution 연산을 수행

ex) Input : 6 x 6, Filter : 4 x 4 (Stride = 1, Padding = 1)

0	0	0	0	0	0	0	0
0	1	2	3	0	2	1	0
0	0	3	1	2	1	4	0
0	4	1	2	1	0	3	0
0	3	2	3	2	1	0	0
0	4	0	2	3	1	3	0
0	2	1	1	1	0	3	0
0	0	0	0	0	0	0	0

Input



2	0	1	1
1	1	2	0
0	2	3	1
0	1	0	1

Filter



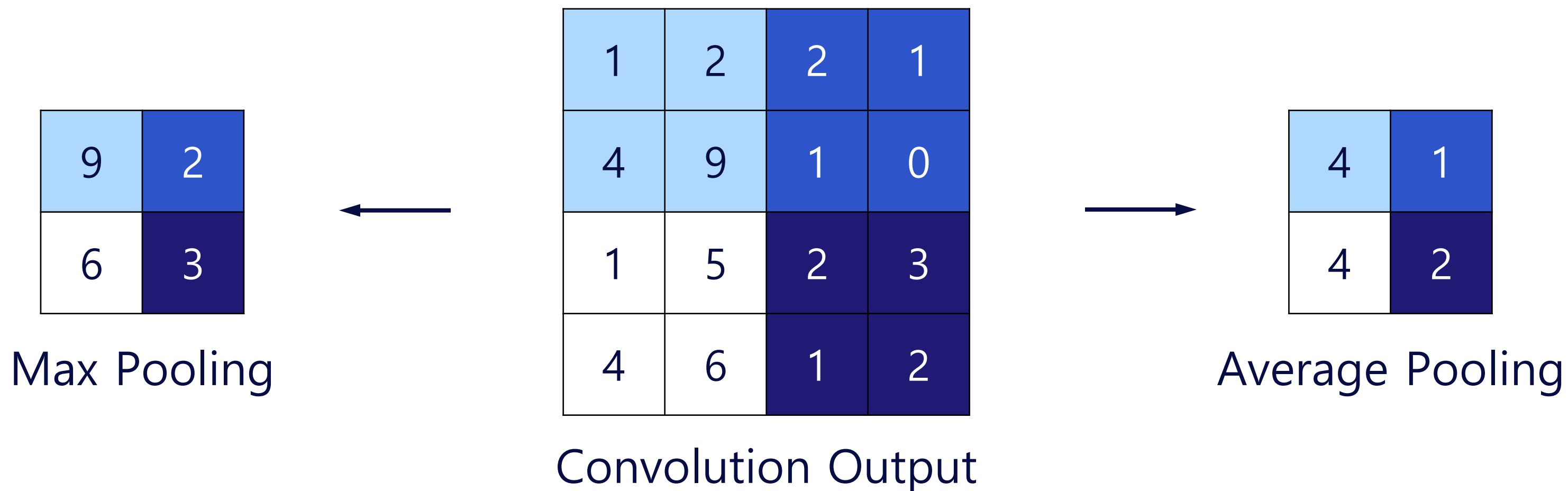
21	22	16	22	18
30	23	25	21	22
31	30	30	23	18
23	33	27	30	19
17	25	20	19	23

Output

Pooling Layer

Feature의 가로 · 세로 방향 Dimension을 줄이는 연산을 수행하는 Layer
- 일반적으로 Convolution Layer를 거친 다음에 Pooling Layer 적용

CNN의 학습 속도를 향상시키기 위한 것으로 정보 손실 존재

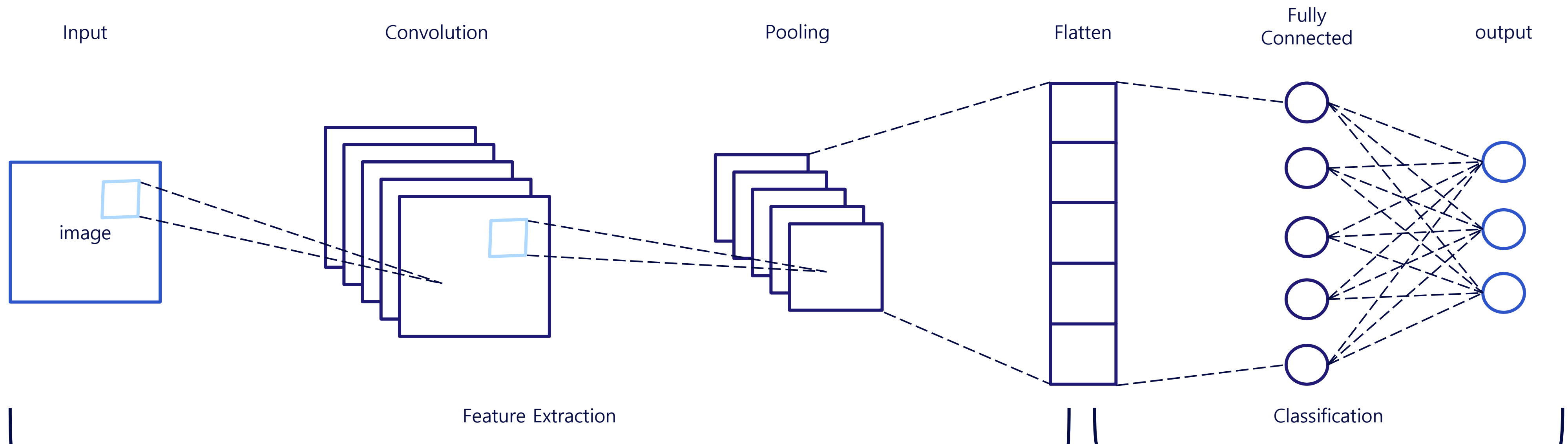


Fully Connected Layer

이미지를 정의된 라벨(클래스)로 분류하기 위한 Layer

- 일반적인 MLP의 구조와 동일

- Pooling Layer에서 나온 Feature를 Flatten 시킨 후에 MLP의 Input으로 놓고 학습 진행



ResNet

Convolution 연산

- Receptive field 를 가정
- Feature map 생성
- Feature map 을 Flatten
- Soft-max Function 으로 예측

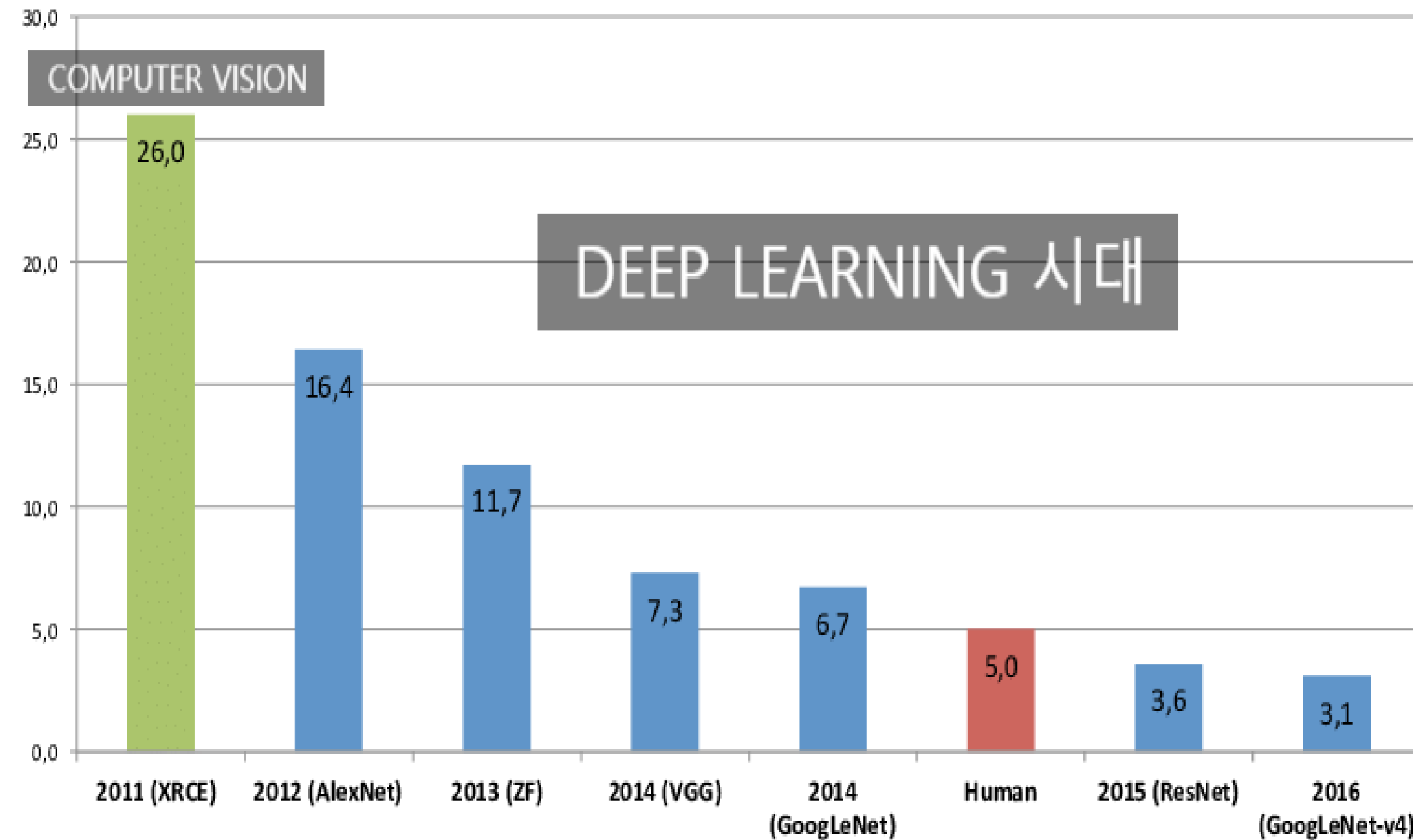
Idea :

1. 이미지는 공간에 대한 정보를 담고있다.
2. 상관관계가 높은 영역의 계산을 한번에 해보자 !

Back Propagation & Convolution

- 오차 역전파법의 경우 실제 수치적으로 계산하는 방법
- Convolution 은 공간적 정보를 담을 거라는 Idea 의 일종

ImageNet Classification Error (Top 5)



ResNet

Layer 를 계속 쌓으면 성능이 계속 올라가나?

- GoogLeNet 의 경우 특징은 inception module
- 하지만 Layer 를 깊게 쌓은 건 아님 단지 효율성
- VGG 와 비슷한 Plain network 를 계속 쌓으면?
- 오히려 성능이 더 떨어진다!

성능은 올리고 싶고,, Layer 를 쌓자니 오히려 성능이 떨어지고,,

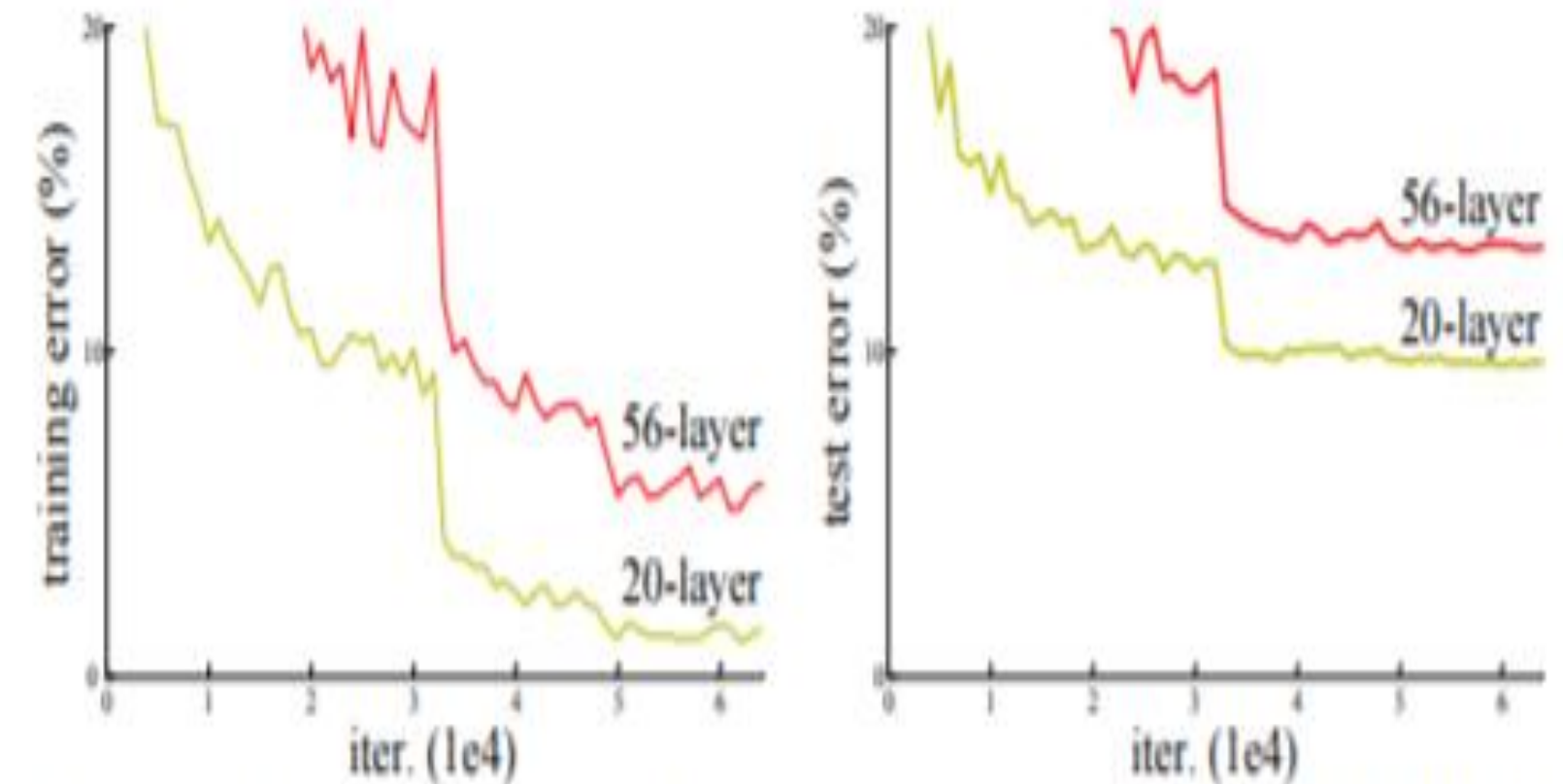
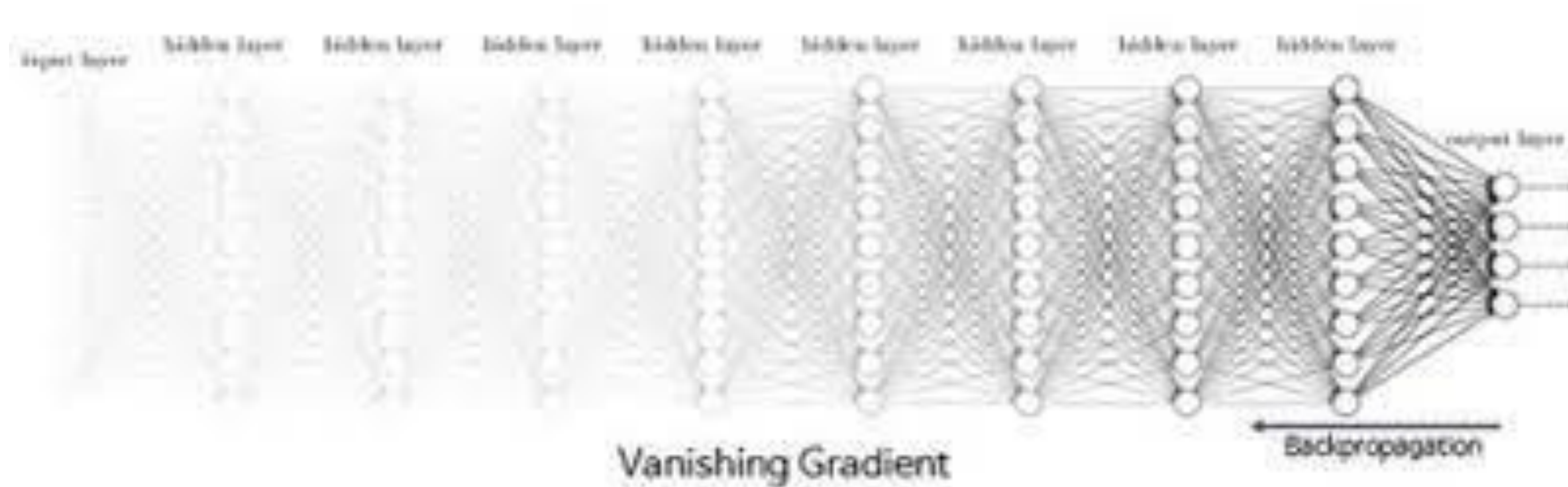


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

ResNet



- Layer 가 깊다면 기울기가 소실된다.
- Input 데이터가 연산 되는 과정에서 원래 정보를 잃는다.
- Information flow 를 이어 가기가 어렵다.
- 따라서 Layer 를 거친 결과 값에 x 를 더해줘서 기존 정보를 살리자!
- input x 를 더해주는 지름길을 만든다.(Short connection)

ResNet 의 시발점

- Layer 를 매우 깊게 쌓으면 진짜 성능이 낮아질까?
- 성능이 떨어지는 이유는 자명하다.
- 주된 이유는 Overfitting
- 근데 Training 에서는 성능이 왜 떨어질까?

Identity mapping

- Input 이 X 라면 output 도 X 인 mapping
- 이러한 매핑을 100층 1000층 쌓으면, 그 Network 는 성능이 떨어지진 않을 것
- 그렇다면 output 전체를 학습하는 게 아니라 **변화된 것**만 학습한다면?

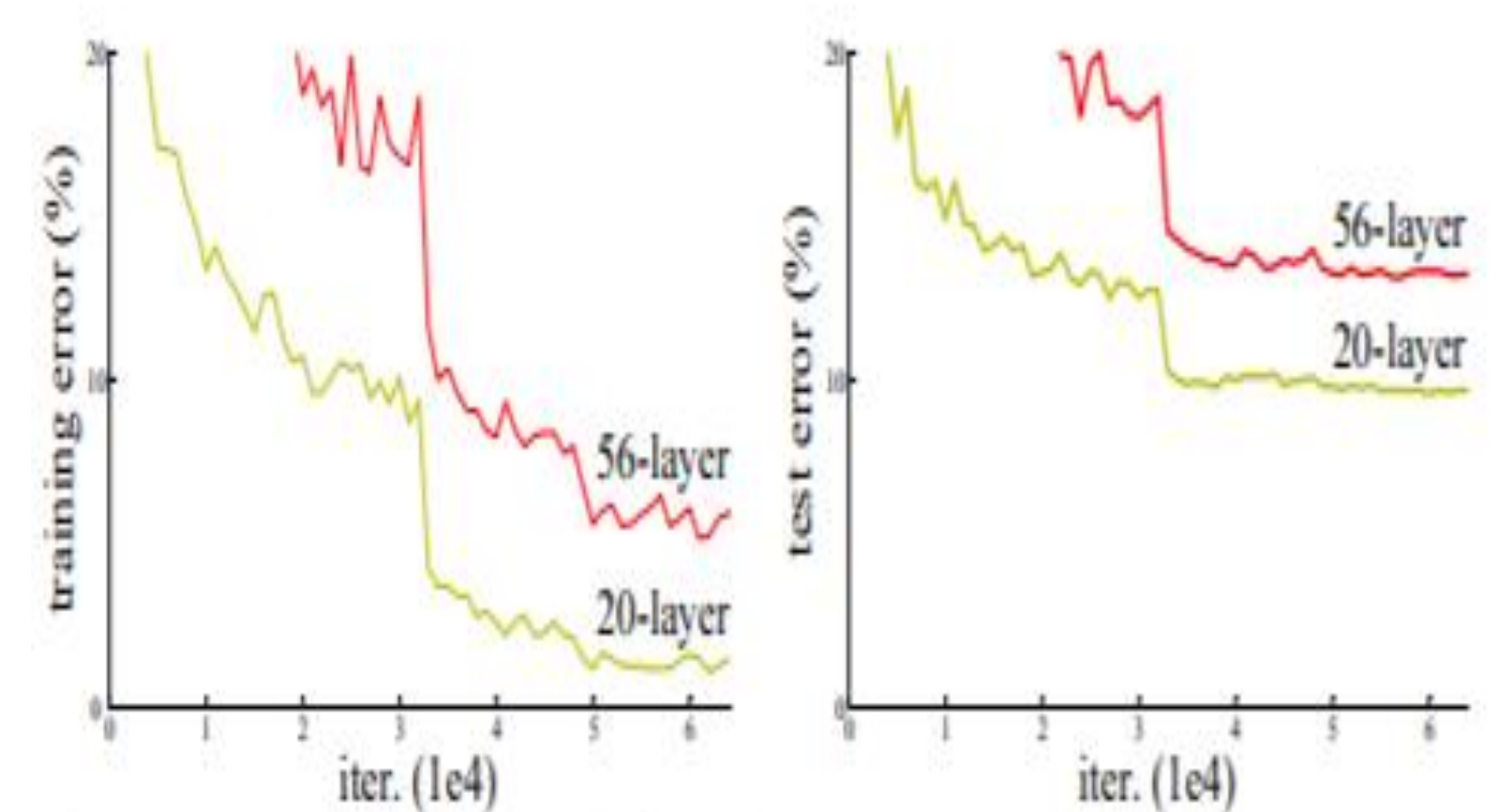


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Deep Residual Learning 의 방향성

- 기존 방식 : x 값의 예측 값인 $H(x)$ 가 실제값 y 와 가까워 지길 원함 따라서 매핑 함수 $H(x)$ 를 구하고 싶음
왜 하필 $F(x)$ 를 더하는가?

기존 학습 방향

- 기존방식 : $y = \sigma_1 W_2 (\sigma_2 (W_1 x))$ (잔차생략)
- Residua Block : $y = F(x, W_i) + W_s x$ (W_s 는 size 맞춰주는 행렬)
- $H'(x) = F'(x) + 1$ 이므로 적어도 기울기가 1 유지 -> Gradient Vanishing 문제 해소
- 극단적으로 최적 값이 Identity mapping 이라고 할 때, $H(x)$ 를 맞추는 것 보다 $F(x)$ 가 0 임을 맞추는 것이 더 쉽다(Idea)
- 기능을 수행하는 $H(x)$ 를 맞추는 것이 숫자인 $F(x)$ 를 맞추는 것 보다 더 어렵다 (Idea)

잔차 학습 (Short cut Connection)

- 각 레이어에서 벌어지는 일
- 기존 Net 에서처럼 $H(x)$ 를 구하는 것이 아닌, $F(x)$ 를 구한다.
- 수식적으로 $H(x) = x + F(x)$ 이기 때문에 관점이 다르지, 계산 맥락은 비슷함
- Residua Block : $y = F(x, W_i) + W_s x$ (W_s 는 size 맞춰주는 행렬)
- Size 가 달라지는 경우는 Pooling 을 거칠 때 달라지기 때문에 학습 가능한 W 를 x 에 곱해줌으로써 덧셈 연산이 가능해지게 함

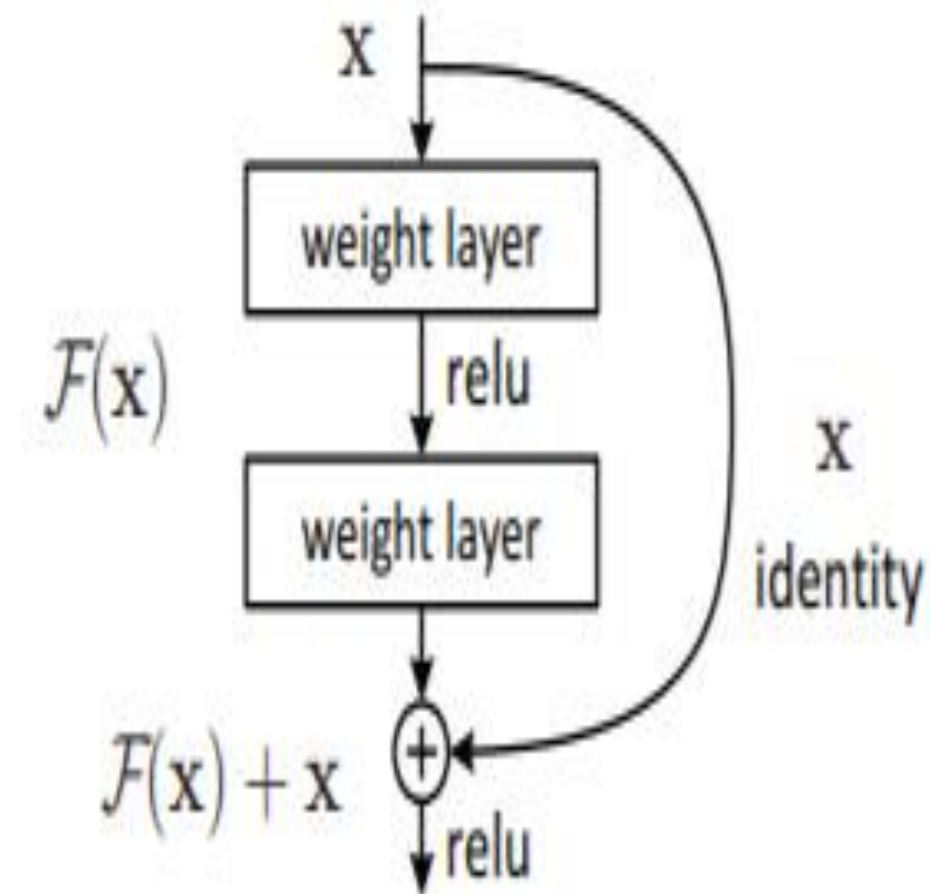
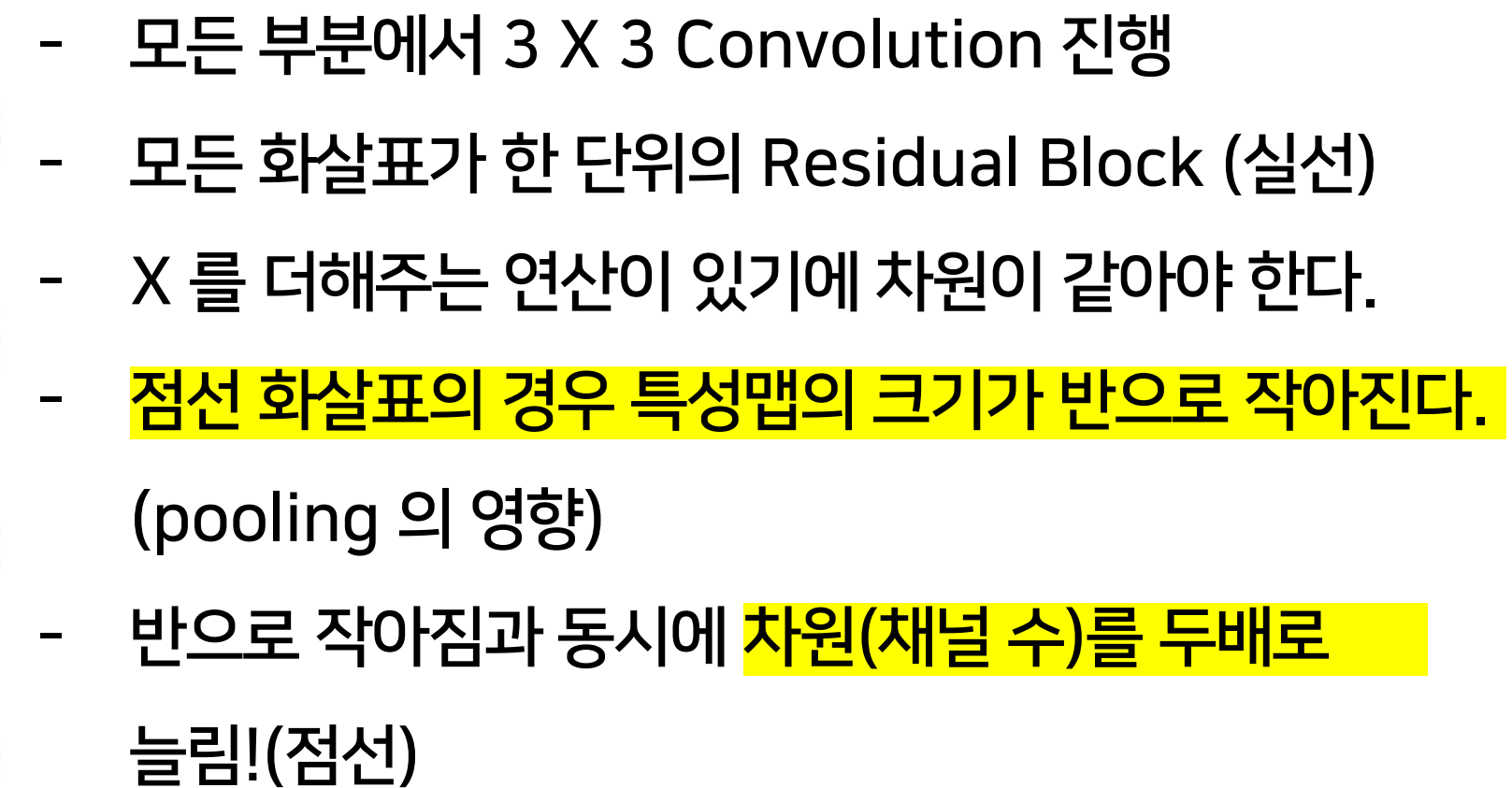
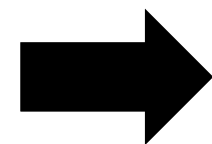


Figure 2. Residual learning: a building block.



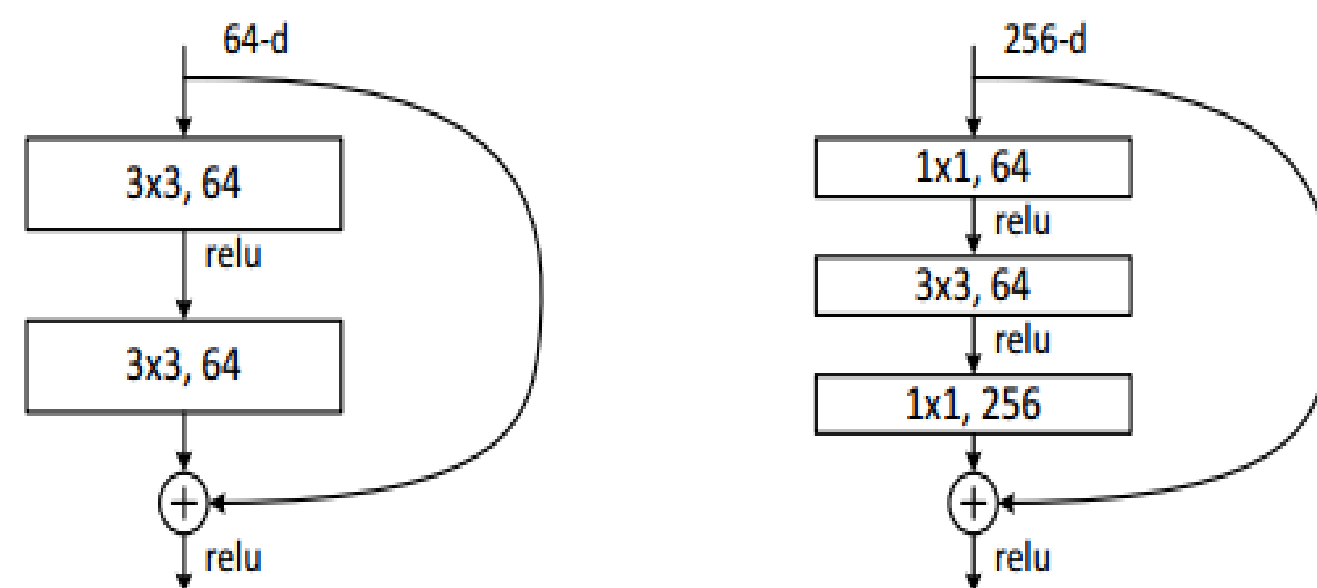


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	$7 \times 7, 64, \text{stride } 2$				
conv2_x	56×56	$3 \times 3 \text{ max pool, stride } 2$				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

- Layer 가 50 개 이상일 때 1 X 1 Conv 로 연산량을 줄여준다.
- 병의 주둥이와 같이 좁은 곳을 통과한다는 의미에서 **Bottle Neck 구조!**

- 최종적으로 152 레이어까지 늘린다.
- VGG- 16 의 연산량 > Resnet-152 연산량
- 이유는? 그냥 x 를 더해주는 것이기 때문에 추가적인 파라미터가 없다!

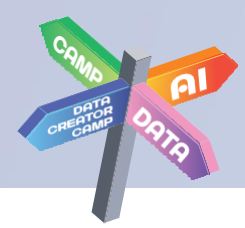
MLP vs CNN 비교 정리

항목	MLP	CNN
입력 형태	1D 벡터 (flatten 필요)	2D/3D 이미지 그대로
연결 방식	모든 뉴런 전결합	지역 연결 (Local)
가중치	위치마다 다른 가중치	필터 공유 (Weight Sharing)
공간 정보	소실됨	보존됨
파라미터 수	매우 많음	상대적으로 적음
이미지 위치 변화	취약함	강건함 (불변성)

Mini Quiz

OX Quiz

1. 입력 데이터는 채널 수와 상관없이 필터 별로 1개의 피쳐 맵이 만들어진다.
2. Pooling 레이어는 학습대상 파라미터가 없다
3. Pooling 레이어를 통과하면 행렬의 크기 감소한다.
4. Pooling 레이어를 통과하면 채널 수가 변경되지 않는다.



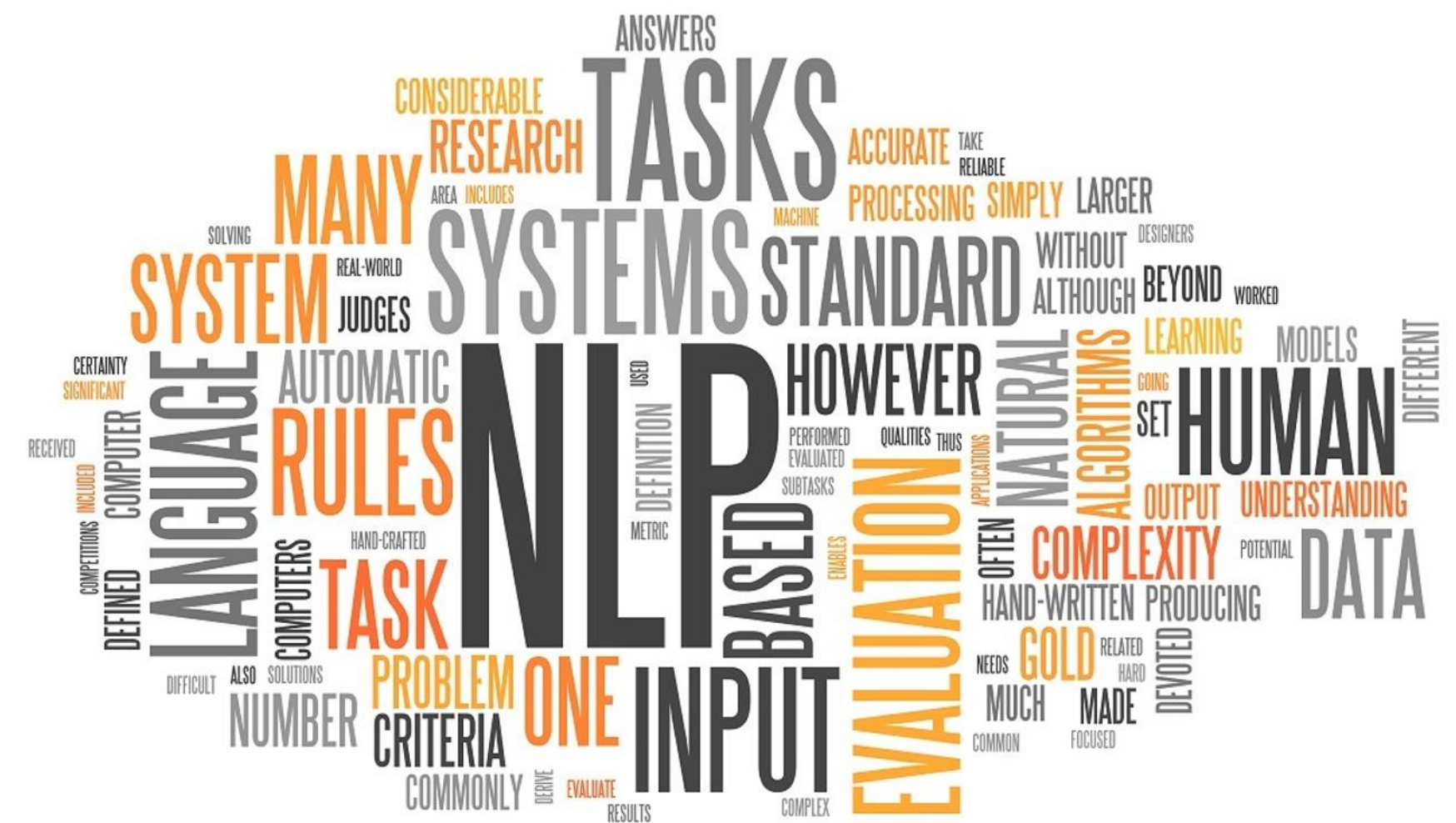
6

텍스트 데이터와 NLP 기초

텍스트 데이터를 숫자 표현으로 변환하고 분류 모델을 활용하는 방법 이해

학습목표

1. 문장, 토큰, Vocabulary의 관계를 설명한다.
2. TF-IDF와 임베딩의 차이를 설명한다.
3. RNN, LSTM, Transformer, BERT가 텍스트를 처리하는 차이를 개념적으로 구분한다.
4. 텍스트를 수치 벡터로 변환하여 이진 분류 모델을 학습한다.



텍스트 데이터란?

예시 : 문장만보고 실제 재난인지 판단하기

“The forest is on **fire**.”

=> 재난 □ (1)

“This new song is **fire**.”

=> 재난 X (0)

“**불**이 숲에 번지고 있다.”

=> 재난 □ (1)

“이 노래 완전 **불**타오른다.”

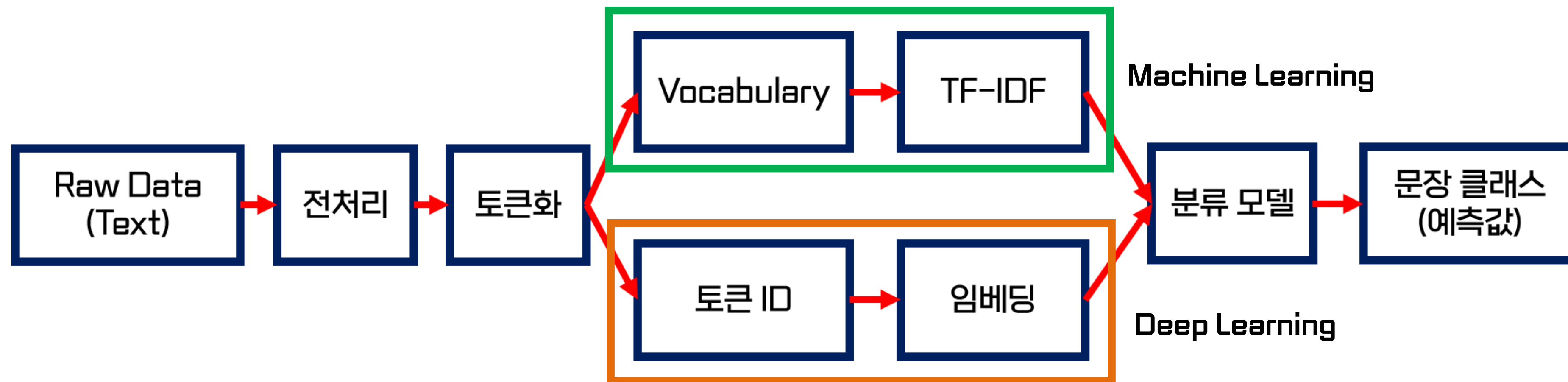
=> 재난 X (0)

“두 문장에 모두 ‘**fire**’ 또는 ‘**불**’이라는 단어가 있지만, 왜 정답은 다른가?”

“문장은 단어 뿐만 아니라 **문맥**까지 고려해야 한다.”

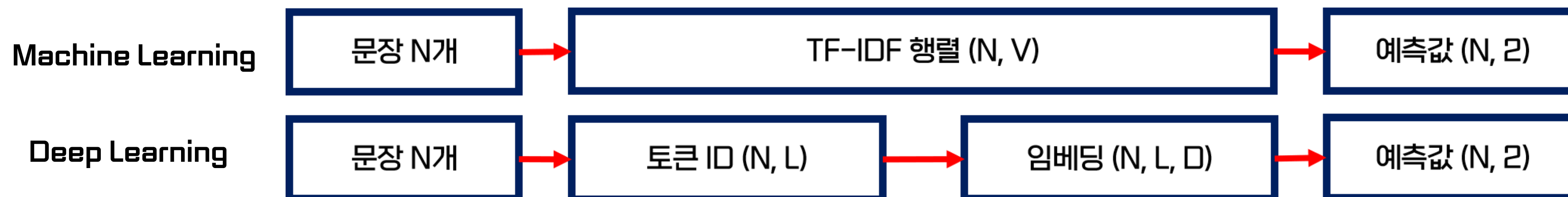
텍스트 분류 전체 파이프라인

“AI는 문장과 단어를 읽지 못한다. 문장과 단어를 **숫자의 배열**로 바꾸어야 한다.”



大

V : vocabulary 크기, L : 문장 길이, D : 임베딩 차원



텍스트 데이터가 어려운 이유

1. 문장마다 길이가 다름

“Fire!” “The wildfire is spreading near the city.”

2. 단어의 종류가 매우 많음

“fire, fires, wildfire, wildfires, burning, burned ...”

3. 같은 단어가 다른 의미로 사용됨

“bank : 은행 / 감독, fire : 화재 / 해고하다”

4. 단어의 순서가 중요함

“dog bites man” “man bites dog”

5. 실제 텍스트에는 잡음이 많음

URL, 해시태그, 이모지, 오타자, 반복 문자, 약어



이미지는 보통 높이와 너비를 조정하면 동일한 크기의 텐서로 만들 수 있습니다.

반면 문장은 길이가 모두 다르고, 사용할 수 있는 단어의 수도 매우 많습니다.

또한 단어의 순서와 문맥에 따라 의미가 바뀝니다.

텍스트 전처리

“다양한 전처리 방법이 있지만 수행하는 목적에 따라 잘 선택해야한다.”

전처리를 많이한다고 무조건 좋다 X

1. 소문자 변환 (Lowercasing) or 대문자 변환 (Uppercasing)

- 텍스트 데이터를 모두 소문자로 변환

```
text = "Hello World"
processed_text = text.lower()

>>> "hello world"
```

2. 불용어 처리 (Stopword Removal)

- 불용어 : 일반적으로 자주 사용되는 단어들
- 텍스트 데이터의 크기를 줄이고, 처리 성능 향상

```
from nltk.corpus import stopwords

stopwords = set(stopwords.words("english"))
tokens = ["I", "love", "to", "go", "the", "park"]
processed_tokens = [token for token in tokens if token.lower() not in stopwords]

>>> ['love', 'go', 'park']
```


텍스트 전처리

3. 어간 추출 (Stemming)

- 단어의 어미를 제거함, 단어의 형태 변화를 고려 X, 단순히 어미를 자름 (단순히 어미를 잘라서 문제 발생 가능 □)

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
word = "running"
```

```
stemmed_word = stemmer.stem(word)
```

```
>>> "run"
```

```
stemmer = PorterStemmer()
```

```
words = ["dogs", "cats", "horses"]
```

```
stemmed_words = [stemmer.stem(word) for word in words]
```

```
>>> ['dog', 'cat', 'hors']
```

4. 표제어 추출 (Lemmatization)

- 단어의 원형인 표제어를 찾는 과정, 단어의 형태 변화와 문법적인 특성을 고려해 단어를 통합하는 방식

```
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

```
word = "running"
```

```
lemmatized_word = lemmatizer.lemmatize(word)
```

```
>>> "running"
```

```
lemmatizer = WordNetLemmatizer()
```

```
words = ["dogs", "cats", "horses"]
```

```
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
```

```
>>> ['dog', 'cat', 'horse']
```

텍스트 전처리

5. 텍스트 정규화(Text Normalization)

1) 동의어 처리 (Synonym Replacement)

- 동의어 처리는 같은 의미의 단어가 다르게 표현된 경우 이를 통합하여 일관성을 유지하는 작업입니다. 예를 들어, 영어에서 "don't"와 "do not"는 동일한 의미를 가지므로 텍스트 정규화를 통해 "don't"을 "do not"으로 바꾸어 일관된 형태로 표현할 수 있습니다. 비슷한 예시로 “챗지피티”, “챗GPT”, “ChatGPT” 등도 목적에 따라 동일하게 처리할 수 있습니다.

2) 오타자 처리 (Typo Correction)

- 오타자 처리는 잘못 입력된 단어를 올바른 형태로 수정하는 작업입니다. 예를 들어, "teh"라는 오타자를 "the"로 수정하여 올바른 표현으로 정규화할 수 있습니다.

3) 줄임말 확장 (Abbreviation Expansion)

- 줄임말 확장은 줄임말이나 약어를 원래의 형태로 전개하는 작업입니다. 예를 들어, "ASAP"라는 줄임말은 "As Soon As Possible"로 전개하여 일반적인 형태로 표현할 수 있습니다.

4) 숫자 처리 (Number Normalization)

- 숫자 처리는 숫자를 통일된 형태로 변환하는 작업입니다. 예를 들어, "100", "100.00", "1,00.0"과 같은 다양한 형태의 숫자를 "100"으로 통일하여 일관된 표현을 유지할 수 있습니다.

토큰화 (Tokenization)

: 문장을 모델이 처리할 단위로 나누기

문장: “machine learning is useful”

토큰화 결과: ["machine", "learning", "is", "useful"]

방식	예시	특징
단어 단위	["machine", "learning"]	이해하기 쉬움
문자 단위	["m", "a", "c", "..."]	Vocabulary는 작지만 sequence가 길어짐
Subword 단위	["learn", "##ing"]	희귀 단어와 신조어에 대응 가능

문장 :“산불이 확산되었다.”

공백 기준: ["산불이", "확산되었다"]

형태소 또는 Subword 기준: ["산불", "이", "확산", "되", "었다"]

Vocabulary & Token ID

Vocabulary : Token을 숫자로 바꾸는 사전
예시)

Token	ID
<PAD>	0
<UNK>	1
fire	2
near	3
city	4
music	5

<PAD> : 문장 길이를 맞추기 위해 추가하는 토큰
<UNK> : Vocabulary에 없는 Token

“fire near city”
↓
[2, 3, 4]

“fire near village”
↓
[2, 3, 1]

ID의 번호가 100과 10이 있을때
100이 10보다 10배 중요하다는 의미 X
그저 labeling 한다고 생각

Padding, Attention Mask

“길이가 다른 문장을 하나의 Batch로 만들기”

문장 A: [2, 3, 4]
문장 B: [5, 8]
문장 C: [2, 7, 9, 4, 6]

AI는 여러 문장을 하나의 Batch로 묶어서 계산, 그러나 문장 길이가 다르면 하나의 직사각형 텐서 X, 짧은 문장에 Padding을 추가, 긴 문장은 일정 길이로 자름. (왼쪽 그림)

A: [2, 3, 4, 0, 0]
B: [5, 8, 0, 0, 0]
C: [2, 7, 9, 4, 6]

동일한 사이즈의 텐서를 만들기 위해 '0'이라는 Padding을 추가함 (왼쪽 그림)

그러나 모델은 2,3,4,5,8,0 등 숫자의 의미를 모르기에 의미있는 부분에 대해서 Check 해줘야함

A Mask: [1, 1, 1, 0, 0]
B Mask: [1, 1, 0, 0, 0]
C Mask: [1, 1, 1, 1, 1]

Attention mask

1: 실제 값이 있는 Token

0: 의미없는 Padding Token

A,B,C 각각의 위치의 값 각각이 무엇을 의미하는지 반드시 명시해줘야함

TF-IDF

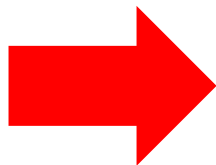
: 특정 단어가 특정 문서 내에서 얼마나 중요한지를 나타내는 통계적 수치 (이러한 정보는 Train에서만 활용)
(그 외에는 정답지를 보고 문제를푸는 것과 동일)

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$
$$IDF(t) = \log \left(\frac{N}{DF(t)} \right)$$

$TF(t, d)$: 문서 d 에서 단어 t 가 등장한 정도
 $DF(t)$: 단어 t 가 등장한 문서 수
 N : 전체 문서 수

예시) D는 문서를 의미

D1: fire near city
D2: fire on stage
D3: earthquake near city



단어	등장 문서 수	IDF의 상대적 크기
fire	2	비교적 작음
near	2	비교적 작음
city	2	비교적 작음
earthquake	1	큼

TF : 한 문서 안에서 해당 단어가 얼마나 많이 등장했는지 (위 예시에서는 모두 1)
IDF : 그 단어가 전체 문서에서 얼마나 희귀한지

$$IDF(fire) = \log \left(\frac{3}{2} \right)$$
$$IDF(earthquake) = \log \left(\frac{3}{1} \right)$$

임베딩 (Embedding)

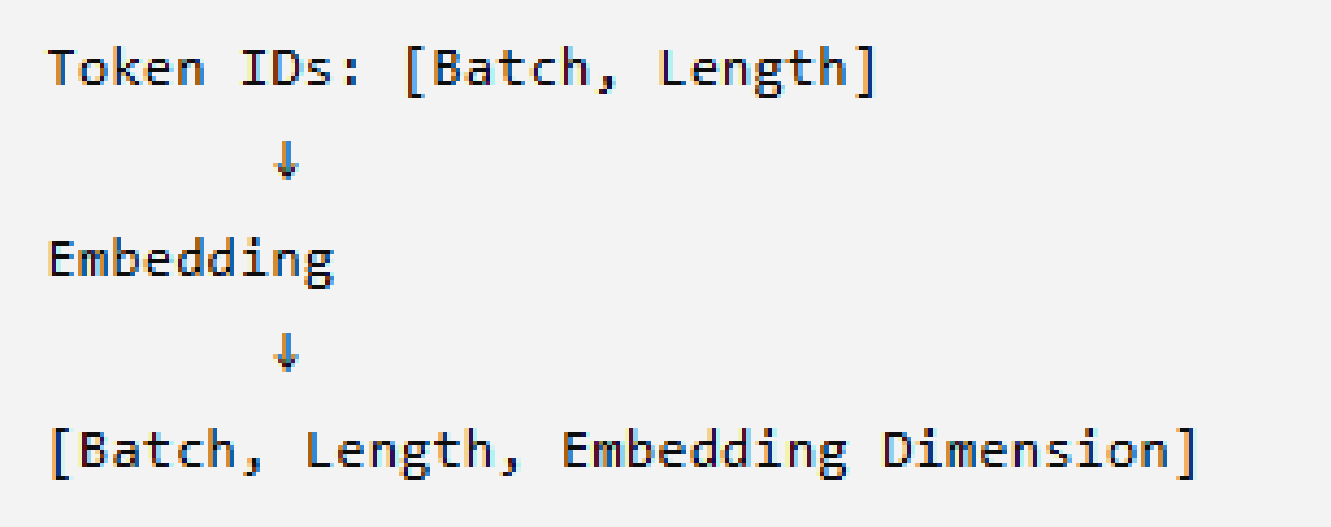
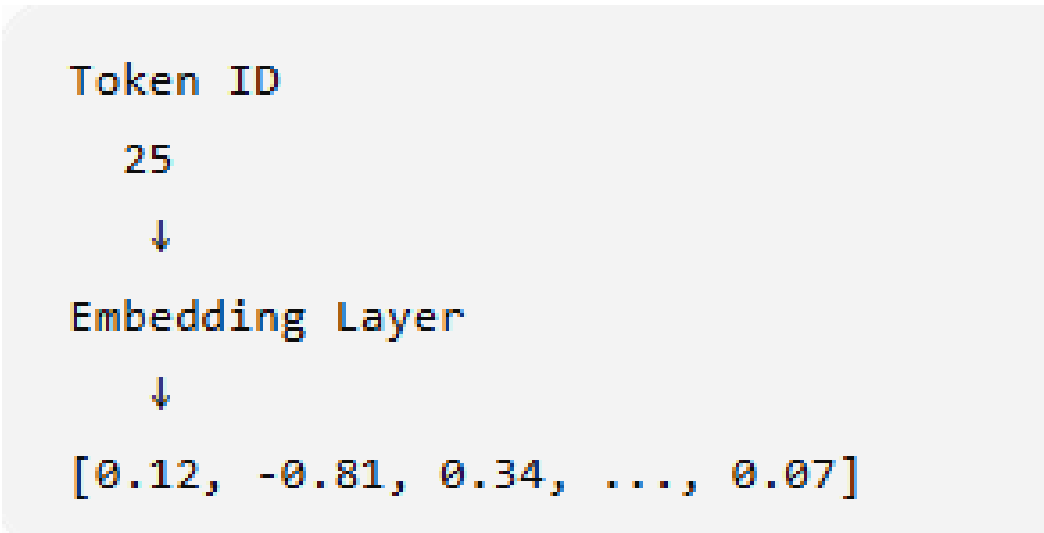
: 단어를 의미 공간의 좌표에 표현하는 방법

구분	TF-IDF	Embedding
표현 형태	희소 벡터	밀집 벡터
차원	Vocabulary 크기	지정한 Embedding 차원
주요 정보	단어의 빈도와 희귀성	학습된 의미적 특징
단어 유사성	직접 표현하기 어려움	벡터 거리로 표현 가능
단어 순서	기본적으로 반영하지 않음	다음 Sequence model에서 활용

TF-IDF는 단어를 분류하고 중요성을 판단할뿐,
비슷한 단어끼리를 연결해주는 것에 한계가 있음

(학습이 잘 이루어지면 fire, wildfire, smoke와 같이
관련 있는 단어가 벡터 공간에서 비슷한 위치에 나타날
수 있습니다. 반면 music, restaurant와 같이 다른
의미의 단어는 상대적으로 멀리 위치할 수 있습니다.)

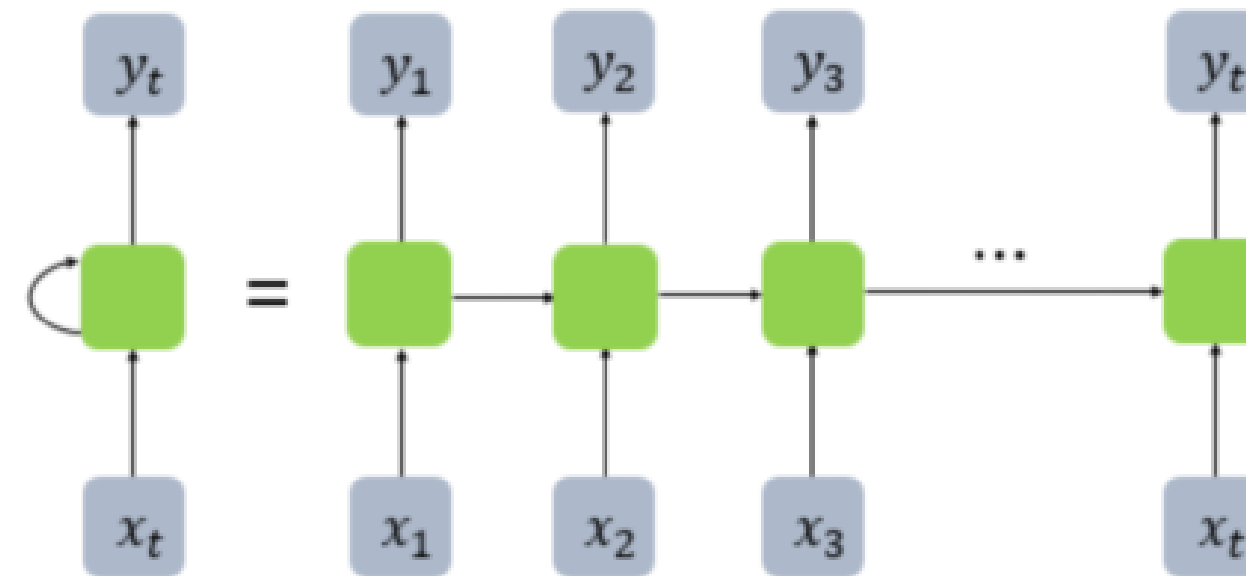
예시)



RNN과 LSTM

1) RNN (Recurrent Neural Network)

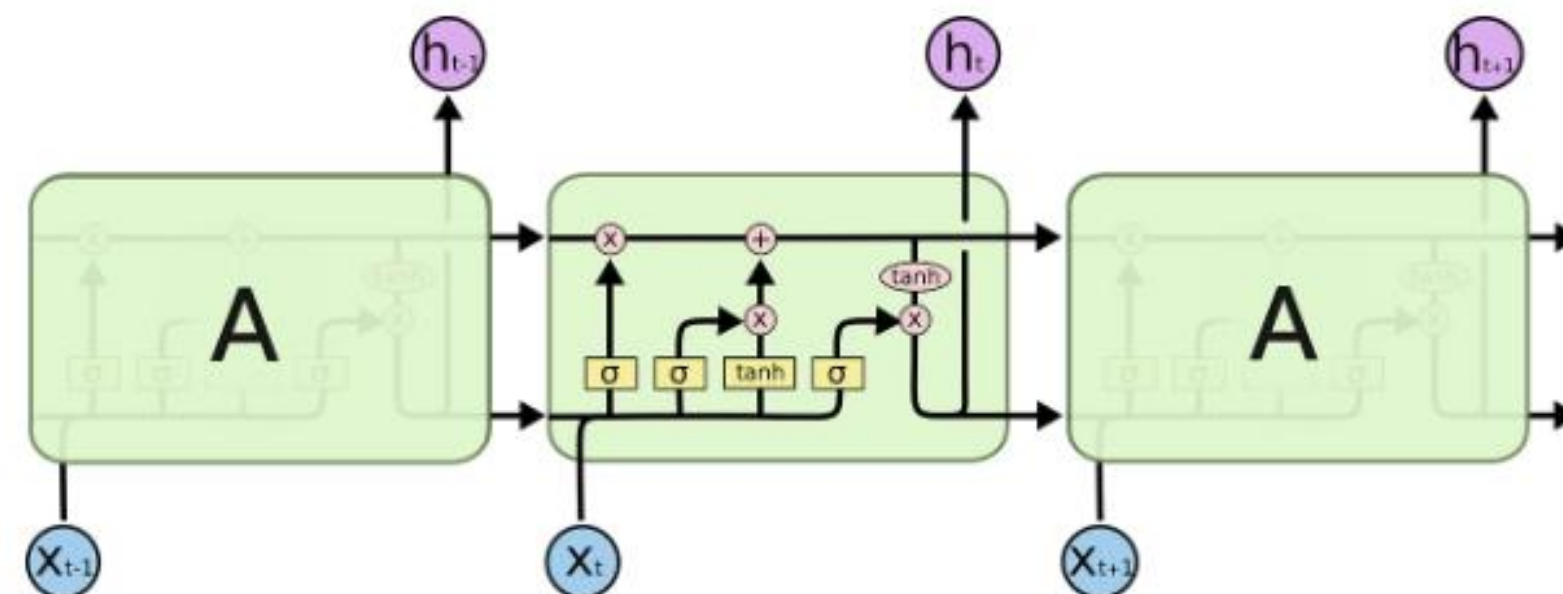
- 특징 : 앞에서 읽은 정보를 Hidden State에 모두 저장



첫 번째 단어부터 마지막 단어까지 모두 순서대로 읽고 저장하여 '기울기 손실' 문제가 발생하여 앞 부분을 잘 기억하지 못하는 한계 발생 가능

2) LSTM (Long short-Term Memory)

- 특징 : Gate를 이용해 필요한 정보를 기억하거나 제거를 조절 (Forget, Input, Output Gate)



Transformer와 BERT

1) Transformer

- 특징 : 문장 전체의 관계를 동시에 봄

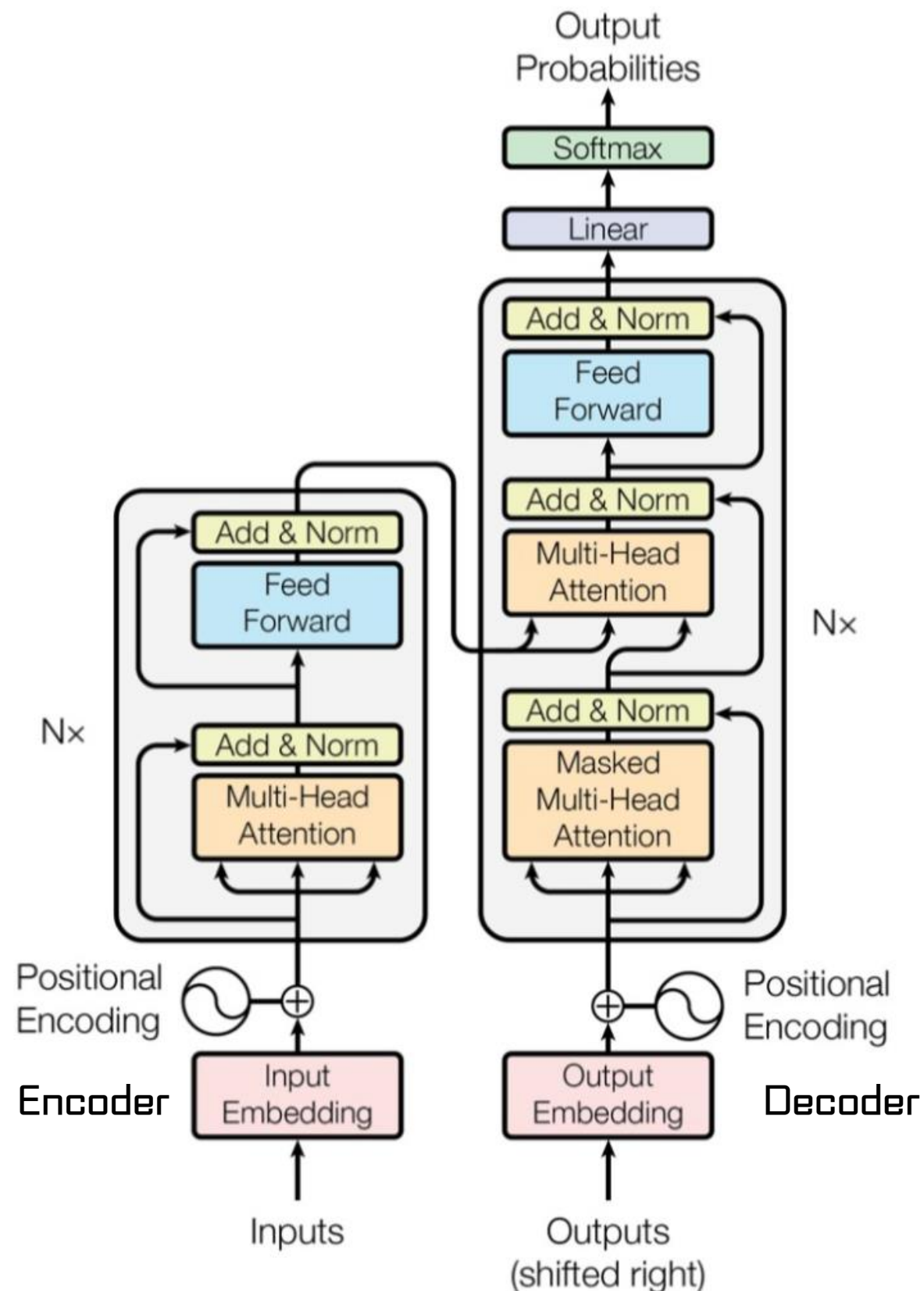
예)

"The fire near the forest is spreading."

```

fire ----- forest
 |               |
 |               |
 |----- spreading

```



Encoder

- 인코더는 input으로 들어온 자연어 sequence를 숫자형 표현으로 변경

Decoder

- 디코더는 인코더로 받은 숫자형 표현을 인간이 이해할 수 있는 자연어로 변경

Attention

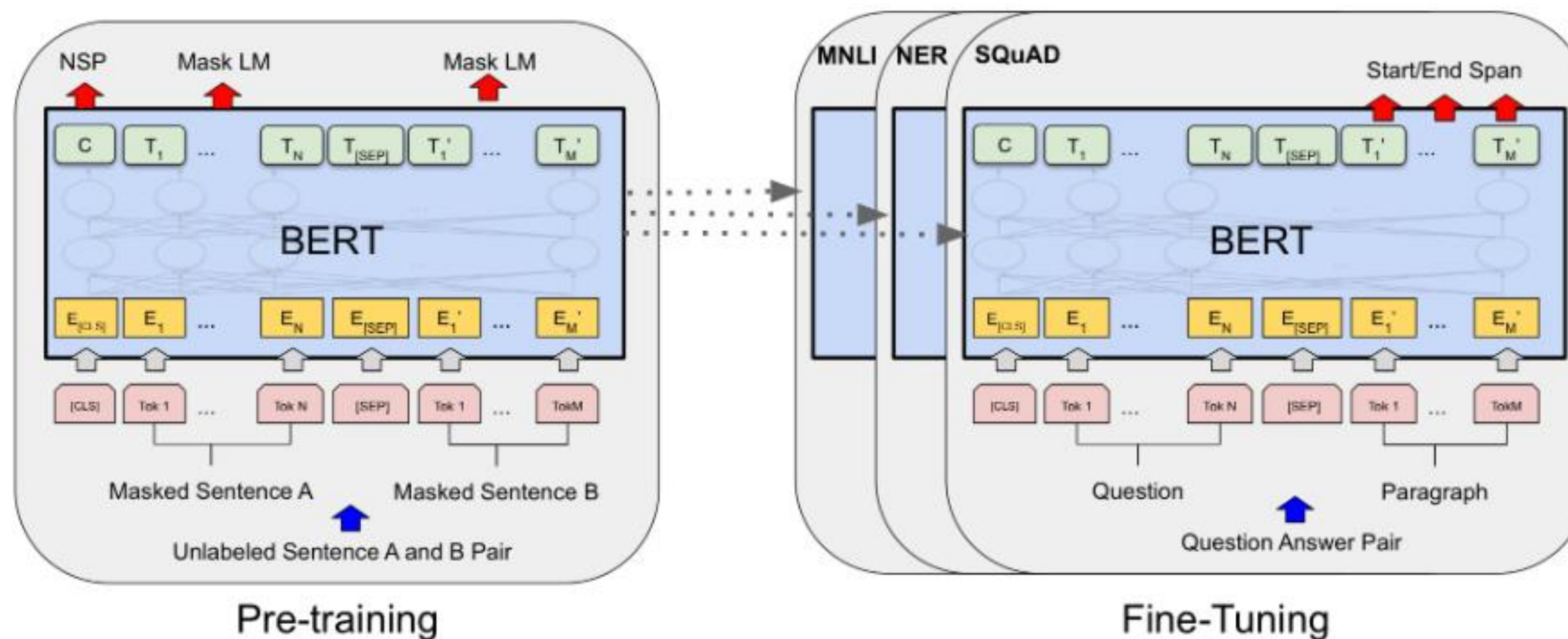
- query와 key-value 쌍을 output으로 나타내는 것
- query와 key를 이용해 value의 가중합을 구해 어떤 정보가 중요한지 나타냄

Transformer와 BERT

2) BERT

- unlabeled text에서 컨텍스트를 양방향으로 고려한 representation을 사전학습 (Pre-training)
- 전학습한 모델을 fine-tune하여 다양한 자연어 태스크를 수행할 수 있도록 만든 모델
 - 사전학습을 통해 언어의 representation, 즉 언어의 정보를 풍성하게 이해하고 있는 거대한 모델은 약간의 학습과 일부 데이터만 있다면 다양한 downstream task를 잘 수행

예) 사람으로 치자면 어렸을 적부터 책을 많이 읽은 아이가 조금만 공부하고 노력하면 국어 시험도 잘 보고, 논술도 잘 하고, 토론도 잘하고, 면접도 잘 보는 등 언어에 관련한 다양한 태스크들을 다 잘 수행하는 것과 유사



 과학기술정보통신부

 NIA

지능정보원

한국지능정보사회진흥원

모델 비교

방법	장점	한계	적합한 상황
TF-IDF + 선형 모델	빠름, 구현 쉬움, 해석 가능	문맥과 긴 순서를 잘 반영하지 못함	기본 성능 확인, 작은 데이터
Embedding + LSTM	단어 순서 반영	순차 계산, 긴 문장 학습의 어려움	순서 정보가 중요한 문제
BERT Fine-tuning	문맥 이해, 사전학습 활용	계산량과 메모리 증가	높은 성능이 필요한 분류 문제

- TF-IDF는 단순하지만 매우 빠르고, 어떤 단어가 예측에 영향을 주었는지 확인하기 쉽습니다. 따라서 새로운 텍스트 분류 문제를 시작할 때 기본 성능을 확인하는 **Baseline**으로 매우 유용합니다.
- LSTM과 BERT는 단어 순서와 문맥을 더 잘 처리할 수 있지만, 학습 시간이 길고 구현과 해석이 더 복잡합니다. 데이터의 크기, 필요한 성능, 계산 자원, 결과 해석의 필요성을 고려하여 모델을 선택해야 합니다.

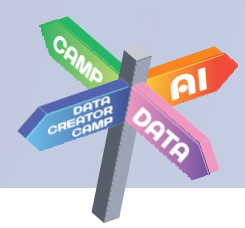
6주차 핵심 내용

1. 모델은 원문을 직접 읽지 않고 **숫자 표현**을 입력받는다.
2. TF-IDF의 Vocabulary와 IDF는 Train 데이터에서만 학습한다.
3. 복잡한 모델을 사용하기 전에 **간단한 Baseline**을 먼저 구축한다.

Mini Quiz

Q. 다음 중 TF-IDF를 이용한 텍스트 분류 모델의 올바른 학습 절차로 가장 적절한 것은?

1. 전체 텍스트 데이터로 TF-IDF Vectorizer를 먼저 학습한 후, 데이터를 Train과 Validation으로 나눈다.
2. 데이터를 Train과 Validation으로 먼저 나눈 후, Train 데이터로 TF-IDF Vectorizer를 학습하고 Validation 데이터에는 Transform만 적용한다.
3. Token ID가 클수록 해당 단어가 문장 분류에 더 중요한 단어이므로, Token ID 값을 모델의 입력 점수로 사용한다.
4. Bigram을 사용하면 문장의 전체 문맥을 이해할 수 있으므로 별도의 Validation 데이터 없이 모델 성능을 평가할 수 있다.



7

오디오 데이터 이해와 특징 추출

소리 데이터를 모델 입력으로 변환하는 기본 원리 이해

오디오 데이터?

“모델은 소리를 듣지 못한다. 숫자로 바꿔서 인식한다.”

사람의 음성, 자동차 경적, 새소리 등을 모델은 어떻게 인식할까? → Waveform 형태의 숫자로 인식한다.

실제 소리



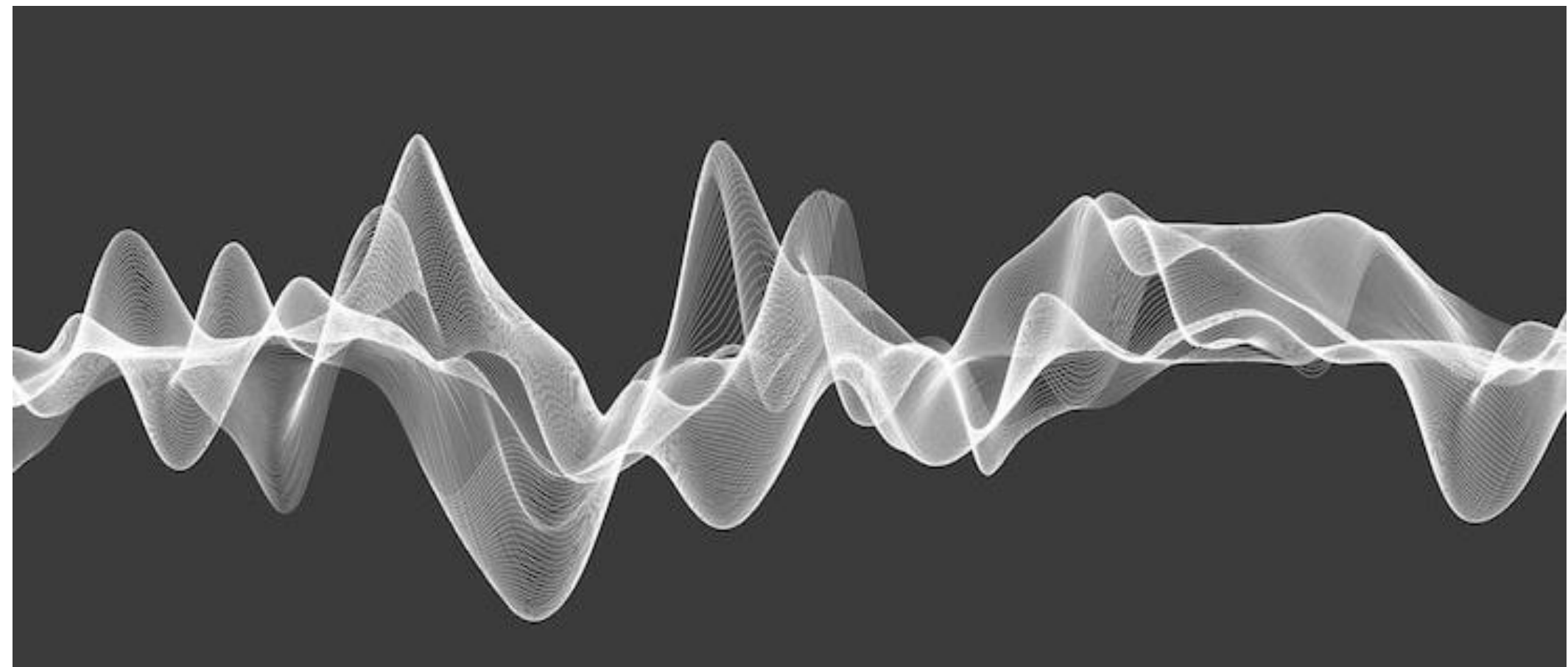
Waveform



Mel-spectrogram 또는 MFCC

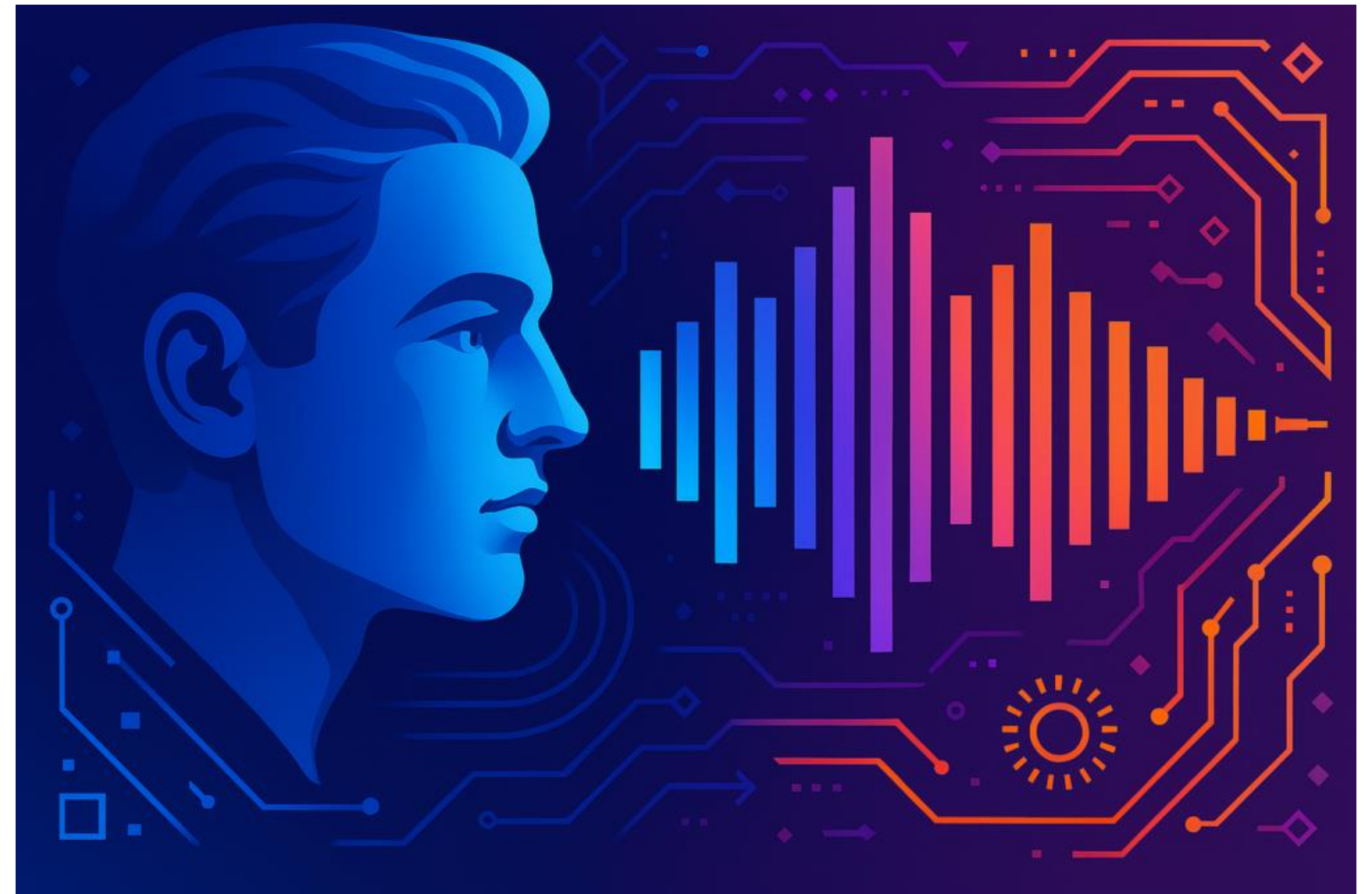


딥러닝 모델 입력



학습목표

1. 소리가 Waveform으로 표현되는 원리를 이해한다.
2. Sampling rate, Duration, Channel의 의미를 이해한다.
3. 서로 다른 길이의 오디오를 일정한 크기로 변환하는 방법을 이해한다.
4. Waveform을 Spectrogram, Mel-spectrogram, MFCC로 변환하는 원리를 이해한다.



오디오 처리 전체 파이프라인

“오디오 데이터는 어떤 과정을 거쳐 모델 입력이 되는가?”

실제 소리
↓
마이크와 디지털 변환
↓
Waveform
[Channel, Sample]
↓
Sampling rate 통일
↓
Mono 또는 다중 Channel 처리
↓
길이 통일
Padding / Crop / Sliding window
↓
특징 변환
Spectrogram / Mel-spectrogram / MFCC
↓
모델 입력 Tensor

-> 소리를 숫자로 표현

-> 같은 시간 길이를 같은 sample 수로 표현

-> 모델 입력 Channel 수를 일관되게 유지

-> 여러 데이터를 하나의 Batch로 구성

-> 시간과 주파수 패턴을 모델이 쉽게 학습하도록 변환

전처리

특징 추출

소리 → 숫자 변환

공기 압력의 변화



마이크가 전기 신호로 변환



일정 시간 간격으로 측정



진폭을 숫자로 저장

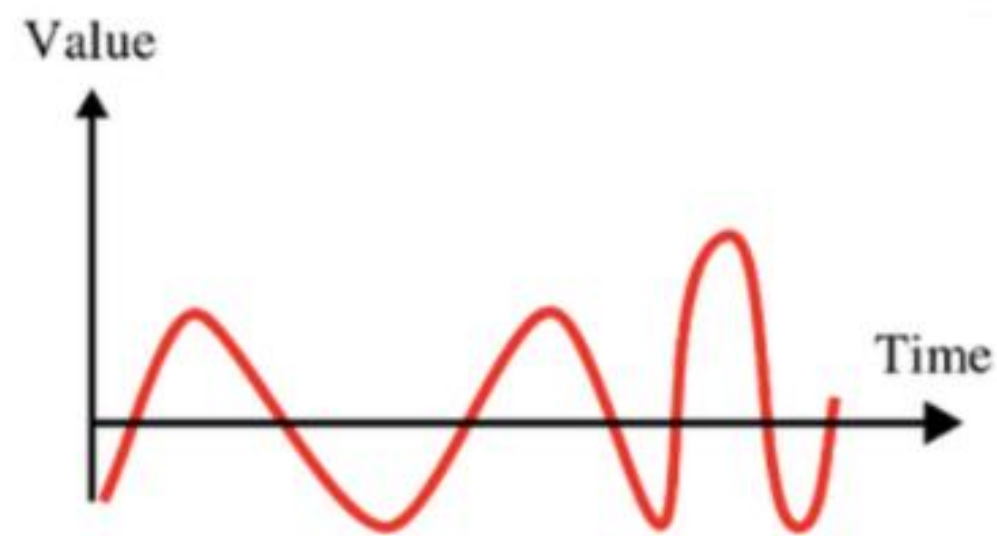


Digital Waveform

실제 소리는 공기 압력이 시간에 따라 변화하는 연속적인 물리 현상입니다. 마이크는 이 압력 변화를 전기 신호로 바꾸고, 오디오 장치는 일정한 시간 간격으로 신호의 값을 측정합니다. 이 과정을 Sampling이라고 합니다.

측정된 값은 무한한 정밀도로 저장할 수 없으므로 일정한 단계의 숫자로 표현합니다. 이를 Quantization이라고 합니다. Bit depth는 진폭을 얼마나 세밀하게 구분할 수 있는지를 나타내지만, 이번 수업에서는 시간축을 얼마나 자주 측정하는지를 나타내는 Sampling rate에 더 집중합니다.

Waveform 값은 실제 소리의 의미를 직접 나타내는 값이 아니라 특정 시점에서 측정된 진폭입니다. 모델은 이러한 값이 시간에 따라 변화하는 패턴을 학습합니다.



a. Analog signal

연속적인 곡선



b. Digital signal

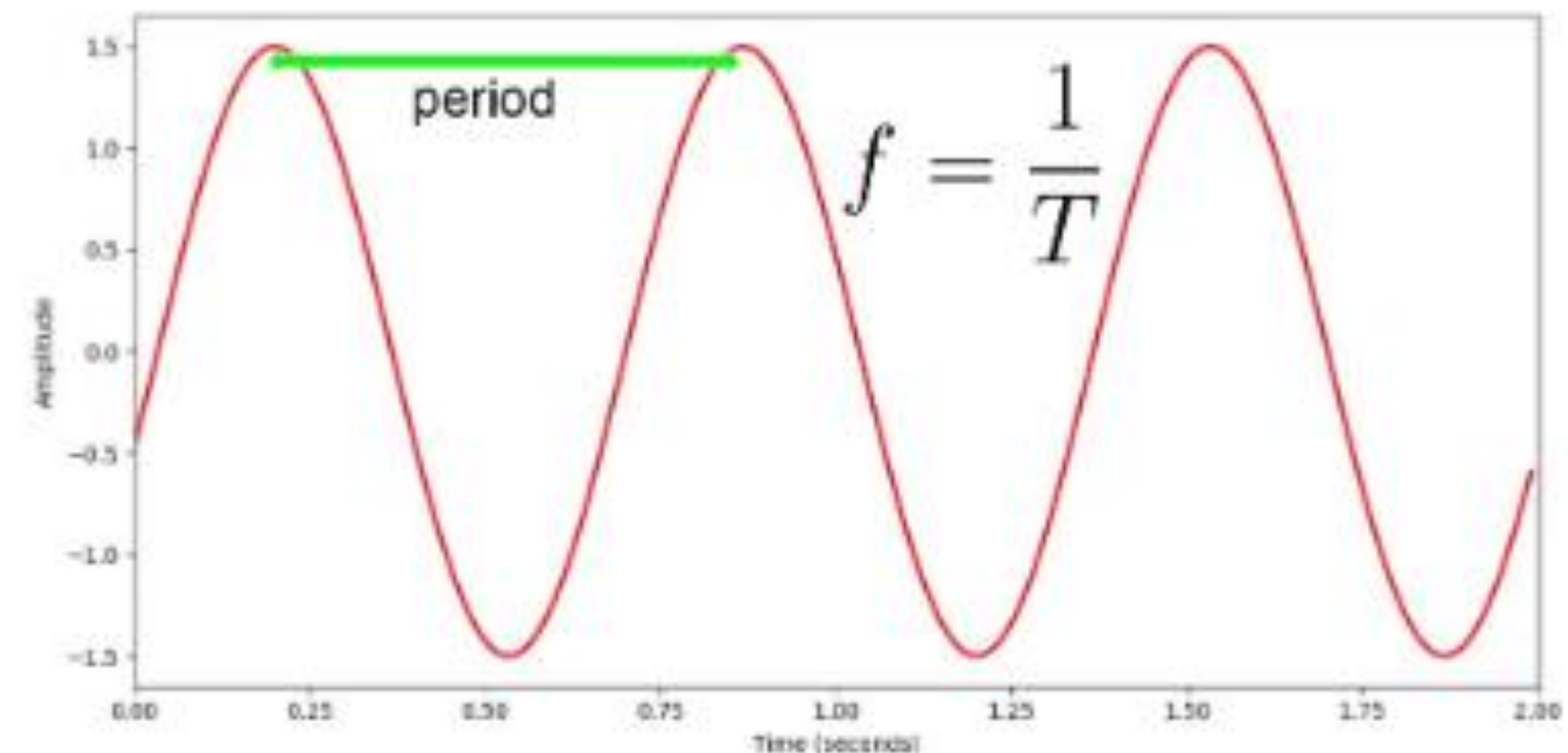
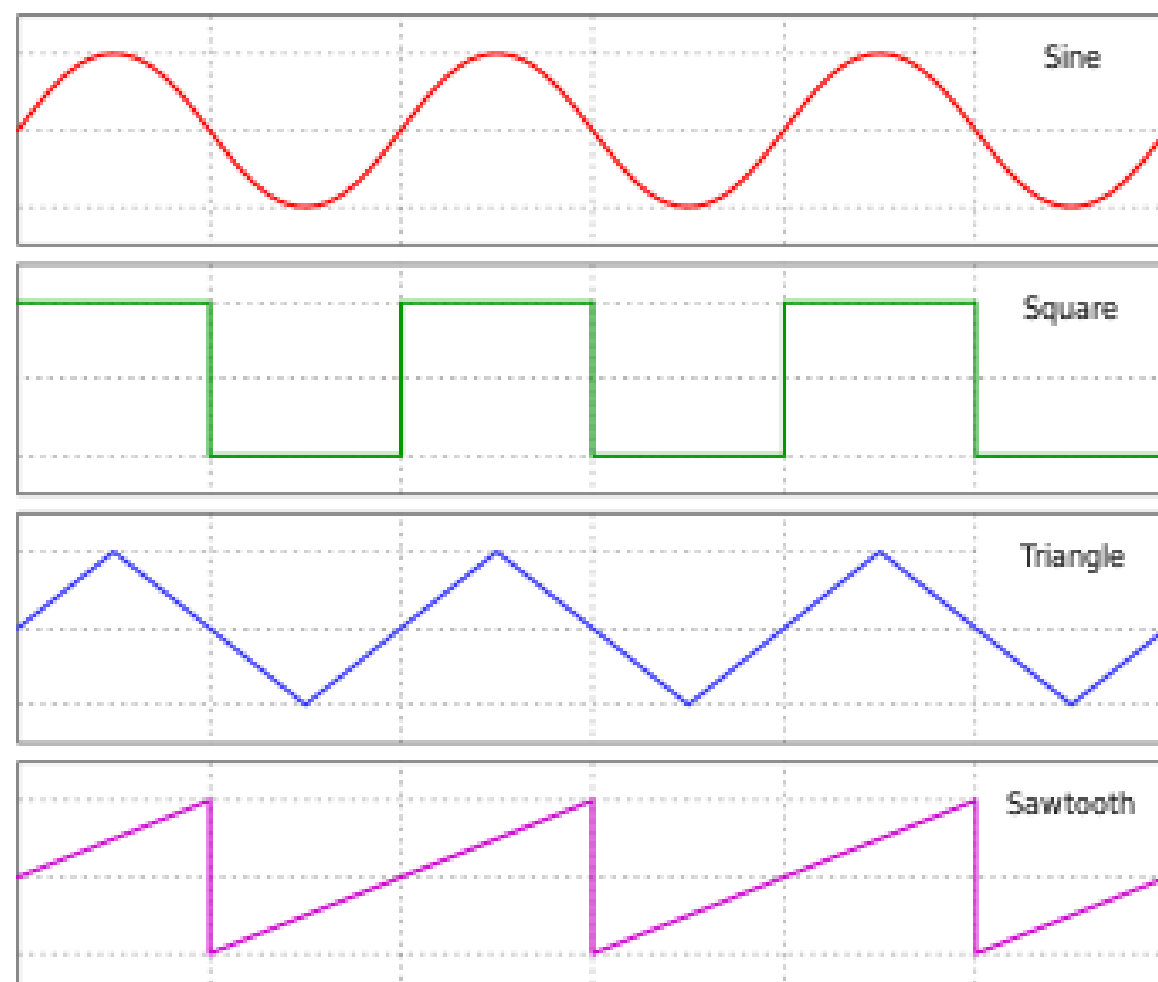
측정된 진폭을 일정한 숫자 단계로 표현

Sampling : 시간축을 일정 간격으로 측정

Quantization : 측정된 진폭을 일정한 숫자 단계로 표현

Waveform (파형)

- Waveform은 특정 시간에서 소리 신호의 진폭이 얼마인지를 순서대로 저장한 1차원 시계열 데이터입니다. 가로축은 시간이고 세로축은 진폭(Amplitude)입니다.
- 특정 지점에서 다음 등장하는 그 값까지를 period(주기)라고 정의한다. 이는 frequency(주파수) 개념으로 확장된다. Frequency는 Hz 단위를 사용하며, 1초에 100번 period(주기)가 발생하는, 즉 100번 진동하는 소리를 100Hz로 정의한다. 그래서 $f = 1/T$ 라는 수식이 성립한다.



Sampling rate와 Duration

- Sampling rate는 오디오 파일의 전체 Sample 수가 아니라 1초당 측정 횟수입니다. 같은 1초 오디오라도 Sampling rate가 다르면 Sample 수가 달라집니다. Duration은 총 시간과 같습니다.

Sampling rate	의미
8,000 Hz	1초에 8,000번 측정
16,000 Hz	1초에 16,000번 측정
44,100 Hz	1초에 44,100번 측정
48,000 Hz	1초에 48,000번 측정

$$\text{Duration} = \frac{\text{Number of samples}}{\text{Sampling rate}}$$

$$\text{Number of samples} = \text{Duration} \times \text{Sampling rate}$$

16 kHz, 1초
→ 16,000 samples

16 kHz, 2초
→ 32,000 samples

Sample 수: 48,000
Sampling rate: 16,000 Hz

Duration = 48,000 / 16,000 = 3초

Sampling rate가 높다고 항상 모델 성능이 좋아지는 것은 아닙니다. 분석하려는 소리의 주파수 범위와 사용할 모델, 계산 자원을 고려하여 결정해야 합니다.

Mono와 Stereo

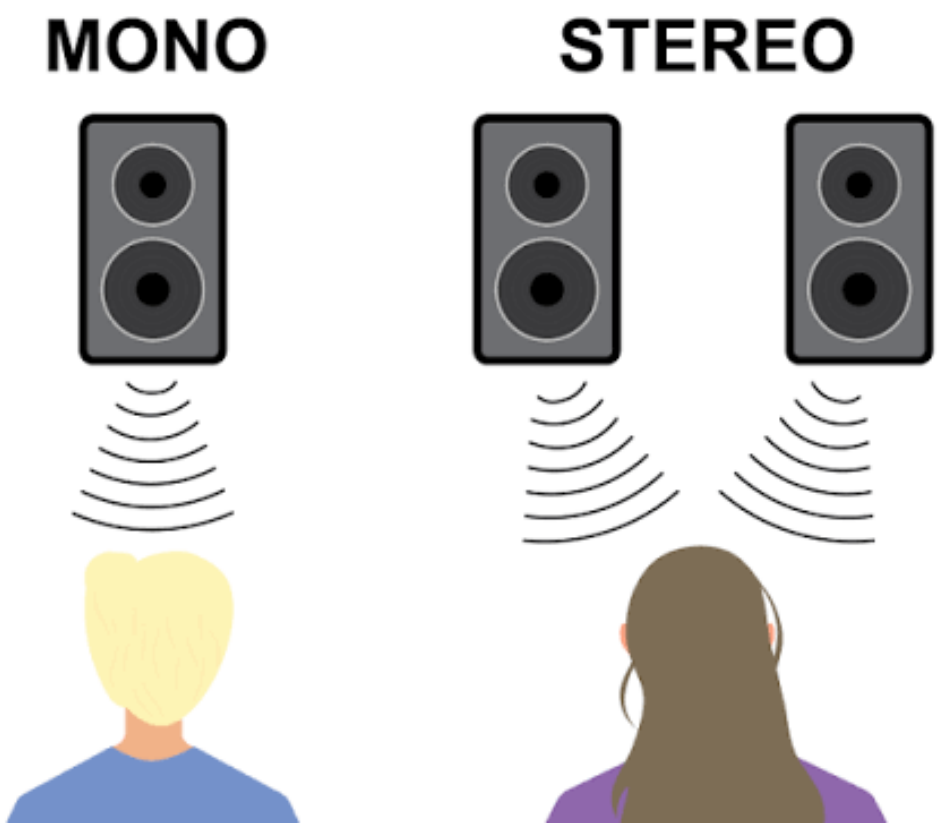
구분	Mono	Stereo
Channel 수	1	2
공간 정보	제한적	좌우 차이를 포함
데이터 크기	상대적으로 작음	상대적으로 큼
모델 입력	단순함	Channel 처리 필요

한 시점의 소리를 몇 개의 신호로 기록하는가? -> Mono & Stereo

Stereo는 두 신호 사이에

- 진폭 차이
- 도착 시간 차이
- 위상 차이
- 소리의 방향과 공간감

등이 존재할 수 있음.



오디오 전처리

딥러닝에서는 여러 데이터를 하나의 Batch로 묶어 동시에 계산합니다. 그러나 각 오디오의 Sample 수가 다르면 하나의 일정한 크기의 Tensor로 구성하기 어렵습니다. 이때, Padding, Crop, Sliding window 등 전처리 기법을 통해 동일한 크기의 Tensor로 변환이 필요합니다.

아래와 같은 길이가 다른 오디오 4개가 존재한다 하였을때,

오디오 A: 0.7초
오디오 B: 1.0초
오디오 C: 2.4초
오디오 D: 8.0초

16 kHz 기준으로 Sample수를 확인해보면, 왼쪽과 같기에 오른쪽과 같은 전처리가 필요합니다. (Padding은 겹치지 않는 값으로 길이를 늘려줌)

0.7초 → 11,200 samples
1.0초 → 16,000 samples
2.4초 → 38,400 samples
8.0초 → 128,000 samples

짧은 오디오 → Padding
긴 오디오 → Crop
매우 긴 오디오 → Sliding window

오디오 전처리 : Crop

Crop은 목표 길이보다 긴 오디오에서 일부 구간만 선택하는 방법입니다.

Crop의 가장 큰 위험은 중요한 소리를 제거할 수 있다는 점입니다.

예시)

전체 오디오

|----- 10초 -----|

Center crop

|----- 3초 -----|

Random crop

|----- 3초 -----|

Event-centered crop

|-- 이벤트 중심 3초 --|

방법들)

방법	특징
앞부분 Crop	구현이 단순하지만 뒤쪽 이벤트를 놓칠 수 있음
Center crop	평가 결과가 일정하지만 이벤트가 중앙에 없을 수 있음
Random crop	다양한 구간을 학습해 증강 효과를 얻을 수 있음
Event-centered crop	이벤트 위치 정보가 있을 때 효과적

오디오 전처리 : Sliding window

Sliding window는 긴 오디오 전체를 한 번에 처리하지 않고 일정한 길이의 여러 구간으로 나누는 방법입니다.

$$\text{Overlap ratio} = 1 - \frac{H}{L}$$

예시)

긴 오디오

|-----|

Window 1

|-----|

Window 2

|-----|

Window 3

|-----|

Window length L : 한 구간의 길이

Hop length H : 다음 구간으로 이동하는 거리

Overlap: 연속 Window가 겹치는 정도

Window length: 2초

Hop length: 1초

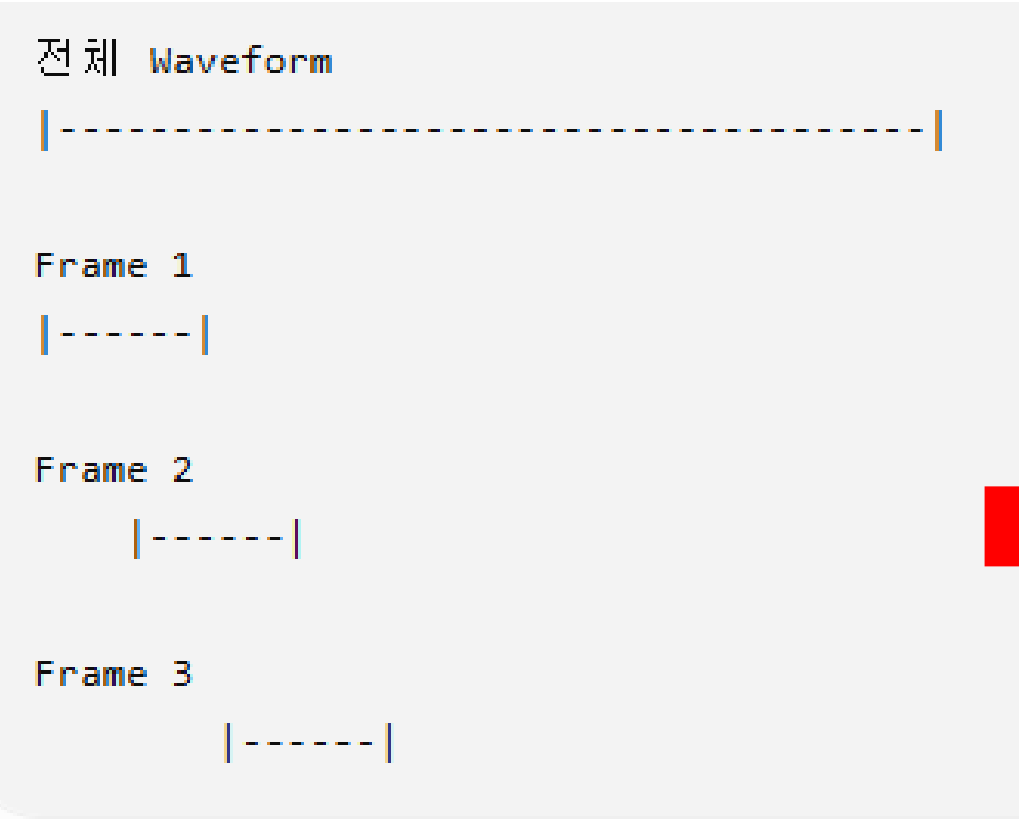
→ 50% overlap

Frame, Window, Hop length

- 실제 오디오는 시간이 흐르면서 주파수 특성이 계속 변합니다. 따라서 전체 오디오를 한 번에 분석하지 않고, 매우 짧은 Frame으로 나누어 각 Frame의 주파수를 계산합니다.
- Window가 짧으면 소리가 시간에 따라 빠르게 변화하는 모습을 세밀하게 볼 수 있습니다. 하지만 한 Frame에 포함되는 신호가 적어 주파수를 세밀하게 구분하기 어렵습니다.
- Window가 길면 가까운 주파수를 더 잘 구분할 수 있지만, 특정 소리가 정확히 언제 발생했는지에 대한 시간 해상도는 낮아집니다.

1. Frame은 분석할 짧은 시간 구간
2. Window는 Frame의 경계를 부드럽게 처리
3. Hop은 다음 Frame으로 이동하는 간격

예시)



16 Khz 오디오 예시

```
win_length = 400 samples  
400 / 16000 = 0.025초 = 25 ms  
  
hop_length = 160 samples  
160 / 16000 = 0.010초 = 10 ms
```

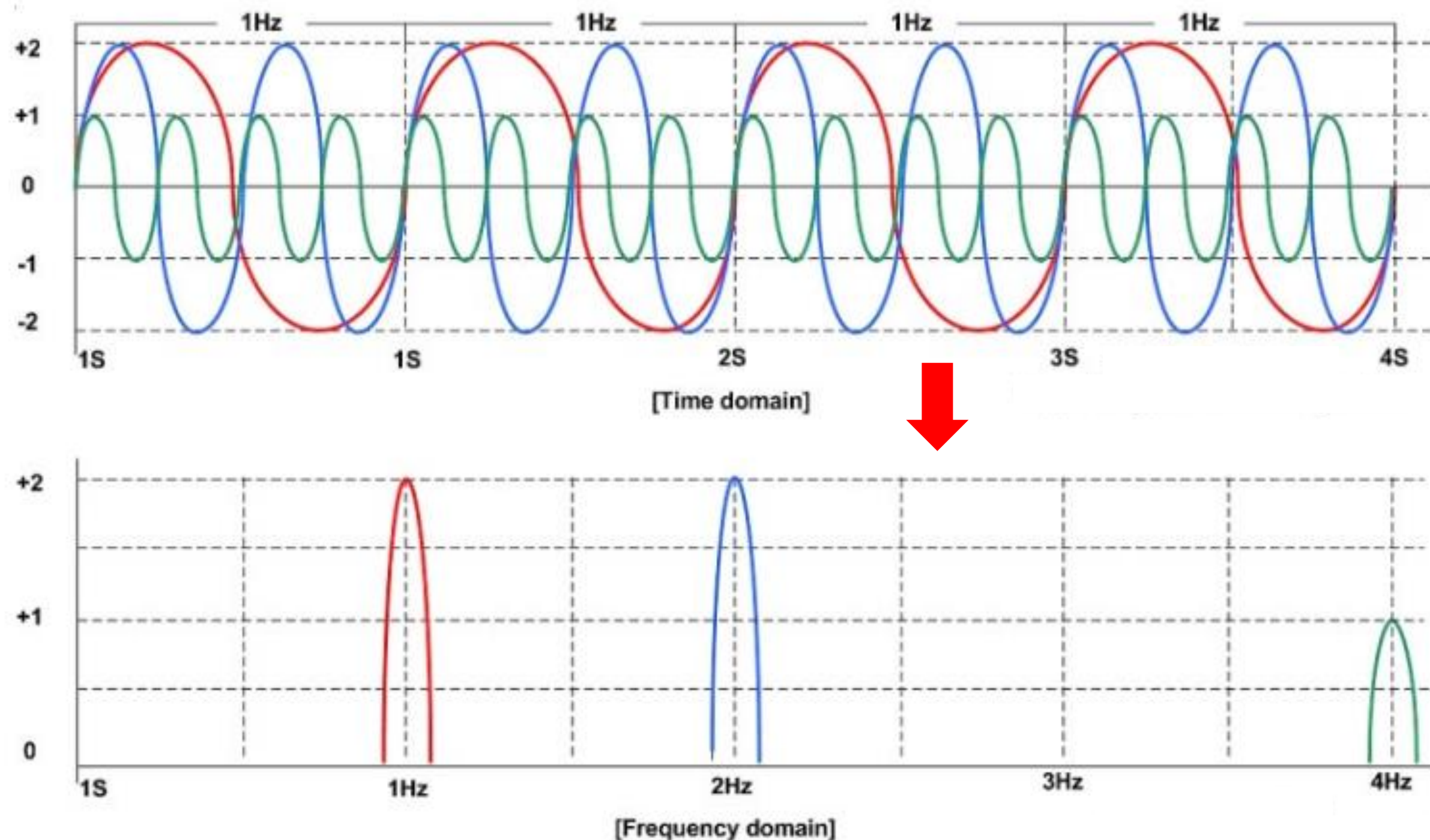
시간-주파수 해상도의 Trade-off 관계

Window 크기	시간 해상도	주파수 해상도
짧음	높음	낮음
길음	낮음	높음

시간 영역과 주파수 영역

시간 영역에서는 신호가 언제 강해지고 약해지는지 볼 수 있습니다. 그러나 여러 주파수가 섞인 실제 소리에서는 어떤 주파수 성분이 포함되어 있는지 Waveform만 보고 판단하기 어렵습니다.

구분	시간 영역	주파수 영역
주요 표현	Waveform	Spectrum
가로축	시간	주파수
확인하기 쉬운 정보	시작 시점, 길이, 진폭 변화	어떤 주파수가 강한지
한계	주파수 구성 파악 어려움	주파수가 언제 나타났는지 알기 어려움



Fourier transform은 복잡한 신호를 여러 개의 단순한 주파수 성분으로 분해합니다. 그 결과 어떤 주파수가 얼마나 강하게 포함되어 있는지를 확인할 수 있습니다.

⇒ 그러나 시간 정보가 사라질 수 도 있음

⇒ 이를 STFT를 이용하여 해결

STFT와 Spectrogram

STFT (Short-Time Fourier Transform)

- 긴 오디오를 짧은 Frame으로 나누고, 각 Frame에 Fourier transform을 적용
- 시간과 주파수를 동시에 표현할 수 있음

가로축 : Time frame, 세로축 : Frequency, 색 or 밝기 : Magnitude or Power

아래는 duration 4초, sampling rate 16kHz 를 적용한 64000sample를 128개의 주파수로 쪼갠 결과 (총 1001 frame)

Waveform



짧은 Frame으로 분할



각 Frame에 Window 적용



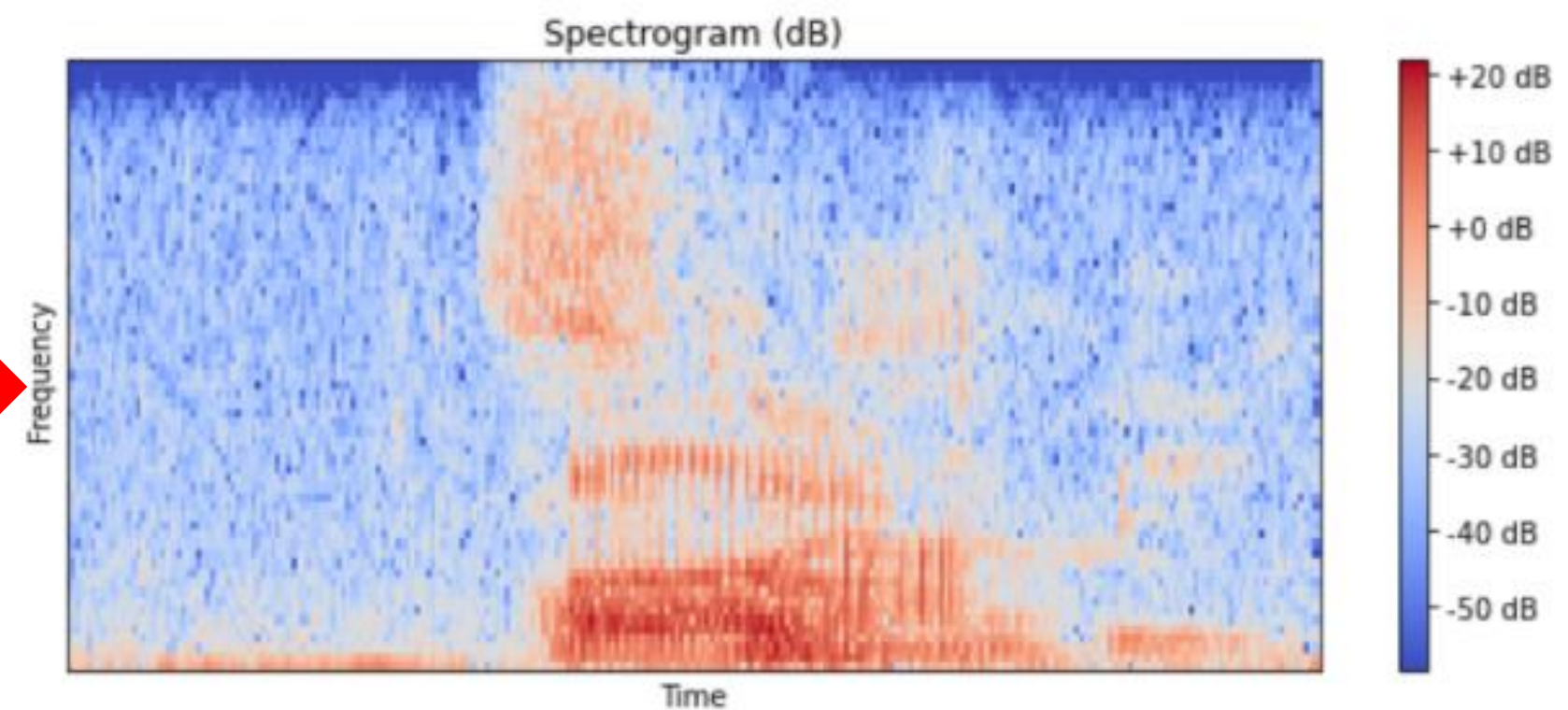
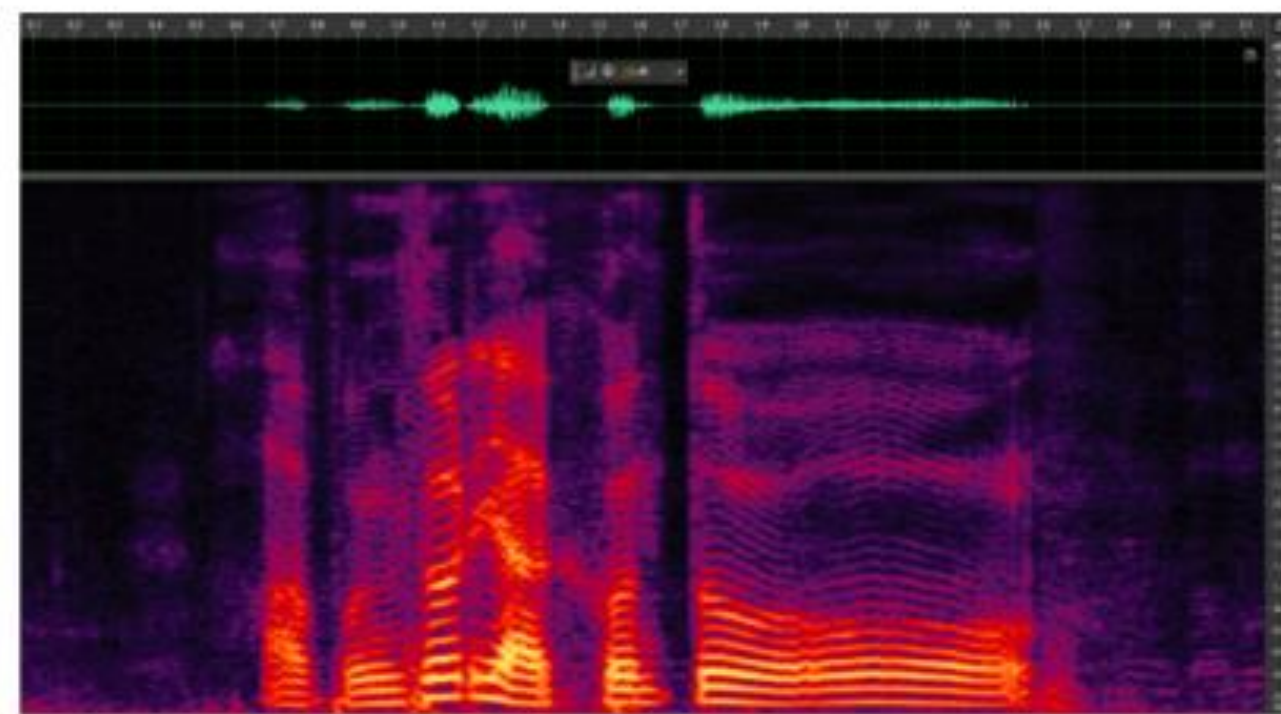
각 Frame에 Fourier transform



시간별 주파수 성분 생성



Spectrogram

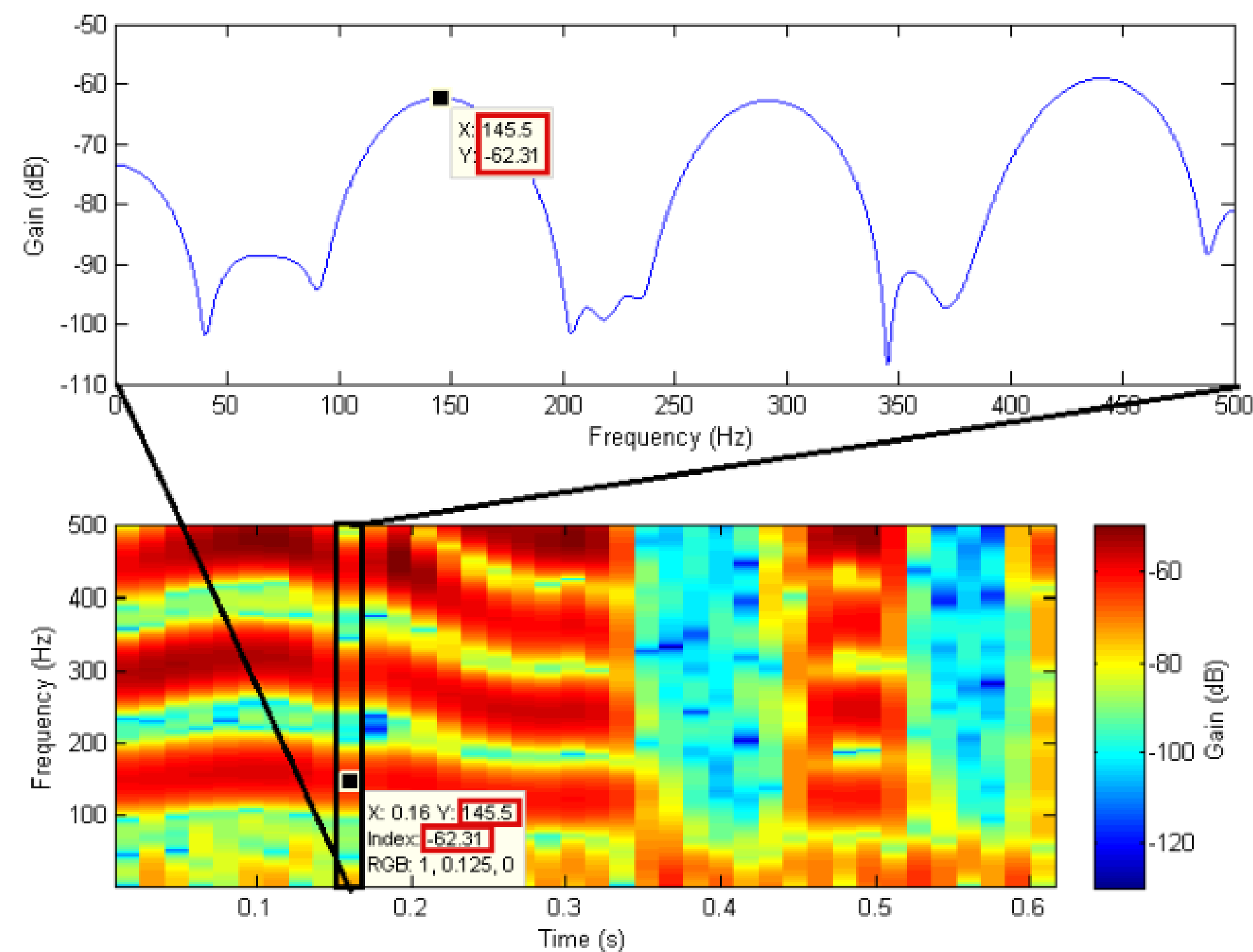


Spectrogram

Spectrogram은 시간에 따른 신호의 주파수 스펙트럼 변화를 나타낸 2차원 이미지이다.

스펙트럼이 특정 순간의 주파수 성분과 그 진폭만을 보여주는 1차원적인 정보라면,

Spectrogram은 여기에 시간축이 추가되어 각 시간대별로 그 순간의 스펙트럼을 보여주는 2차원적인 정보를 제공한다.



Mel-spectrogram, MFCC

멜 스펙트로그램(Mel Spectrogram)

오디오 신호의 스펙트로그램을 멜 척도로 변환한 것이다. 여기서 멜 척도란 실제 주파수를 **인간의 청각 인식에 더 가깝게** 매핑하는 비선형 척도이다. 인간의 청각은 높은 주파수에서 넓은 주파수 간격을 더 넓게 인식하는 특성이 있는데, 쉽게 말해 낮은 주파수에서는 미세한 차이를 잘 감지하지만 높은 주파수에서는 덜 민감하게 인식한다는 것이다. 멜 척도는 이를 반영하여 낮은 주파수(약 20 ~ 500Hz라고 한다)에서 더 세밀한 해상도를 제공하고, 높은 주파수에서는 주파수 간격을 넓게 설정하여 상대적으로 낮은 해상도를 제공한다.

MFCC(Mel-Frequency Cepstral Coefficients)

멜 스펙트럼(Mel Spectrum)을 기반으로 하는 **오디오 처리 기법**이다. 오디오 신호의 짧은 구간(프레임)에 대한 멜 스펙트로그램을 사용하여 DCT(Discrete Cosine Transform)를 통해 더욱 압축한 형태로 변환하는데, 이 과정을 통해 중요한 특성을 추출하게 된다. 결과적으로 MFCC를 통해 오디오 데이터의 feature를 뽑아 벡터화할 수 있게 된다. MFCC는 음성 인식에서 굉장히 유용하게 쓰인다.

- Mel: 사람의 주파수 지각 특성을 반영
- Frequency: 주파수 정보를 사용
- Cepstral: Log spectrum을 다시 변환한 표현
- Coefficient: 변환 결과로 얻은 계수

Mini Quiz

1. 16 kHz로 저장된 2초 오디오는 몇 개의 Sample을 가지는가?
2. Spectrogram에서 가로축과 세로축은 각각 무엇인가?

8

오디오 딥러닝과 응용 모델

음성 데이터를 딥러닝 모델에 적용하고, 다양한 데이터 유형과 연결하는 방법 이해

학습목표

1. Log-Mel spectrogram이 CNN에 입력되는 과정을 이해한다.
2. 오디오 데이터 증강의 목적과 종류를 이해한다.
3. Wave2Vec2와 AST의 기본 구조를 구분한다.
4. 멀티모달 학습에 대해 이해한다.

오디오 딥러닝 종류

문제 유형	입력	출력 예시
Audio classification	오디오 구간	개, 자동차, 비, 천둥
Sound event detection	긴 오디오	이벤트 종류와 발생 시간
Speech recognition	음성	발화 내용 또는 문장
Audio regression	오디오	소음 크기, 감정 강도, 품질 점수

Single-label classification

한 오디오에 하나의 정답

예 :

yes / no / stop / go 중 하나

Multi-label classification

한 오디오에 여러 정답

예 :

rain + thunder + wind

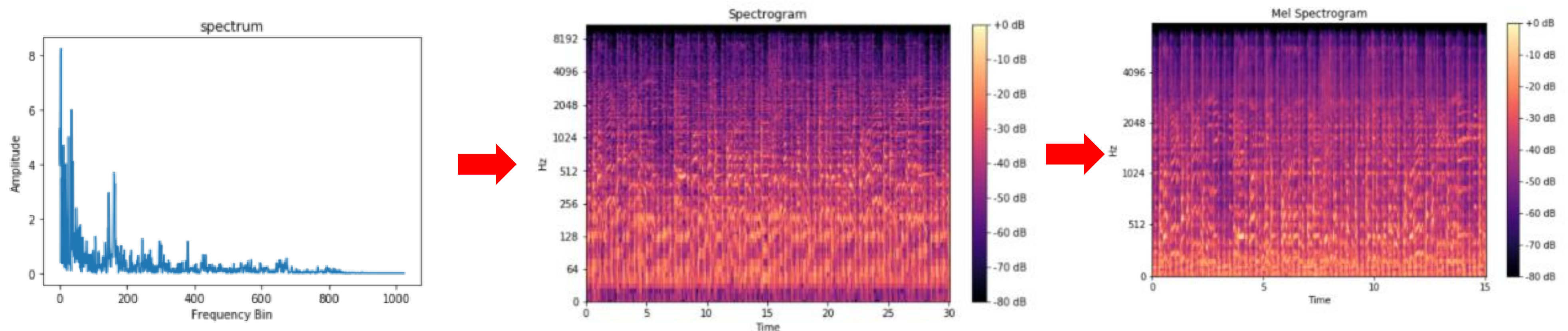
- 같은 CNN을 사용하더라도 한 오디오에 하나의 정답만 존재하는지, 여러 소리가 동시에 존재할 수 있는지에 따라 출력층과 손실함수가 달라집니다.
- Audio **classification**은 전체 파일에 하나 또는 여러 개의 Label을 예측합니다. Sound event **detection**은 어떤 소리가 포함되어 있는지뿐 아니라 언제 시작하고 끝났는지까지 예측합니다.

Audio CNN

1. Log-Mel spectrogram

- 오디오 신호를 딥러닝 모델이나 음성 인식 시스템에 입력하기 위해 시각적 이미지 형태로 변환한 데이터입니다. 사람의 귀가 소리를 인지하는 방식(저주파에 민감, 진폭은 로그 단위로 인식)을 주파수와 진폭 축에 모두 반영하여 특징을 극대화한 것이 핵심

표현	가로축	세로축	쉽게 볼 수 있는 정보
Waveform	시간	진폭	시작·종료, 무음, 전체 크기
Log-Mel spectrogram	시간	Mel 주파수	주파수 구성과 시간에 따른 변화



Audio CNN

Mel Spectrogram으로 전처리된 오디오는 2차원 데이터이기에 이미지와 비슷하게 다룰 수 있다.
(사이렌, 엔진, 비소리, 강아지 짖는소리가 섞인 음성데이터를 분류한다는 가정)

모델 및 결과 예시)

Log-Mel spectrogram

[B, 1, 64, 100]

↓

Conv2D + ReLU

[B, 16, 64, 100]

↓

Pooling

[B, 16, 32, 50]

↓

Conv2D + ReLU

[B, 32, 32, 50]

↓

Pooling

[B, 32, 16, 25]

↓

Global Average Pooling

[B, 32]

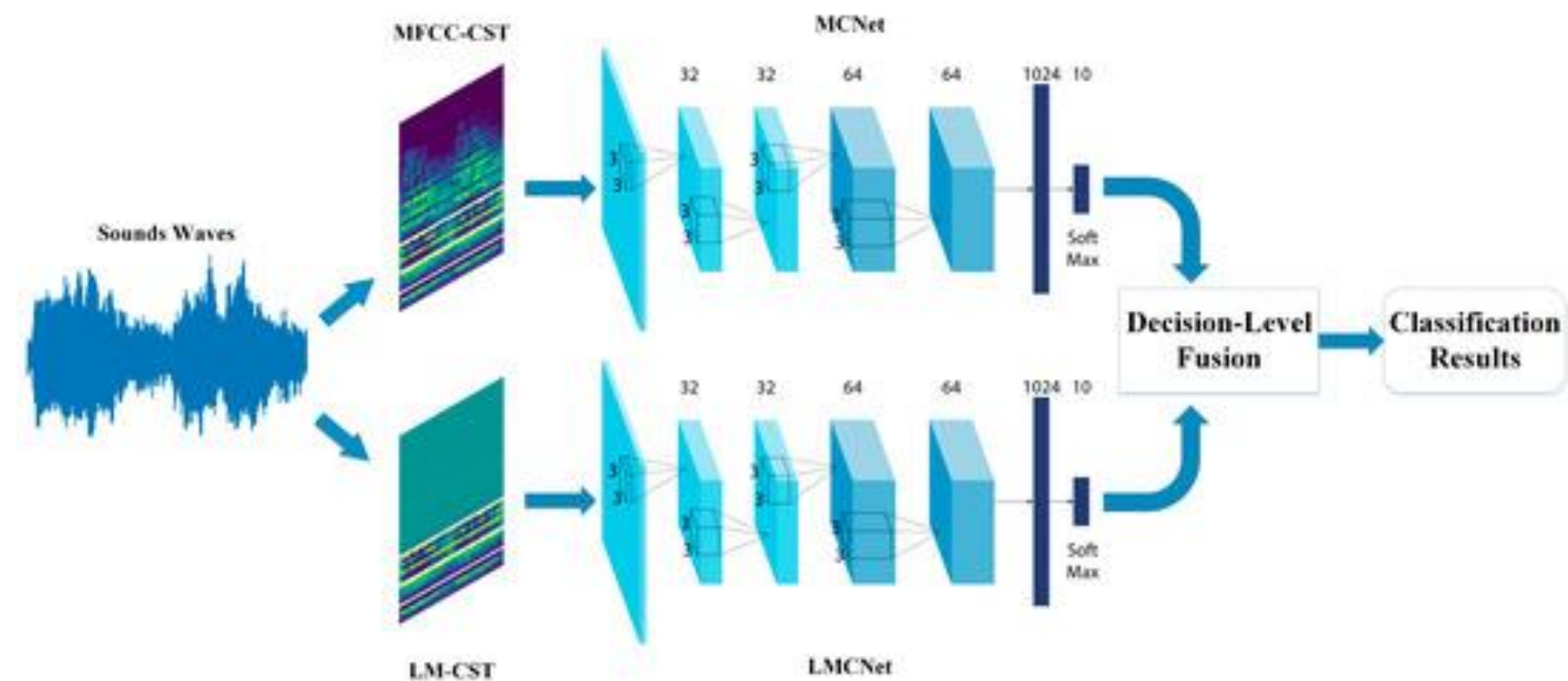
↓

Linear Layer

[B, K]

↓

Class prediction



구성요소	역할
Convolution	시간-주파수 특징 추출
Activation	비선형 패턴 표현
Pooling	크기와 계산량 감소
Global Average Pooling	전체 Feature map 요약
Linear layer	클래스별 점수 출력

siren	0.82
engine	0.11
rain	0.05
dog bark	0.02

Audio 증강

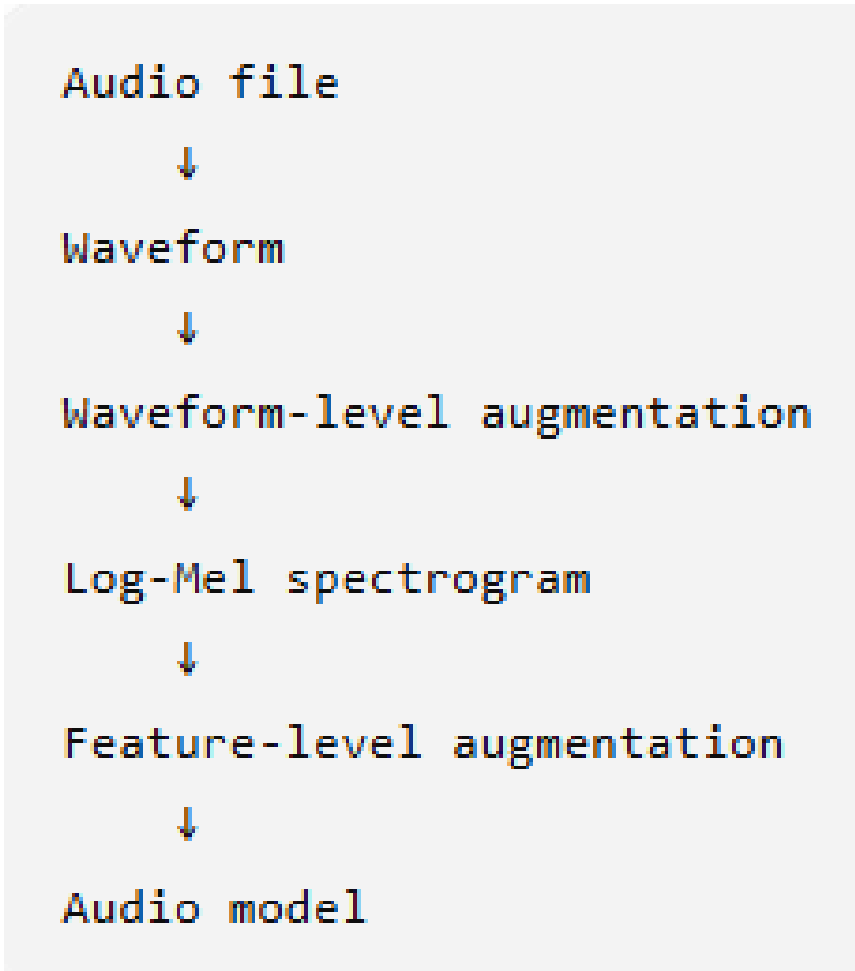
- 동일한 사람이 동일한 단어를 말하더라도 녹음 장비, 마이크와의 거리, 배경 소음과 공간의 울림에 따라 Waveform과 Spectrogram은 달라집니다.
- 모델이 조용한 환경의 음성만 학습하면 단어 자체의 특징뿐 아니라 깨끗한 녹음 조건까지 함께 학습할 수 있습니다. 이후 실제 환경에서 잡음이 포함된 음성이 들어오면 같은 단어임에도 새로운 종류의 입력으로 판단할 수 있습니다.
- 이러한 문제를 완화하기 위해 학습 데이터에 다양한 변형을 적용하여 모델이 여러 녹음 조건을 경험하게 합니다. 이를 Audio augmentation이라고 합니다.

Waveform-level augmentation

증강 방법	변화시키는 요소	학습시키려는 특성	주요 주의점
Random gain	음량	음량이 달라도 같은 소리	Clipping 가능
Time shift	발생 위치	시작 위치가 달라도 같은 소리	경계 처리
Random crop	선택 구간	일부 구간 변화	이벤트가 제거될 수 있음
Speed perturbation	속도와 길이	발화 속도 변화	의미 손상 가능
Pitch shift	음높이	화자 음높이 차이	클래스 특징 변화 가능
Reverberation	공간의 울림	다른 방과 거리	과도한 반향
Noise addition	배경 소리	다양한 소음 환경	강도 조절 필요

Feature-level augmentation

- Time masking
-> 일부 시간 masking
- Frequency masking
-> 일부 주파수 masking



Audio 증강

Noise Augmentation

- Noise augmentation은 실제 배경 Noise를 원래 오디오에 추가하여 다양한 녹음 환경을 모사하는 방법입니다.
- 중요한 부분은 Noise의 종류와 크기입니다. 실제 적용 환경과 관련이 없는 Noise만 사용하면 현실적인 일반화 성능을 얻기 어렵습니다.
- 산림 환경에서 오디오를 분석한다면 바람, 비, 곤충, 차량과 센서 잡음이 현실적인 Noise가 될 수 있습니다. 실내 음성 인식이라면 에어컨, 키보드, 사람들의 대화와 같은 Noise가 적절할 수 있습니다.

“최대한 원본 음성을 방해하지 않는 선에서 Noise 증강 적용”

noise 증강 수식

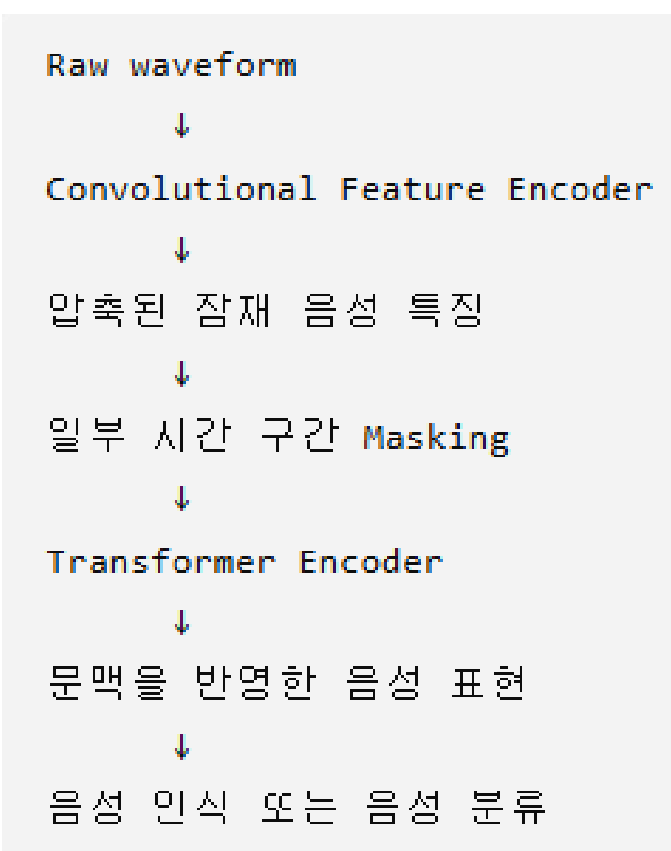
$$x_{\text{noisy}}(t) = s(t) + \alpha n(t)$$

기호	의미
$s(t)$	원래 신호
$n(t)$	배경 Noise
α	Noise 크기를 조절하는 계수
$x_{\text{noisy}}(t)$	혼합된 최종 신호

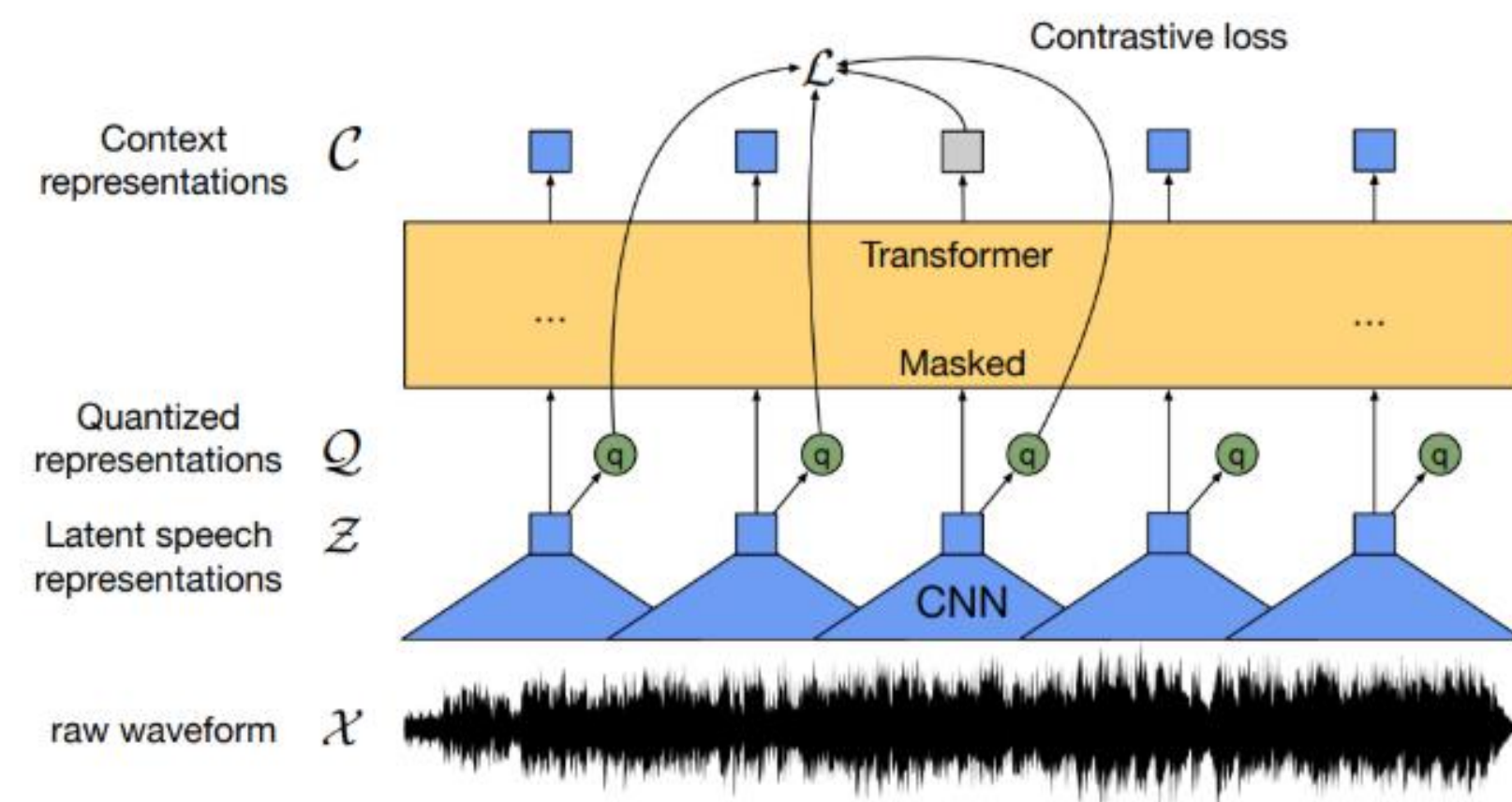
Wav2Vec2

“정답 문장이 없는 대규모 음성 데이터에서 일부 구간을 가리고, 주변 문맥을 이용해 해당 구간의 올바른 음성 표현을 찾도록 학습한다.”

- 2020 Meta에서 발표한 음성인식 모델
- wav2vec 2.0은 Raw waveform에서 일반적인 음성 표현을 먼저 학습한 뒤, 새로운 음성 문제에 Fine-tuning하는 모델이다.



구성요소	역할
Raw waveform	모델에 입력되는 원시 음성 신호
CNN Feature Encoder	긴 Waveform을 짧은 잠재 특징 Sequence로 압축
Masking	일부 시간 구간의 특징을 가려 학습 문제 생성
Transformer	앞뒤 음성 정보를 이용해 전체 문맥 표현 생성
Contrastive Learning	올바른 음성 표현과 잘못된 후보를 구분하도록 학습



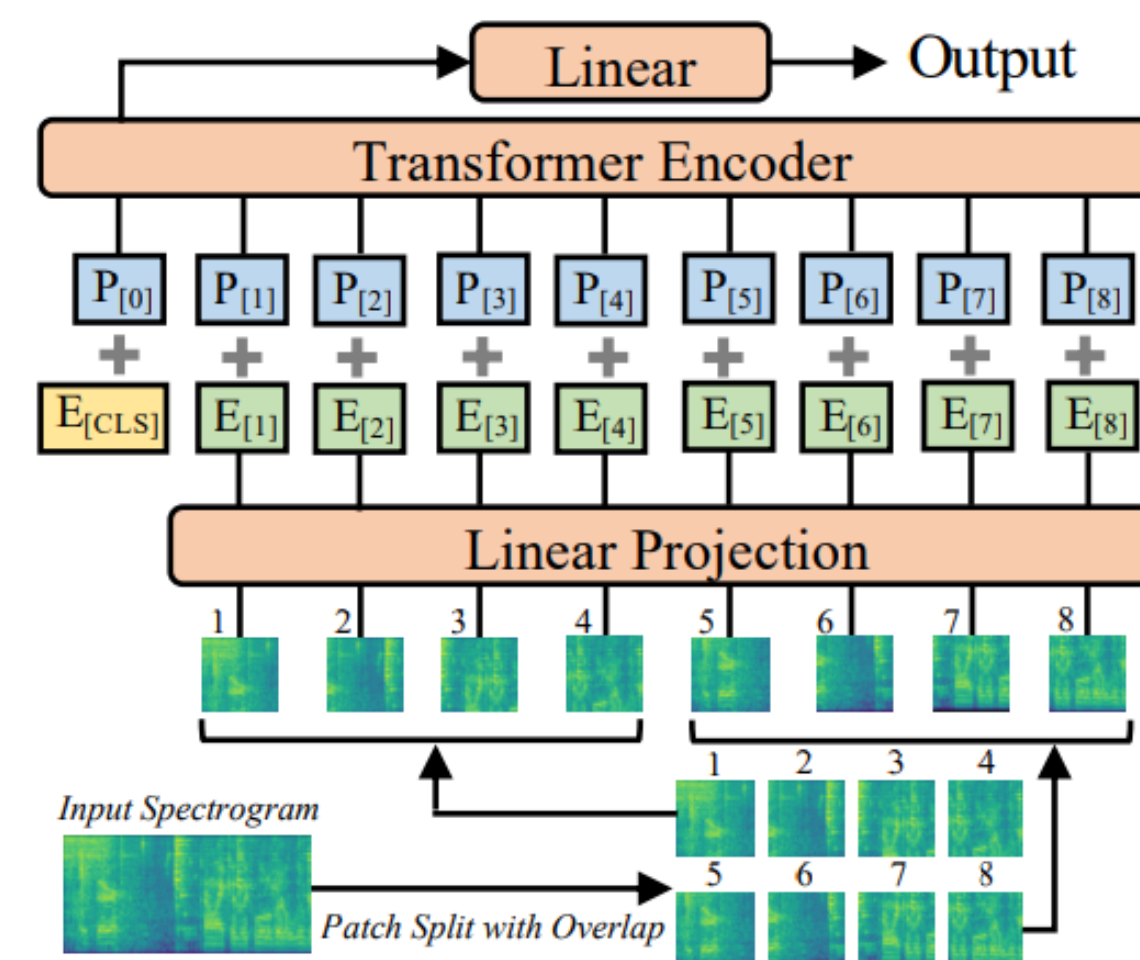
AST (Audio Spectrogram Transformer)

“Log-Mel spectrogram을 작은 시간-주파수 Patch로 나누고, 각 Patch를 하나의 Token처럼 Transformer에 입력한다.”

- 기존에 audio는 대부분 CNN을 이용한 이미지와 비슷한 처리를 진행하였으나, CNN을 제외하고 Attention만을 활용한 더 뛰어난 모델

Log-Mel spectrogram
[Frequency × Time]
↓
작은 Patch로 분할
↓
Patch를 1차원 Vector로 변환
↓
Patch Embedding
+ Positional Embedding
↓
Transformer Encoder
↓
Classification Head
↓
Audio Class

구성요소	역할
Log-Mel spectrogram	시간과 주파수별 에너지 표현
Patch 분할	Spectrogram을 작은 시간-주파수 영역으로 나눔
Patch embedding	각 Patch를 Transformer 입력 벡터로 변환
Positional embedding	Patch의 시간 및 주파수 위치 정보 제공
Self-Attention	서로 떨어진 Patch 사이의 관계 학습
Classification head	최종 오디오 클래스 예측



이미지 & 텍스트 & 오디오 연결

“형태가 다른 세 종류의 데이터는 결국 숫자로 표현되기에 연결될 수 있다.”

데이터 유형	원래 데이터	숫자 표현	대표 모델
이미지	사진	Pixel tensor	CNN
텍스트	문장	TF-IDF, Token ID, Embedding	LSTM, BERT
오디오	소리	Waveform, Log-Mel, MFCC	CNN, Wave2Vec2, AST

이미지에서는 CNN이 Pixel로부터 특징을 만들고, 텍스트에서는 BERT가 Token으로부터 문맥 표현을 만듭니다. 오디오에서는 CNN이 Spectrogram으로부터 시간-주파수 특징을 만들거나, Wave2Vec2가 Raw waveform으로부터 음성 표현을 만듭니다.

이 데이터들은 결국 **모델 내부에서 숫자로 변환**되기에 다양한 방법으로 합치할 수 있다.

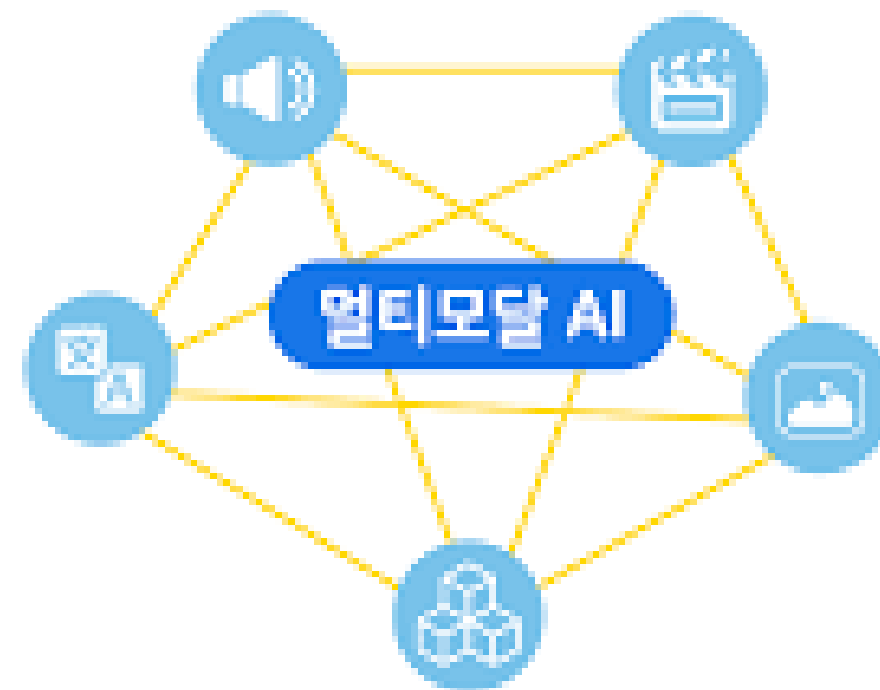
멀티모달 (Multimodal) 딥러닝

1. 모달리티(Modality)란?

- 우리가 세상을 인식하는 정보의 종류나 형태를 의미합니다. (예: 시각, 청각, 텍스트, 촉각 등)
- 지금까지 우리는 주로 단일 모달(Single-modal), 즉 **한 종류의 데이터**만 다루었습니다. (예: 이미지만으로 분류, 텍스트만으로 감성 분석)

2. 멀티모달(Multimodal) 딥러닝이란?

- 두 가지 이상의 모달리티 데이터를 함께 사용하여, 단일 모달리티만 사용했을 때보다 더 높은 수준의 이해와 추론을 목표로 하는 인공지능 분야입니다.
- **핵심**: 사람은 이미지를 보면서 동시에 설명을 듣고, 소리를 들으며 자막을 읽습니다. 이처럼 여러 감각을 함께 사용하는 **인간의 학습 방식을 모방**하는 것입니다.



멀티모달 (Multimodal) 딥러닝

3. 모달리티(Modality) 융합(Fusion) 종류

- 서로 다른 두 데이터를 합치는(Fusion) 데에는 대표적으로 세 가지 전략이 있습니다.

1) Early Fusion

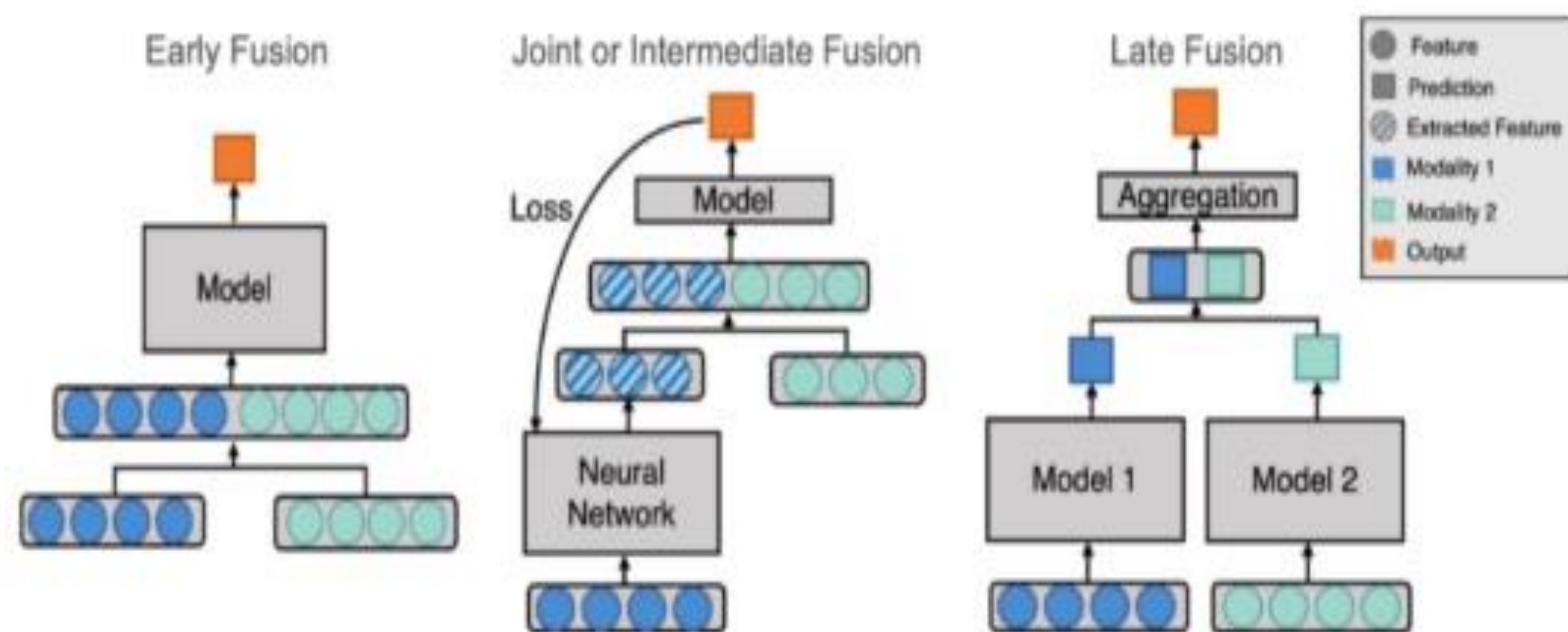
- 방법: 각 데이터를 전처리한 후, 모델에 입력하기 전 입력단에서 바로 합쳐버리는 방식입니다.

2) Late Fusion

- 방법: 각 데이터에 대해 독립적인 모델을 따로따로 통과시킨 후, 각 모델이 내놓은 최종 결과(예측값)를 합치는 방식입니다.

3) Joint or Intermediate Fusion

- 방법: 각 데이터가 별도의 인코더(Encoder)를 통과하며 특징(Feature)을 추출한 뒤, 모델의 중간 레이어에서 이 특징들을 합치는 방식입니다. (가장 많이 사용되는 방식)



멀티모달 (Multimodal) 딥러닝

4. 멀티모달 Task 이해

- 서로 다른 Image 데이터들을 Fusion하여 활용한 예시들입니다.

1) 의료 영상 융합 (Medical Image Fusion)

- 입력: CT/MRI (해부학적 구조) + PET (기능적/대사 활동)
- 출력: 두 정보가 결합된 진단용 융합 이미지
 - 목표: 암세포의 정확한 위치(CT/MRI)와 활성도(PET)를 동시에 파악하여 진단 및 치료 계획의 정확도를 높입니다.

2) 위성/항공 영상 융합 (Satellite/Aerial Image Fusion)

- 입력: 저해상도 다중분광(컬러) 이미지 + 고해상도 전정색(흑백) 이미지
- 출력: 고해상도 다중분광(컬러) 융합 이미지 (Pansharpening)
- 목표: 각기 다른 장점을 가진 위성 센서 정보를 합쳐, 색상과 해상도를 모두 만족하는 고품질 이미지를 생성합니다.

3) RGB-D 이미지 융합 (RGB + Depth Image Fusion)

- 입력: RGB 컬러 이미지 + Depth (깊이) 이미지
- 출력: 객체 분할 마스크, 3D 장면 재구성 등
- 목표: 컬러 정보만으로는 알기 어려운 객체의 3차원 형태와 거리를 깊이 정보로 보강하여, 더 정확한 장면 이해, 객체 분할, 자율주행 등에 활용합니다.

멀티모달 (Multimodal) 딥러닝

5. 멀티모달의 미래

- 패러다임의 전환: 과거에는 각 데이터(이미지, 텍스트)를 처리하는 모델을 따로 만들어 '융합'하는 방식이 일반적이었습니다.
- 현재와 미래: 이제는 처음부터 텍스트, 이미지, 오디오, 비디오 등 세상의 모든 데이터를 한 번에 학습하는 초거대 모델, 즉 LMM(Large Multimodal Model)이 대세가 되고 있습니다.
- 대표적인 모델: Google의 Gemini, OpenAI의 GPT-4V
 - 이 모델들은 단순히 데이터를 융합하는 것을 넘어, 여러 종류의 데이터를 넘나들며 하나의 대화 속에서 종합적으로 추론하고 소통합니다.
 - (예시: 냉장고 속 재료 사진을 보여주며 "이걸로 만들 수 있는 저녁 메뉴 추천해 줘"라고 질문하면, 이미지를 인식하고 레시피를 텍스트로 생성해 줍니다.)

멀티모달 (Multimodal) 딥러닝

6. 멀티모달의 Challenge들

문제	설명
시간 정렬	Audio, Video, Sensor의 기록 시간이 다를 수 있음
Sampling 간격	Audio는 초당 수천 개, 기상자료는 분·시간 단위
Missing modality	일부 시점에 영상이나 오디오가 없을 수 있음
품질 차이	한 Modality만 매우 강하거나 매우 noisy할 수 있음
Modality dominance	모델이 한 데이터 유형에만 의존할 수 있음
데이터 누수	같은 장소·세션이 Train과 Validation에 포함될 수 있음
계산량	여러 Encoder를 함께 사용하므로 비용 증가

이 외에도 다양한 해결해야할 문제들이 있으며, 유망한 분야임

Mini Quiz

다음 중 오디오 데이터 증강, wav2vec 2.0, AST에 대한 설명으로 가장 적절한 것은?

1. Noise augmentation은 Log-Mel spectrogram의 일부 주파수 영역을 제거하는 방법이며, SNR이 높을수록 잡음이 강하다.
2. wav2vec 2.0은 Log-Mel spectrogram을 Patch로 나누어 Transformer에 입력하고, AST는 Raw waveform의 일부 구간을 Masking하여 학습한다.
3. Audio augmentation은 실제 환경에서 발생할 수 있는 변화를 학습 데이터에 적용하는 방법이며, wav2vec 2.0은 주로 Raw waveform을, AST는 주로 Log-Mel spectrogram을 입력으로 사용한다.
4. Time masking과 Frequency masking은 Validation 데이터에 무작위로 적용하여 평가 데이터의 다양성을 높이는 것이 일반적이다.



STOP



**본콘텐츠는 한국지능정보사회진흥원(NIA)의 동의없이 무단 사용할 수 없으며
상업적 목적으로 이용을 금합니다.**

2026 데이터+AI 혁신 챌린지
데이터+AI 크리에이터 캠프



감사합니다



과학기술정보통신부

NIA 지능정보원
한국지능정보사회진흥원